

Kiểm thử phần mềm với các mô hình ngôn ngữ lớn: Khảo sát, Cảnh quan và Tầm nhìn

Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, Qing Wang

Tóm tắt—Các mô hình ngôn ngữ lớn được huấn luyện trước (LLM) gần đây đã nổi lên như một công nghệ đột phá trong xử lý ngôn ngữ tự nhiên và trí tuệ nhân tạo, với khả năng xử lý các tập dữ liệu quy mô lớn và thể hiện hiệu suất đáng kể trên nhiều nhiệm vụ khác nhau. Trong khi đó, kiểm thử phần mềm là một nhiệm vụ quan trọng, đóng vai trò nền tảng để đảm bảo chất lượng và độ tin cậy của các sản phẩm phần mềm. Khi phạm vi và độ phức tạp của các hệ thống phần mềm tiếp tục tăng lên, nhu cầu về các kỹ thuật kiểm thử phần mềm hiệu quả hơn ngày càng trở nên cấp thiết, khiến nó trở thành một lĩnh vực chín muồi cho các phương pháp tiếp cận đổi mới như sử dụng LLM. Bài báo này cung cấp một đánh giá toàn diện về việc sử dụng LLM trong kiểm thử phần mềm. Nó phân tích 102 nghiên cứu liên quan đã sử dụng LLM cho kiểm thử phần mềm, từ cả góc độ kiểm thử phần mềm và góc độ LLM. Bài báo trình bày một cuộc thảo luận chi tiết về các nhiệm vụ kiểm thử phần mềm mà LLM thường được sử dụng, trong đó chuẩn bị trường hợp kiểm thử và sửa chữa chương trình là những nhiệm vụ tiêu biểu nhất. Bài báo cũng phân tích các LLM thường được sử dụng, các loại kỹ thuật nhắc nhở được sử dụng, cũng như các kỹ thuật đi kèm với các LLM này.

Bài viết này cũng tóm tắt những thách thức chính và cơ hội tiềm năng theo hướng này. Công trình này có thể đóng vai trò như một lộ trình cho các nghiên cứu trong tương lai trong lĩnh vực này, nêu bật những hướng đi tiềm năng và xác định những khoảng trống trong hiểu biết hiện tại của chúng ta về việc sử dụng LLM trong kiểm thử phần mềm.

Từ khóa —Mô hình ngôn ngữ lớn được huấn luyện trước, Kiểm thử phần mềm, LLM, GPT

1. GIỚI THIỆU

Kiểm thử phần mềm là một công việc quan trọng, đóng vai trò nền tảng để đảm bảo chất lượng và độ tin cậy của các sản phẩm phần mềm. Nếu thiếu quy trình kiểm thử phần mềm nghiêm ngặt, các doanh nghiệp phần mềm sẽ ngán ngại đưa sản phẩm của mình ra thị trường, vì biết những hậu quả tiềm tàng của việc cung cấp phần mềm lỗi cho người dùng cuối.

Bằng cách tiến hành các quy trình thử nghiệm kỹ lưỡng và tỉ mỉ, các doanh nghiệp phần mềm có thể giảm thiểu sự xuất hiện của các lỗi phần mềm nghiêm trọng, các vấn đề về khả năng sử dụng hoặc vi phạm an ninh có thể dẫn đến tổn thất tài chính hoặc đe dọa lòng tin của người dùng. Ngoài ra, thử nghiệm phần mềm giúp giảm chi phí bảo trì bằng cách xác định và giải quyết các vấn đề sớm trong vòng đời phát triển, ngăn ngừa các biến chứng nghiêm trọng hơn về sau [1], [2].

Tầm quan trọng của kiểm thử phần mềm đã thu hút sự chú ý đáng kể trong cộng đồng nghiên cứu và công nghiệp. Trong lĩnh vực kỹ thuật phần mềm, đây là một lĩnh vực nghiên cứu vô cùng phổ biến và sôi động. Có thể thấy rõ tầm quan trọng không thể phủ nhận của kiểm thử phần mềm bằng cách đơn giản là xem xét các hội nghị và hội thảo tập trung vào kỹ thuật phần mềm. Trong số các sự kiện này, các chủ đề liên quan đến kiểm thử phần mềm luôn chiếm ưu thế về số lượng bài gửi và thường xuyên được chọn để xuất bản.

Mặc dù lĩnh vực kiểm thử phần mềm đã trở nên phổ biến đáng kể, vẫn còn hàng tá thách thức chưa được giải quyết hiệu quả. Ví dụ, một trong những thách thức đó là việc tạo trường hợp kiểm thử đơn vị tự động. Mặc dù có nhiều phương pháp khác nhau, bao gồm các kỹ thuật dựa trên tìm kiếm [3], [4], dựa trên ràng buộc [5] hoặc dựa trên ngẫu nhiên [6] để tạo ra một bộ kiểm thử đơn vị, nhưng phạm vi bao phủ và ý nghĩa của các bài kiểm thử được tạo ra vẫn còn xa mức độ hài lòng [7], [8]. Tương tự, khi nói đến kiểm thử giao diện người dùng đồ họa (GUI) trên thiết bị di động, các nghiên cứu hiện có với các phương pháp dựa trên quy tắc/ngẫu nhiên [9], [10], các phương pháp dựa trên mô hình [11], [12] và các phương pháp dựa trên học máy [13] không thể hiểu được thông tin ngữ nghĩa của trang GUI và thường không đạt được phạm vi bao phủ toàn diện [14], [15]. Xét đến những hạn chế này, nhiều nỗ lực nghiên cứu hiện đang được tiến hành để khám phá các kỹ thuật đổi mới có thể nâng cao hiệu quả của các nhiệm vụ kiểm thử phần mềm, trong đó các mô hình ngôn ngữ lớn là những mô hình hứa hẹn nhất.

Các mô hình ngôn ngữ lớn (LLM) như T5 và GPT-3 đã cách mạng hóa lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) và trí tuệ nhân tạo (AI). Những mô hình này, ban đầu được huấn luyện trước trên các tập dữ liệu lớn, đã thể hiện hiệu suất đáng kể trên nhiều nhiệm vụ NLP bao gồm trả lời câu hỏi, dịch máy và tạo văn bản [16]–[19]. Trong những năm gần đây, đã có sự tiến bộ đáng kể trong LLM với sự xuất hiện của các mô hình có khả năng xử lý các tập dữ liệu quy mô lớn hơn nữa. Sự mở rộng về kích thước mô hình này không chỉ dẫn đến hiệu suất được cải thiện mà còn mở ra những khả năng mới để áp dụng LLM như Trí tuệ Nhân tạo Tổng quát. Trong số các LLM tiên tiến này, các mô hình như ChatGPT1 và LLaMA2 tự hào có hàng tỷ

* J. Wang, Y. Huang, Z. Liu, Q. Wang thuộc Phòng thí nghiệm trọng điểm quốc gia về trò chơi thông minh, Viện Phần mềm, Viện Hàn lâm Khoa học Trung Quốc và Đại học Viện Hàn lâm Khoa học Trung Quốc, Bắc Kinh, Trung Quốc. J. Wang và Q. Wang là tác giả liên hệ.
Email: {junjie, yuchao2019, liuzhe2020, wq}@iscas.ac.cn • C.
Chen làm việc tại Đại học Monash, Melbourne, Úc. Email: chunyang.chen@monash.edu • S. Wang làm
việc tại Đại học York, Toronto, Canada.
Email: wangsong@yorku.ca

1. <https://openai.com/blog/chatgpt> 2. <https://ai.meta.com/blog/large-language-model-llama-meta-ai/>

của các tham số. Những mô hình như vậy nắm giữ tiềm năng to lớn trong việc giải quyết các nhiệm vụ thực tiễn phức tạp trong các lĩnh vực như tạo mã và sáng tạo nghệ thuật. Với khả năng mở rộng và nâng cao năng lực, LLM đã trở thành yếu tố thay đổi cuộc chơi trong xử lý ngôn ngữ tự nhiên (NLP) và trí tuệ nhân tạo (AI), đồng thời thúc đẩy những tiến bộ trong các lĩnh vực khác như lập trình và kiểm thử phần mềm.

LLM đã được sử dụng cho nhiều nhiệm vụ liên quan đến mã hóa bao gồm tạo mã và đề xuất mã [20]- [23]. Một mặt, trong kiểm thử phần mềm, có nhiều nhiệm vụ liên quan đến tạo mã, chẳng hạn như tạo kiểm thử đơn vị [7], trong đó việc sử dụng LLM được kỳ vọng sẽ mang lại hiệu suất tốt. Mặt khác, kiểm thử phần mềm có những đặc điểm riêng biệt phân biệt nó với việc tạo mã. Ví dụ, việc tạo mã chủ yếu tập trung vào việc tạo ra một đoạn mã chính xác duy nhất, trong khi kiểm thử phần mềm thường yêu cầu tạo ra nhiều đầu vào kiểm thử khác nhau để đảm bảo phạm vi bao phủ tốt hơn của phần mềm đang được kiểm thử [1]. Sự tồn tại của những khác biệt này tạo ra những thách thức và cơ hội mới khi sử dụng LLM cho kiểm thử phần mềm.

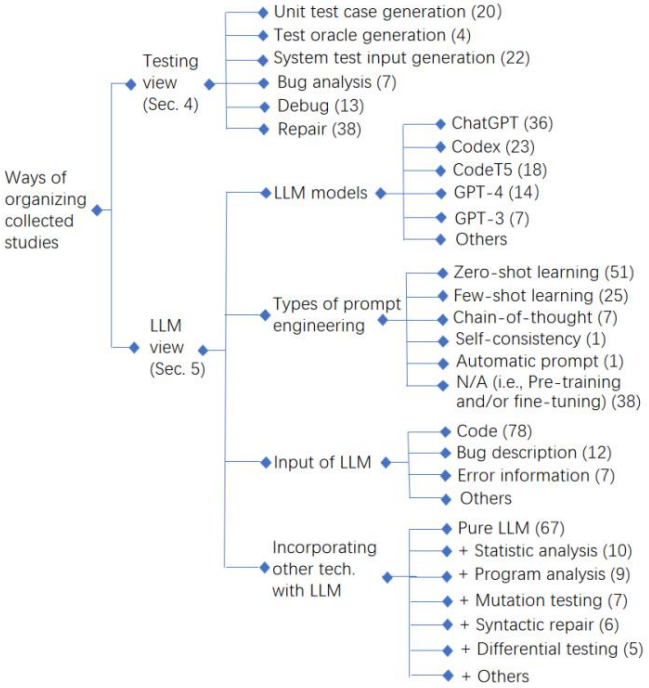
Hơn nữa, mọi người đã được hưởng lợi từ hiệu suất tuyệt vời của LLM trong các nhiệm vụ tạo lập và suy luận, dẫn đến sự xuất hiện của hàng tá phương pháp mới sử dụng LLM để kiểm thử phần mềm.

Bài viết này trình bày một đánh giá toàn diện về việc sử dụng LLM trong kiểm thử phần mềm. Chúng tôi thu thập 102 bài báo liên quan và tiến hành phân tích kỹ lưỡng từ cả hai góc độ kiểm thử phần mềm và LLM, được tóm tắt sơ lược trong Hình 1.

Từ góc độ kiểm thử phần mềm, phân tích của chúng tôi bao gồm việc xem xét các nhiệm vụ kiểm thử phần mềm cụ thể mà LLM được sử dụng. Kết quả cho thấy LLM thường được sử dụng để chuẩn bị trường hợp kiểm thử (bao gồm tạo trường hợp kiểm thử đơn vị, tạo oracle kiểm thử và tạo đầu vào kiểm thử hệ thống), gỡ lỗi chương trình và sửa lỗi, trong khi chúng tôi không tìm thấy các thực tiễn áp dụng LLM trong các nhiệm vụ của giai đoạn đầu vòng đời kiểm thử (như yêu cầu kiểm thử, kế hoạch kiểm thử, v.v.). Đối với mỗi nhiệm vụ kiểm thử, chúng tôi sẽ cung cấp các hình minh họa chi tiết thể hiện việc sử dụng LLM để giải quyết nhiệm vụ, nêu bật các thực tiễn thường được sử dụng, theo dõi xu hướng phát triển công nghệ và tóm tắt hiệu suất đạt được, nhằm giúp người đọc có được cái nhìn tổng quan toàn diện về cách LLM được sử dụng trong các nhiệm vụ kiểm thử khác nhau.

Từ góc nhìn của LLM (Learning Learning), phân tích của chúng tôi bao gồm các LLM thường được sử dụng trong các nghiên cứu này, các loại kỹ thuật điều khiển phản hồi (prompt engineering), đầu vào của LLM, cũng như các kỹ thuật đi kèm với các LLM này. Kết quả cho thấy khoảng một phần ba các nghiên cứu sử dụng LLM thông qua huấn luyện trước hoặc tinh chỉnh lực độ, trong khi những nghiên cứu khác sử dụng kỹ thuật điều khiển phản hồi để giao tiếp với LLM nhằm định hướng hành vi của chúng để đạt được kết quả mong muốn. Đối với kỹ thuật điều khiển phản hồi, các chiến lược học không cần huấn luyện (zero-shot learning) và học ít cần huấn luyện (few-shot learning) được sử dụng phổ biến nhất, trong khi các tiến bộ khác như thúc đẩy chuỗi suy nghĩ và tính tự nhất quán hiếm khi được sử dụng. Kết quả cũng cho thấy các kỹ thuật kiểm thử truyền thống như kiểm thử khác biệt (differential testing) và kiểm thử đột biến (mutation testing) thường được kết hợp với LLM để giúp tạo ra các bài kiểm thử đa dạng hơn.

Hơn nữa, chúng tôi tóm tắt những thách thức chính và cơ hội tiềm năng theo hướng này. Mặc dù việc kiểm thử phần mềm với LLM đã có sự tăng trưởng đáng kể trong...



Hình 1: Cấu trúc nội dung của bài báo này (các số trong ngoặc đơn chỉ số lượng bài báo liên quan, và một bài báo có thể bao gồm không hoặc nhiều mục)

Hai năm qua, vẫn còn nhiều thách thức trong việc đạt được độ bao phủ cao của quá trình kiểm thử, giải quyết vấn đề về oracle kiểm thử, đánh giá nghiêm ngặt và ứng dụng thực tiễn của LLM trong kiểm thử phần mềm. Vì đây là một lĩnh vực mới nổi, nên có rất nhiều cơ hội nghiên cứu, bao gồm việc khám phá LLM ở giai đoạn đầu của quá trình kiểm thử, khám phá LLM cho nhiều loại kiểm thử phần mềm và phi chức năng hơn, khám phá kỹ thuật phản hồi nhanh tiên tiến, cũng như kết hợp LLM với các kỹ thuật truyền thống.

Bài báo này đóng góp những điểm sau: • Chúng tôi phân tích kỹ lưỡng 102 nghiên cứu liên quan đã sử dụng LLM cho kiểm thử phần mềm, về xu hướng xuất bản, phân bố địa điểm xuất bản, v.v. • Chúng tôi tiến hành phân tích toàn diện từ góc độ kiểm thử phần mềm để hiểu rõ sự phân bố các nhiệm vụ kiểm thử phần mềm bằng LLM và trình bày một cuộc thảo luận chi tiết về cách giải quyết các nhiệm vụ này bằng LLM.

• Chúng tôi tiến hành phân tích toàn diện từ góc độ các mô hình logic vòng lặp (LLM), và khám phá các LLM thường được sử dụng, các loại kỹ thuật nhắc nhở, đầu vào của LLM, cũng như các kỹ thuật đi kèm với các LLM này. • Chúng tôi nêu bật những thách thức trong các nghiên cứu hiện có và trình bày các cơ hội tiềm năng cho các nghiên cứu tiếp theo.

Chúng tôi tin rằng công trình này sẽ có giá trị đối với cả các nhà nghiên cứu và những người thực hành trong lĩnh vực kỹ thuật phần mềm, vì nó cung cấp một cái nhìn tổng quan toàn diện về hiện trạng và tầm nhìn tương lai của việc sử dụng LLM cho việc kiểm thử phần mềm. Đối với các nhà nghiên cứu, công trình này có thể đóng vai trò như một lộ trình cho các nghiên cứu trong tương lai trong lĩnh vực này, làm nổi bật các hướng nghiên cứu tiềm năng và xác định những khoảng trống trong hiểu biết hiện tại của chúng ta về việc sử dụng LLM trong kiểm thử phần mềm. Đối với các nhà thực hành, công trình này có thể cung cấp những hiểu biết sâu sắc về những lợi ích và hạn chế tiềm tàng của việc sử dụng LLM cho kiểm thử phần mềm, cũng như hướng dẫn thực tiễn về cách tích hợp chúng một cách hiệu quả.

các quy trình kiểm thử hiện có. Bằng cách cung cấp một bức tranh chi tiết về hiện trạng và tầm nhìn tương lai của việc sử dụng LLM cho kiểm thử phần mềm, công trình này có thể giúp đẩy nhanh việc áp dụng công nghệ này trong cộng đồng kỹ thuật phần mềm và cuối cùng góp phần cải thiện chất lượng và độ tin cậy của các hệ thống phần mềm.

2. BỐI CẢNH 2.1 Mô hình

Ngôn ngữ Lớn (LLM)

Gần đây, các mô hình ngôn ngữ được đào tạo trước (PLM) đã được đề xuất bằng cách đào tạo trước các mô hình dựa trên Transformer trên các tập dữ liệu quy mô lớn, cho thấy khả năng mạnh mẽ trong việc giải quyết các nhiệm vụ xử lý ngôn ngữ tự nhiên (NLP) khác nhau [16]–[19].

Các nghiên cứu đã chỉ ra rằng việc mở rộng mô hình có thể dẫn đến cải thiện năng lực của mô hình, thúc đẩy các nhà nghiên cứu điều tra hiệu ứng mở rộng thông qua việc tăng kích thước tham số hơn nữa.

Điều thú vị là, khi quy mô tham số vượt quá một ngưỡng nhất định, các mô hình ngôn ngữ lớn hơn này không chỉ thể hiện sự cải thiện hiệu suất đáng kể mà còn có những khả năng đặc biệt như học tập theo ngữ cảnh, những khả năng không có ở các mô hình nhỏ hơn như BERT.

Để phân biệt các mô hình ngôn ngữ ở các thang đo tham số khác nhau, cộng đồng nghiên cứu đã đặt ra thuật ngữ mô hình ngôn ngữ lớn (LLM) cho các mô hình ngôn ngữ tham số (PLM) có kích thước đáng kể. LLM thường đề cập đến các mô hình ngôn ngữ có hàng trăm tỷ (hoặc hơn) tham số và được huấn luyện trên dữ liệu văn bản khổng lồ như GPT-3, PaLM, Codex và LLaMA. LLM được xây dựng bằng kiến trúc Transformer, xếp chồng các lớp chú ý đa đầu trong một mạng nơ-ron rất sâu. Các LLM hiện có áp dụng các kiến trúc mô hình (Transformer) và mục tiêu huấn luyện trước (mô hình hóa ngôn ngữ) tương tự như các mô hình ngôn ngữ nhỏ, nhưng mở rộng đáng kể kích thước mô hình, dữ liệu huấn luyện trước và tổng sức mạnh tính toán. Điều này cho phép LLM hiểu ngôn ngữ tự nhiên tốt hơn và tạo ra văn bản chất lượng cao dựa trên ngữ cảnh hoặc lời nhắc được cung cấp.

Lưu ý rằng, trong các tài liệu hiện có, không có sự đồng thuận chính thức nào về quy mô tham số tối thiểu cho LLM, vì khả năng của mô hình cũng liên quan đến kích thước dữ liệu và tổng tính toán. Trong một khảo sát gần đây về LLM [17], các tác giả tập trung thảo luận về các mô hình ngôn ngữ có kích thước mô hình lớn hơn 10B. Theo tiêu chí của họ, LLM đầu tiên là T5 do Google phát hành năm 2019, tiếp theo là GPT-3 do OpenAI phát hành năm 2020, và có hơn ba mươi LLM được phát hành từ năm 2021 đến năm 2023, cho thấy sự phổ biến của nó. Trong một khảo sát khác về việc thống nhất LLM và đồ thị tri thức [24], các tác giả phân loại LLM thành ba loại: chỉ mã hóa (ví dụ: BERT), mã hóa-giải mã (ví dụ: T5) và kiến trúc mạng chỉ giải mã (ví dụ: GPT-3). Trong bài đánh giá của chúng tôi, chúng tôi xem xét các tiêu chí phân loại của hai cuộc khảo sát và chỉ xem xét kiến trúc mạng mã hóa-giải mã và chỉ giải mã của các mô hình ngôn ngữ được huấn luyện trước, vì cả hai đều có thể hỗ trợ các nhiệm vụ tạo sinh. Chúng tôi không xem xét kiến trúc mạng chỉ mã hóa vì chúng không thể xử lý các nhiệm vụ tạo sinh, được đề xuất tương đối sớm (ví dụ: BERT vào năm 2018) và hầu như không có mô hình nào sử dụng kiến trúc này sau năm 2021. Nói cách khác, các LLM được thảo luận trong bài báo này không chỉ bao gồm các mô hình có tham số trên 10B (như đã đề cập trong [17]) mà còn bao gồm các mô hình khác sử dụng mạng mã hóa-giải mã và chỉ giải mã.

kiến trúc (như đã đề cập trong [24]), chẳng hạn như BART với 140 triệu tham số và GPT-2 với kích thước tham số từ 117 triệu đến 1,5 tỷ. Điều này cũng có thể bao gồm nhiều nghiên cứu hơn để chứng minh bối cảnh của chủ đề này.

2.2 Kiểm thử phần mềm Kiểm thử

phần mềm là một quy trình quan trọng trong phát triển phần mềm, bao gồm việc đánh giá chất lượng của sản phẩm phần mềm. Mục tiêu chính của kiểm thử phần mềm là xác định các khiếm khuyết hoặc lỗi trong hệ thống phần mềm có thể dẫn đến hành vi không chính xác hoặc không mong muốn. Toàn bộ vòng đời của kiểm thử phần mềm thường bao gồm các nhiệm vụ sau (mình họa trong Hình 4):

- Phân tích yêu cầu: phân tích các yêu cầu phần mềm và xác định mục tiêu, phạm vi và tiêu chí kiểm thử.

- Kế hoạch kiểm thử: xây dựng một kế hoạch kiểm thử nêu rõ các bước kiểm thử, chiến lược, mục tiêu thử nghiệm và lịch trình.
- Thiết kế và xem xét kiểm thử: Phát triển và xem xét các trường hợp kiểm thử và bộ kiểm thử phù hợp với kế hoạch kiểm thử và yêu cầu của ứng dụng phần mềm.
- Chuẩn bị trường hợp kiểm thử: Chuẩn bị các trường hợp kiểm thử thực tế dựa trên thiết kế đã tạo ở giai đoạn trước.
- Thực hiện kiểm thử: Thực hiện các bài kiểm thử đã được thiết kế ở giai đoạn trước. Hệ thống phần mềm được chạy với các trường hợp kiểm thử và kết quả được ghi lại.
- Báo cáo kiểm thử: Phân tích kết quả kiểm thử và tạo báo cáo tóm tắt quá trình kiểm thử và xác định bất kỳ lỗi hoặc sự cố nào được phát hiện.
- Sửa lỗi và kiểm thử hồi quy: Báo cáo các lỗi hoặc sự cố được xác định trong quá trình kiểm thử cho nhóm phát triển để sửa chữa. Sau khi các lỗi được sửa, kiểm thử hồi quy được thực hiện để đảm bảo rằng các thay đổi không gây ra lỗi hoặc sự cố mới.

- Phát hành phần mềm: Sau khi hệ thống phần mềm đã vượt qua tất cả các giai đoạn kiểm thử và các lỗi đã được khắc phục, phần mềm có thể được phát hành cho khách hàng hoặc người dùng cuối.

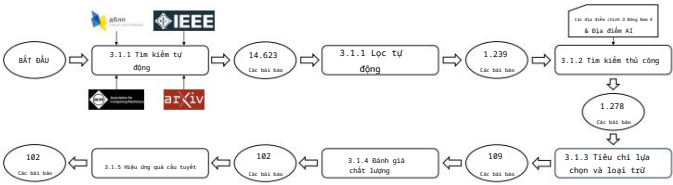
nguồn: [25].

Quá trình kiểm thử là một quá trình lặp đi lặp lại và có thể bao gồm nhiều chu kỳ của các giai đoạn nêu trên, tùy thuộc vào độ phức tạp của hệ thống phần mềm và các yêu cầu kiểm thử.

Trong giai đoạn thử nghiệm, nhiều loại thử nghiệm khác nhau có thể được thực hiện, bao gồm thử nghiệm đơn vị, thử nghiệm tích hợp, thử nghiệm hệ thống và thử nghiệm nghiệm thu.

- Kiểm thử đơn vị (Unit Testing) bao gồm việc kiểm thử từng đơn vị hoặc thành phần riêng lẻ của ứng dụng phần mềm để đảm bảo chúng hoạt động chính xác.
- Kiểm thử tích hợp (Integration Testing) bao gồm việc kiểm thử các mô-đun hoặc thành phần khác nhau của ứng dụng phần mềm cùng nhau để đảm bảo chúng hoạt động chính xác như một hệ thống.
- Kiểm thử hệ thống (System Testing) bao gồm việc kiểm thử toàn bộ hệ thống phần mềm, bao gồm tất cả các thành phần tích hợp và các phụ thuộc bên ngoài.
- Kiểm thử chấp nhận (Acceptance Testing) bao gồm việc kiểm thử ứng dụng phần mềm để đảm bảo nó đáp ứng các yêu cầu kinh doanh và sẵn sàng để triển khai.

Ngoài ra, có thể có thử nghiệm chức năng, thử nghiệm hiệu năng, thử nghiệm đơn vị, thử nghiệm bảo mật, thử nghiệm khả năng truy cập, v.v., nhằm khám phá nhiều khía cạnh khác nhau của phần mềm đang được thử nghiệm [25].



Hình 2: Tổng quan về quy trình thu gom giấy

3. QUY TRÌNH LỰA CHỌN VÀ XEM XÉT BÀI VIẾT

3.1 Phương pháp thu thập bài báo Hình 2 minh họa

quy trình tìm kiếm và lựa chọn bài báo của chúng tôi. Để thu thập càng nhiều tài liệu liên quan càng tốt, chúng tôi sử dụng cả tìm kiếm tự động (từ cơ sở dữ liệu kho lưu trữ bài báo) và tìm kiếm thủ công (từ các diễn đàn kỹ thuật phần mềm và trí tuệ nhân tạo lớn). Chúng tôi đã tìm kiếm các bài báo từ tháng 1 năm 2019 đến tháng 6 năm 2023 và tiếp tục tiến hành vòng tìm kiếm thứ hai để bao gồm các bài báo từ tháng 7 năm 2023 đến tháng 10 năm 2023.

3.1.1 Tìm kiếm tự động

Để đảm bảo thu thập được các bài báo từ nhiều lĩnh vực nghiên cứu khác nhau, chúng tôi tiến hành tìm kiếm rộng rãi bằng cách sử dụng bốn cơ sở dữ liệu khoa học phổ biến: thư viện số ACM, thư viện số IEEE Xplore, arXiv và DBLP.

Chúng tôi tìm kiếm các bài báo có tiêu đề chứa các từ khóa liên quan đến các nhiệm vụ kiểm thử phần mềm và các kỹ thuật kiểm thử (như được hiển thị bên dưới) trong ba cơ sở dữ liệu đầu tiên. Trong trường hợp của DBLP, chúng tôi sử dụng thêm các từ khóa liên quan đến LLM (như được hiển thị bên dưới) để lọc ra các nghiên cứu không liên quan, vì chỉ dựa vào các từ khóa liên quan đến kiểm thử sẽ dẫn đến một số lượng lớn các nghiên cứu tiềm năng. Mặc dù việc sử dụng hai bộ từ khóa cho DBLP có thể dẫn đến việc bỏ sót một số nghiên cứu liên quan, nhưng chúng tôi tin rằng đây vẫn là một chiến lược khả thi. Điều này là do một số lượng đáng kể các nghiên cứu có trong cơ sở dữ liệu này đã có thể được tìm thấy trong ba cơ sở dữ liệu đầu tiên, và cơ sở dữ liệu thứ tư chỉ đóng vai trò là nguồn bổ sung để thu thập thêm các bài báo. • Từ khóa liên quan đến các nhiệm vụ và kỹ thuật kiểm thử phần mềm: test OR bug OR issue OR defect OR

fault OR error OR failure OR crash OR debug OR debugger OR repair OR fix OR assert OR verification OR validation OR fuzz OR fuzzer OR mutation.

• Từ khóa liên quan đến LLM: LLM HOẶC mô hình ngôn ngữ HOẶC mô hình tạo sinh HOẶC mô hình lớn HOẶC GPT-3 HOẶC ChatGPT HOẶC GPT-4 HOẶC LLaMA HOẶC PaLM2 HOẶC CodeT5 HOẶC CodeX HOẶC CodeGen HOẶC Bard HOẶC InstructGPT. Lưu ý rằng, chúng tôi chỉ liệt kê mười LLM phổ biến nhất (dựa trên tìm kiếm của Google), vì chúng là các từ khóa tìm kiếm phù hợp với tiêu đề bài báo, chứ không phải phù hợp với nội dung bài báo.

Chiến lược tìm kiếm trên dựa trên tiêu đề bài báo có thể thu hồi một số lượng lớn bài báo, và chúng tôi tiếp tục thực hiện lọc tự động dựa trên nội dung bài báo. Cụ thể, chúng tôi lọc các bài báo có nội dung chứa “LLM” hoặc “mô hình ngôn ngữ” hoặc “mô hình tạo sinh” hoặc “mô hình lớn” hoặc tên của LLM (sử dụng LLM trong [17], [24] ngoại trừ những LLM nằm trong tiêu chí loại trừ của chúng tôi). Điều này có thể giúp loại bỏ các bài báo không liên quan đến mô hình thần kinh.

3.1.2 Tìm kiếm thủ công

Để bù đắp cho những thiếu sót có thể xảy ra do tìm kiếm tự động, chúng tôi cũng tiến hành tìm kiếm thủ công.

Để đảm bảo thu thập được các bài báo có tính liên quan cao, chúng tôi tiến hành tìm kiếm thủ công trong kỳ yếu hội nghị và bài báo tạp chí từ các diễn đàn kỹ thuật phần mềm hàng đầu (được liệt kê trong Bảng 2).

Ngoài ra, do tính chất liên ngành của công trình này, chúng tôi cũng đưa vào các báo cáo hội nghị thuộc lĩnh vực trí tuệ nhân tạo. Chúng tôi chọn ra mười địa điểm hàng đầu dựa trên chỉ số h5 từ Google Scholar và loại trừ ba địa điểm về thị giác máy tính, cụ thể là CVPR, ICCV, ECCV, như được liệt kê trong Bảng 2.

3.1.3 Tiêu chí lựa chọn và loại trừ Việc tìm kiếm

trên các cơ sở dữ liệu và địa điểm xuất bản được thiết kế rất toàn diện. Điều này cho phép chúng tôi thu thập càng nhiều bài báo càng tốt vào kho dữ liệu của mình. Tuy nhiên, tính toàn diện rộng rãi này dẫn đến việc có những bài báo không liên quan trực tiếp đến phạm vi của khảo sát này. Do đó, chúng tôi xác định một bộ tiêu chí lựa chọn và loại trừ cụ thể, sau đó áp dụng chúng cho từng bài báo trong kho dữ liệu và loại bỏ những bài báo không đáp ứng các tiêu chí. Điều này đảm bảo rằng mỗi bài báo được thu thập đều phù hợp với phạm vi và câu hỏi nghiên cứu của chúng tôi.

Tiêu chí lựa chọn. Chúng tôi xác định các tiêu chí sau đây để lựa chọn bài báo: • Bài báo đề

xuất hoặc cải tiến một phương pháp, nghiên cứu hoặc công cụ/khung nhằm mục đích kiểm thử phần mềm hoặc hệ thống cụ thể bằng LLM.

• Bài báo này áp dụng các mô hình LLM vào thực tiễn kiểm thử phần mềm, bao gồm tất cả các nhiệm vụ trong vòng đời kiểm thử phần mềm như đã trình bày trong Phần 2.2.

• Bài báo trình bày một nghiên cứu thực nghiệm về việc sử dụng LLM trong thực tiễn kiểm thử phần mềm. • Bài báo đề cập đến các kỹ thuật kiểm thử cụ thể (ví dụ: (kiểm thử mờ) sử dụng LLM.

Nếu một bài báo đáp ứng bất kỳ tiêu chí nào sau đây, chúng tôi sẽ đưa nó vào.

Tiêu chí loại trừ. Các nghiên cứu sau đây sẽ bị loại trừ trong quá trình lựa chọn nghiên cứu: • Bài báo không

liên quan đến các nhiệm vụ kiểm thử phần mềm, ví dụ: tạo chú thích mã. • Bài báo không sử dụng LLM, ví dụ:

sử dụng mạng nơ-ron hồi quy. • Bài báo chỉ đề cập đến LLM trong phần công việc tương lai hoặc

thảo luận thay vì sử dụng LLM trong phương pháp tiếp cận. • Bài báo sử dụng các mô hình ngôn ngữ với kiến trúc chỉ có bộ mã hóa, ví dụ: BERT, không thể được sử dụng trực tiếp cho các nhiệm vụ tạo ngôn ngữ (như đã được chứng minh trong Phần 2.1).

• Bài báo tập trung vào việc kiểm tra hiệu suất của LLM, chẳng hạn như tính công bằng, tính ổn định, tính bảo mật, v.v. [125]–[127]. •

Bài báo tập trung vào việc đánh giá hiệu suất của các công cụ hỗ trợ LLM, ví dụ: đánh giá chất lượng mã của công cụ tạo mã Copilot [128]–[130].

Đối với các bài báo được thu thập thông qua tìm kiếm tự động và tìm kiếm thủ công, chúng tôi tiến hành kiểm tra thủ công để xem liệu chúng có đáp ứng tiêu chí lựa chọn của chúng tôi hay không và loại bỏ những bài không đáp ứng tiêu chí loại trừ. Cụ thể, hai tác giả đầu tiên đọc kỹ từng bài báo để xác định xem liệu chúng có đáp ứng tiêu chí lựa chọn hay không.

BẢNG 1: Chi tiết các bài báo đã thu thập

Chủ đề	ID	Tên bài báo	Năm	thậm	chiếu
1		Tạo trường hợp kiểm thử đơn vị	2020	[26]	
2		Tạo trường hợp kiểm thử đơn vị	2022	[27]	
3		Tạo trường hợp kiểm thử đơn vị	2022	[28]	
4		Tạo trường hợp kiểm thử đơn vị	2023	[29]	
5		Tạo trường hợp kiểm thử đơn vị	2023	[30]	
6		Tạo trường hợp kiểm thử đơn vị	2023	[31]	
7		Tạo trường hợp kiểm thử đơn vị	2023	[32]	
8		Tạo trường hợp kiểm thử đơn vị	2023	[33]	
9		Tạo trường hợp kiểm thử đơn vị	2023	[34]	
10		Tạo trường hợp kiểm thử đơn vị	2023	[35]	
11		Tạo trường hợp kiểm thử đơn vị	2023	[8]	
12		Tạo trường hợp kiểm thử đơn vị	2023	[36]	
13		Tạo trường hợp kiểm thử đơn vị	2023	[37]	
14		Tạo trường hợp kiểm thử đơn vị	2023	[38]	
15		Tạo trường hợp kiểm thử đơn vị	2023	[39]	
16		Tạo trường hợp kiểm thử đơn vị	2023	[40]	
17		Tạo trường hợp kiểm thử đơn vị	2023	[7]	
18		Tạo trường hợp kiểm thử đơn vị	2023	[41]	
19		Tạo trường hợp kiểm thử đơn vị	2023	[42]	
20		Tạo trường hợp kiểm thử đơn vị	2023	[43]	
21		Tạo Oracle thử nghiệm	2022	[44]	
22		Kiểm thử oracle 23	2023	[45]	
23		Tạo mã kiểm thử; Sửa chữa chương trình bằng cách sử dụng học chuyển giao cho các tác vụ liên quan đến mã.	2023	[46]	
24		Kiểm tra việc tạo oracle; Sửa chữa chương trình 25.	2023	[47]	
25		Tạo dữ liệu đầu vào kiểm thử hệ thống	2021	[48]	
26		Tạo dữ liệu đầu vào kiểm thử hệ thống	2022	[49]	
27		Tạo dữ liệu đầu vào kiểm thử hệ thống 28	2022	[50]	
28		Tạo dữ liệu đầu vào kiểm thử hệ thống	2021	[51]	
29		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[52]	
30		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[53]	
31		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[54]	
32		Tạo dữ liệu đầu vào kiểm thử hệ thống 33	2023	[55]	
33		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[56]	
34		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[57]	
35		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[58]	
36		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[59]	
37		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[60]	
38		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[61]	
39		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[14]	
40		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[62]	
41		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[63]	
42		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[64]	
43		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[65]	
44		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[66]	
45		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[67]	
46		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[68]	
47		Tạo dữ liệu đầu vào kiểm thử hệ thống	2023	[69]	
48		Phân tích lỗi	2022	[70]	
49		Phân tích lỗi	2023	[71]	
50		Phân tích lỗi	2023	[72]	
51		Phân tích lỗi	2023	[73]	
52		Phân tích lỗi	2022	[74]	
53		Phân tích lỗi	2023	[75]	
54		Phân tích lỗi	2023	[76]	
55		Gỡ lỗi	2022	[77]	
56		Gỡ lỗi	2022	[78]	
57		GỠ LỖI 58	2023	[79]	
58		Gỡ lỗi	2023	[80]	
59		Gỡ lỗi	2023	[81]	
60		Gỡ lỗi	2023	[82]	
61		Gỡ lỗi	2023	[83]	
62		GỠ LỖI	2023	[84]	
63		Gỡ lỗi	2023	[85]	
64		GỠ LỖI 65	2023	[86]	
65		Gỡ lỗi	2023	[87]	
66		GỠ LỖI	2023	[88]	
67		Sửa chữa chương trình 68	2023	[89]	
68		Sửa chữa chương trình	2023	[90]	
69		Sửa chữa chương trình	2023	[91]	
70		Sửa chữa chương trình	2023	[92]	
71		Sửa chữa chương trình	2023	[93]	
72		Sửa chữa chương trình	2023	[94]	
73		Sửa chữa chương trình	2023	[95]	
74		Sửa chữa chương trình	2023	[96]	
75		Sửa chữa chương trình	2023	[97]	
76		Sửa chữa chương trình	2023	[98]	
77		Sửa chữa chương trình	2023	[99]	
78		Sửa chữa chương trình	2023	[100]	
79		Sửa chữa chương trình	2023	[101]	
80		Sửa chữa chương trình	2023	[102]	
81		Sửa chữa chương trình	2023	[103]	
82		Sửa chữa chương trình	2023	[104]	
83		Sửa chữa chương trình	2023	[105]	
84		Sửa chữa chương trình	2023	[106]	
85		Sửa chữa chương trình	2023	[107]	
86		Sửa chữa chương trình	2023	[108]	
87		Sửa chữa chương trình	2023	[109]	
88		Sửa chữa chương trình	2023	[110]	
89		Sửa chữa chương trình	2023	[111]	
90		Sửa chữa chương trình	2023	[112]	
91		Sửa chữa chương trình	2023	[113]	
92		Sửa chữa chương trình	2023	[114]	
93		Sửa chữa chương trình	2023	[115]	
94		Sửa chữa chương trình	2023	[116]	
95		Sửa chữa chương trình	2023	[117]	
96		Sửa chữa chương trình	2023	[118]	
97		Sửa chữa chương trình	2023	[119]	
98		Sửa chữa chương trình	2023	[120]	
99		Sửa chữa chương trình	2023	[121]	
100		Sửa chữa chương trình 100	2022	[122]	
101		Sửa chữa chương trình 101	2023	[123]	
102		Sửa chữa chương trình	2023	[124]	

BẢNG 2: Kỳ yếu hội nghị và các tạp chí được xem xét để tìm kiếm thủ công

	Từ viết tắt	Địa điểm
Hội nghị	ICSE	Hội nghị quốc tế về kỹ thuật phần mềm
	ESEC/FSE	Hội nghị và hội thảo chung về kỹ thuật phần mềm châu Âu
		Các nguyên lý cơ bản của kỹ thuật phần mềm
	ASE	Hội nghị quốc tế về kỹ thuật phần mềm tự động
	ISSTA	Hội thảo quốc tế về kiểm thử và phân tích phần mềm
	ICST	Hội nghị quốc tế về kiểm thử, xác minh và thẩm định phần mềm
	ESEM	Hội thảo quốc tế về kỹ thuật phần mềm thực nghiệm và đo lường
	MSR	Hội nghị quốc tế về khai thác kho lưu trữ phần mềm
	QRS	Hội nghị quốc tế về chất lượng, độ tin cậy và bảo mật phần mềm
	ICSME	Hội nghị quốc tế về bảo trì và phát triển phần mềm
	ISSRE	Hội thảo quốc tế về kỹ thuật độ tin cậy cây phần mềm
Tạp chí	TSE	Giao dịch về kỹ thuật phần mềm
	TOSEM	Giao dịch về kỹ thuật phần mềm và phương pháp luận
	EMSE	Kỹ thuật phần mềm thực nghiệm
	ASE	Kỹ thuật phần mềm tự động
	JSS	Tạp chí Hệ thống và Phần mềm
	JSEP	Tạp chí Phần mềm: Sự tiến hóa và Quy trình
	STVR	Kiểm thử, xác minh và độ tin cậy phần mềm
	IEEE SOFTW. Phần mềm IEEE	
	IET SOFTW. Phần mềm IET	
	IST	Công nghệ Thông tin và Phần mềm
Điểm	SQJ	Tạp chí chất lượng phần mềm
	ICLR	Hội nghị quốc tế về biểu diễn học tập
	NeurIPS	Hội nghị về Hệ thống xử lý thông tin thần kinh
	ICML	Hội nghị quốc tế về học máy
	AAAI	Hội nghị AAAI về Trí tuệ Nhân tạo
	EMNLP	Hội nghị về các phương pháp thực nghiệm trong xử lý ngôn ngữ tự nhiên
	ACL	Hội nghị thường niên của Hiệp hội Ngôn ngữ học tính toán
	IJCAI	Hội nghị chung quốc tế về trí tuệ nhân tạo

Bài báo nên được đưa vào dựa trên các tiêu chí lựa chọn và tiêu chí loại trừ, và bất kỳ bài báo nào có quyết định khác sẽ bị loại bỏ. Đã chuyển giao cho tác giả thứ ba đưa ra quyết định cuối cùng.

3.1.4 Đánh giá chất lượng

Ngoài ra, chúng tôi thiết lập các tiêu chí đánh giá chất lượng để loại trừ các nghiên cứu chất lượng thấp như được trình bày bên dưới. Đối với mỗi câu hỏi, chất lượng của nghiên cứu được đánh giá là “có”, “một phần” hoặc “không”. Các bài thi được gán giá trị lần lượt là 1, 0,5 và 0. Những bài thi có điểm dưới 8 sẽ bị loại khỏi vòng tuyển chọn.

nguyên cứu của chúng tôi.

- Có mục tiêu nghiên cứu nào được nêu rõ ràng liên quan đến phần mềm không?
Đang thử nghiệm?
- Có kỹ thuật nào được xác định và có thể lặp lại không?
- Có đóng góp cụ thể nào cho việc kiểm thử phần mềm không?
- Có mô tả rõ ràng nào về những chương trình LLM nào được sử dụng không?
được chấp nhận?
- Có lời giải thích rõ ràng nào về cách thức hoạt động của các LLM không?
đã được sử dụng?
- Có phương pháp luận rõ ràng nào để xác thực công nghệ không?
độc đáo?
- Các dự án được chọn để thẩm định có phù hợp không?
Nhằm phục vụ cho mục tiêu nghiên cứu?
- Có những kỹ thuật kiểm soát hoặc tiêu chuẩn cơ sở nào để chứng minh hiệu quả của kỹ thuật được đề xuất?
- Các chỉ số đánh giá có phù hợp không (ví dụ: đánh giá hiệu quả của kỹ thuật được đề xuất) đối với nghiên cứu mục tiêu?
- Kết quả trình bày trong nghiên cứu có phù hợp với
Mục tiêu nghiên cứu và liệu chúng có được trình bày một cách rõ ràng hay không.
và theo cách thức phù hợp?

3.1.5 Hiệu ứng quả cầu tuyết

Sau khi tìm kiếm trong các kho dữ liệu, kỳ yếu hội nghị và tạp chí, đồng thời áp dụng các tiêu chí bao gồm loại trừ và đánh giá chất lượng, chúng ta thu được kết quả ban đầu. tập hợp các bài báo. Tiếp theo, để giảm thiểu nguy cơ bỏ sót tài liệu liên quan trong khảo sát này, chúng tôi cũng thực hiện tìm kiếm ngược.



Hình 3: Xu hướng số lượng bài báo theo năm

sự lan rộng [131] bằng cách kiểm tra các tài liệu tham khảo được trích dẫn bởi Tổng hợp các bài báo đã được thu thập cho đến nay. Xin lưu ý rằng, quy trình này không bao gồm các nghiên cứu mới, có thể là do chủ đề được khảo sát. Đây là một lĩnh vực khá mới và các nghiên cứu tham khảo thường được công bố trước đó, và chúng tôi đã bao gồm một bộ sưu tập tương đối toàn diện. Tìm kiếm tự động và thủ công.

3.2 Kết quả thu thập

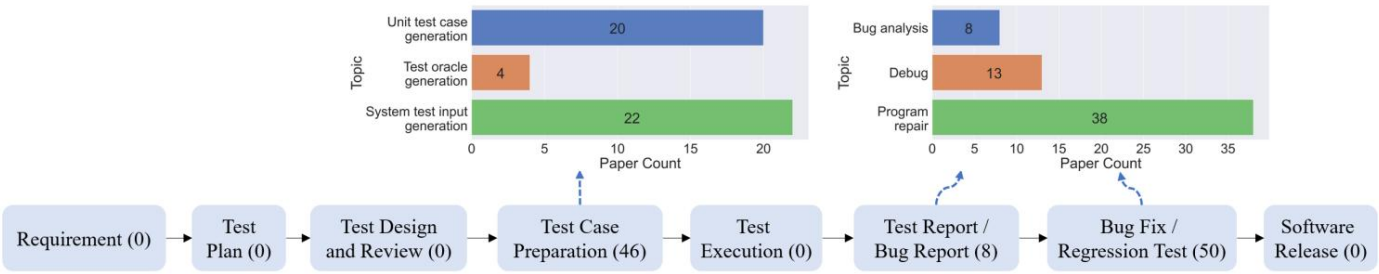
Như thể hiện trong Hình 2, quá trình thu thập đã bắt đầu. với tổng cộng 14.623 bài báo được thu thập từ bốn nguồn. Các cơ sở dữ liệu học thuật sử dụng tìm kiếm từ khóa. Sau đó, thực hiện lọc tự động, tìm kiếm thủ công và áp dụng. tiêu chí lựa chọn/loại trừ và đánh giá chất lượng, chúng tôi Cuối cùng, đã thu thập được tổng cộng 102 bài báo liên quan đến phần mềm. Thử nghiệm với LLM. Bảng 1 trình bày chi tiết các dữ liệu đã thu thập. các bài báo. Ngoài ra, chúng tôi cũng sử dụng Bảng 5 (ở cuối phần...). (bài báo) để cung cấp cái nhìn tổng quan toàn diện hơn về những điều này. các bài báo liên quan đến các đặc điểm cụ thể sẽ được minh họa trong Phần 4 và Phần 5.

Xin lưu ý rằng, có hai nghiên cứu tương ứng. phần mở rộng của một bài báo đã được xuất bản trước đó của cùng tác giả. tác giả ([46] và [132], [68] và [133]), và chúng tôi chỉ giữ Phiên bản mở rộng để tránh trùng lặp.

3.3 Tổng quan về các bài báo được tập hợp

Trong số các bài báo, 47% được xuất bản trong lĩnh vực phần mềm. các địa điểm thi kỹ thuật, trong đó có 19 bài thi thuộc hệ thống ICSE. Có 5 bài báo từ FSE, 5 bài báo từ ASE và 3 bài báo từ ISSTA. 2% số bài báo được xuất bản trên các tạp chí khoa học viễn tưởng. Các diễn đàn tình báo như EMNLP và ICLR, và 5% bài báo được xuất bản trên các diễn đàn phân tích chương trình hoặc an ninh. như PLDI và S&P. Ngoài ra, 46% số bài báo chưa nhưng chưa được xuất bản thông qua các kênh bình duyệt, tức là chúng là được công bố trên arXiv. Điều này dễ hiểu vì lĩnh vực này đang nổi lên và nhiều tác phẩm vừa mới hoàn thành và đang trong quá trình thực hiện. quá trình nộp bài. Mặc dù những bài báo này không Chúng tôi trải qua quá trình đánh giá ngang hàng và có quy trình đánh giá chất lượng. điều đó loại bỏ những bài báo có chất lượng thấp, có khả năng Đảm bảo chất lượng của cuộc khảo sát này.

Hình 3 minh họa xu hướng của các bài báo mà chúng tôi đã thu thập. mỗi năm. Chúng ta có thể thấy rằng theo thời gian, số lượng Số lượng bài báo trong lĩnh vực này đang tăng lên gần như theo cấp số nhân. Năm 2020 Vào năm 2021, chỉ có lần lượt 1 và 2 bài báo. Năm 2022 có 19 bài báo, và năm 2023 đã có 82 bài.



Hình 4: Phân bố các nhiệm vụ kiểm thử với LLM (phù hợp với chu kỳ sống kiểm thử phần mềm [134]–[136], số trong ngoặc đơn cho biết số lượng nghiên cứu được thu thập cho mỗi nhiệm vụ và một bài báo có thể liên quan đến nhiều nhiệm vụ)

các bài báo. Có thể sẽ có nhiều bài báo hơn nữa trong tương lai, điều này cho thấy sự phổ biến và sự quan tâm mà lĩnh vực này đang nhận được.

4 PHÂN TÍCH TỪ KIỂM THỬ PHẦN MỀM-
QUAN ĐIỂM

Phần này trình bày phân tích của chúng tôi từ góc độ kiểm thử phần mềm và sắp xếp các nghiên cứu đã thu thập theo các nhiệm vụ kiểm thử. Hình 4 liệt kê sự phân bố của từng nhiệm vụ kiểm thử liên quan, phù hợp với vòng đời kiểm thử phần mềm. Trước tiên, chúng tôi cung cấp tổng quan về sự phân bố, sau đó là phân tích chi tiết hơn cho từng nhiệm vụ. Lưu ý rằng, đối với mỗi tiêu mục tiếp theo, tổng số tích lũy của các tiêu mục có thể không phải lúc nào cũng khớp với tổng số bài báo vì một bài báo có thể thuộc nhiều hơn một tiêu mục.

Chúng ta có thể thấy rằng LLM đã được sử dụng hiệu quả trong cả giai đoạn giữa và cuối của vòng đời kiểm thử phần mềm. Trong giai đoạn chuẩn bị trường hợp kiểm thử, LLM đã được sử dụng cho các nhiệm vụ như tạo trường hợp kiểm thử đơn vị, tạo oracle kiểm thử và tạo đầu vào kiểm thử hệ thống. Những nhiệm vụ này rất quan trọng trong giai đoạn giữa của kiểm thử phần mềm để giúp phát hiện các vấn đề và ngăn chặn việc phát triển tiếp theo cho đến khi các vấn đề được giải quyết. Hơn nữa, trong các giai đoạn sau như báo cáo thử nghiệm/báo cáo lỗi và giai đoạn sửa lỗi, LLM (Learning Learning Module) được sử dụng cho các nhiệm vụ như phân tích lỗi, gỡ lỗi và sửa chữa. Những nhiệm vụ này rất quan trọng vào cuối giai đoạn thử nghiệm khi cần phải giải quyết các lỗi phần mềm để chuẩn bị cho việc phát hành sản phẩm.

4.1 Tạo trường hợp kiểm thử đơn vị

Việc tạo trường hợp kiểm thử đơn vị bao gồm việc viết các trường hợp kiểm thử đơn vị để kiểm tra từng đơn vị/thành phần riêng lẻ của phần mềm một cách độc lập và đảm bảo chúng hoạt động chính xác. Đối với một phương thức cần kiểm thử (thường được gọi là phương thức trọng tâm), trường hợp kiểm thử đơn vị tương ứng bao gồm một tiền tố kiểm thử và một oracle kiểm thử. Cụ thể, tiền tố kiểm thử thường là một chuỗi các câu lệnh gọi phương thức hoặc câu lệnh gán, nhằm mục đích đưa phương thức trọng tâm về trạng thái có thể kiểm thử; và sau đó, oracle kiểm thử đóng vai trò là đặc tả để kiểm tra xem hành vi hiện tại của phương thức trọng tâm có đáp ứng được kỳ vọng hay không, ví dụ như khẳng định kiểm thử.

Để giảm bớt nỗ lực thủ công trong việc viết các bài kiểm thử đơn vị, các nhà nghiên cứu đã đề xuất nhiều kỹ thuật khác nhau để tạo điều kiện thuận lợi cho việc tạo bài kiểm thử đơn vị tự động. Các kỹ thuật tạo bài kiểm thử đơn vị truyền thống tận dụng các chiến lược dựa trên tìm kiếm [3], [4], dựa trên ràng buộc [5] hoặc dựa trên ngẫu nhiên [6] để tạo ra một bộ bài kiểm thử đơn vị với mục tiêu chính là tối đa hóa phạm vi bao phủ trong phần mềm được kiểm thử.

Tuy nhiên, phạm vi bao phủ và ý nghĩa của các bài kiểm tra được tạo ra vẫn còn xa mức độ hài lòng.

Vì LLM đã chứng minh được những kết quả đầy hứa hẹn trong các nhiệm vụ như tạo mã, và vì cả việc tạo mã lẫn tạo trường hợp kiểm thử đơn vị đều liên quan đến việc tạo mã nguồn, nên các nghiên cứu gần đây đã mở rộng phạm vi tạo mã để bao gồm cả việc tạo trường hợp kiểm thử đơn vị.

Mặc dù đạt được thành công bước đầu, vẫn có những điểm khác biệt tinh tế giữa việc tạo trường hợp kiểm thử đơn vị và việc tạo mã chung, cho thấy cần có những phương pháp tiếp cận phù hợp hơn.

Huấn luyện sơ bộ hoặc tinh chỉnh các mô hình logic bậc cao (LLM) để tạo ra các trường hợp kiểm thử đơn vị. Do những hạn chế của LLM ở giai đoạn đầu, phần lớn các nghiên cứu đã công bố trước đây đều áp dụng phương pháp huấn luyện sơ bộ hoặc tinh chỉnh này. Hơn nữa, trong một số nghiên cứu gần đây, phương pháp này vẫn tiếp tục được sử dụng để tăng cường sự quen thuộc của LLM với kiến thức chuyên môn. Alagarasamy et al. [29] đã huấn luyện trước LLM với phương pháp trọng tâm và các câu lệnh khẳng định để cho phép LLM có kiến thức nền tảng vững chắc hơn về các khẳng định, sau đó tinh chỉnh LLM cho nhiệm vụ tạo trường hợp thử nghiệm, trong đó mục tiêu là học mối quan hệ giữa phương pháp trọng tâm và trường hợp thử nghiệm tương ứng. Tufano et al. [26] đã sử dụng một lược đồ tương tự bằng cách huấn luyện trước LLM trên một kho ngữ liệu Java lớn không được giám sát và tinh chỉnh có giám sát một nhiệm vụ dịch thuật hạ nguồn để tạo ra các bài kiểm tra đơn vị.

Hashtroudi et al. [32] đã tận dụng các bài kiểm tra do nhà phát triển viết sẵn cho mỗi dự án để tạo ra một tập dữ liệu cụ thể cho dự án nhằm thích ứng miền khi tinh chỉnh LLM, điều này có thể tạo điều kiện thuận lợi cho việc tạo ra các bài kiểm tra đơn vị để đọc đối với con người. Rao et al. [35] đã huấn luyện trước mô hình ngôn ngữ kiểu GPT bằng cách sử dụng tín hiệu huấn luyện trước xem xét rõ ràng việc ánh xạ giữa mã và tệp kiểm thử. Steenhoek et al. [42] sử dụng học tăng cường để tối ưu hóa các mô hình bằng cách cung cấp phần thưởng dựa trên các số liệu chất lượng tính có thể được tính toán tự động cho các trường hợp kiểm thử đơn vị được tạo ra.

Thiết kế các lời nhắc hiệu quả để tạo trường hợp kiểm thử đơn vị. Sự phát triển của LLM đã cho phép chúng vượt trội trong các nhiệm vụ được nhắm mục tiêu mà không cần đào tạo trước hoặc tinh chỉnh. Do đó, hầu hết các nghiên cứu sau này thường tập trung vào cách thiết kế lời nhắc, để giúp LLM hiểu rõ hơn ngữ cảnh và sắc thái của nhiệm vụ này. Xie et al. [36] đã tạo ra các trường hợp kiểm thử đơn vị bằng cách phân tích dự án, trích xuất thông tin cần thiết và tạo ra ngữ cảnh trọng tâm thích ứng bao gồm một phương thức trọng tâm và các phụ thuộc của nó trong giới hạn mã thông báo lời nhắc tối đa được xác định trước của LLM, và kết hợp ngữ cảnh này vào một lời nhắc để truy vấn LLM.

BẢNG 3: Hiệu suất tạo trường hợp kiểm thử đơn vị

Độ chính xác của tập dữ liệu 5 dự án	Java từ Defects4J	16,21% 10	Thạc sĩ luật	Giấy
dự án Java CodeSearchNet HumanEval SF110		5%-13% (độ phủ đường	BART	[26]
	40% [36]	dây) 89% (độ phủ đường dây), 90% (độ phủ nhánh)	ChatGPT	
	41% [7]	Không	ChatGPT	
	78%	áp dụng 87% (độ phủ đường dây), 92% (độ phủ chi	Bộ luật	[39]
	2% [39]	nhánh) 2% (độ phủ đường dây), 1% (độ phủ chi nhánh)	Bộ luật	

Lưu ý rằng, [39] các thí nghiệm với Codex, CodeGen và ChatGPT, và hiệu suất tốt nhất đạt được bởi Codex.

Dakhel et al. [38] đã giới thiệu MuTAP để cải thiện hiệu quả của các trường hợp thử nghiệm được tạo ra bởi LLM về việc phát hiện lại lỗi bằng cách tận dụng thử nghiệm đột biến. Họ tăng cường gợi ý với những dị nhân sống sót, vì những dị nhân đó làm nổi bật điều gì đó. Những hạn chế của các trường hợp thử nghiệm trong việc phát hiện lỗi. Zhang và cộng sự. [40] đã tạo ra các bài kiểm tra bảo mật với các phụ thuộc dễ bị tổn thương

với bảng Thạc sĩ Luật (LLM).

Yuan et al. [7] lần đầu tiên thực hiện một nghiên cứu thực nghiệm để đánh giá khả năng tạo kiểm thử đơn vị của ChatGPT với cả Phân tích định lượng và nghiên cứu người dùng về tính chính xác, đầy đủ, dễ đọc và khả năng sử dụng. Và kết quả. cho thấy các bài kiểm tra được tạo ra vẫn còn thiếu chính xác. các vấn đề, bao gồm nhiều lỗi biên dịch và thực thi khác nhau những thất bại. Họ cũng đề xuất một phương pháp tận dụng Chính ChatGPT cũng cần cải thiện chất lượng nội dung do nó tạo ra. các bài kiểm tra với một trình tạo kiểm tra ban đầu và một bộ tinh chỉnh kiểm tra lặp đi lặp lại. Cụ thể, bộ tinh chỉnh kiểm tra lặp đi lặp lại đã sửa đổi một cách lặp đi lặp lại. các lỗi biên dịch trong các bài kiểm tra được tạo ra bởi lần đầu tiên công cụ tạo kiểm thử, tuân theo mô hình xác thực và sửa lỗi. Thông báo lỗi biên dịch sẽ nhắc nhở LLM. và ngữ cảnh mã bổ sung. Guilhaume et al. [31] và Li et al. [41] đã đánh giá chất lượng của các bài kiểm tra đơn vị được tạo ra bởi LLM bằng cách sử dụng các số liệu khác nhau và khác nhau lời nhắc.

Tạo bộ kiểm thử kèm theo tài liệu bổ sung. Vikram et al. [34] đã tiến thêm một bước nữa bằng cách điều tra tiềm năng sử dụng LLM để tạo ra các bài kiểm tra dựa trên thuộc tính khi được cung cấp tài liệu API. Họ tin rằng Việc lập tài liệu về phương thức API có thể hỗ trợ LLM trong việc Tạo ra logic để tạo ra các đầu vào ngẫu nhiên cho phương pháp đó. và rút ra những thuộc tính có ý nghĩa của kết quả để kiểm tra. Thay vì tạo các bài kiểm tra đơn vị từ mã nguồn, Plein et al. [33] đã tạo ra các bài kiểm tra dựa trên lỗi do người dùng viết báo cáo.

LLM và phương pháp tìm kiếm để tạo kiểm thử đơn vị. Các nghiên cứu nêu trên sử dụng LLM cho toàn bộ đơn vị. nhiệm vụ tạo trường hợp thử nghiệm, trong khi Lemieux et al. [37] tập trung vào một hướng khác, tức là, trước tiên hãy để các kỹ thuật kiểm thử phần mềm dựa trên tìm kiếm truyền thống (ví dụ: Pynguin [137]) trong tạo trường hợp kiểm thử đơn vị cho đến khi đạt được sự cải thiện về phạm vi kiểm thử. bị đình trệ, sau đó yêu cầu LLM cung cấp các trường hợp kiểm thử ví dụ. đối với các chức năng chưa được bao phủ đầy đủ. Những ví dụ này có thể giúp ích cho... Quá trình tạo bài kiểm tra ban đầu chuyển hướng tìm kiếm của nó đến những thông tin hữu ích hơn. các khu vực của không gian tìm kiếm.

Tang et al. [8] tiến hành so sánh có hệ thống các bài kiểm tra các bộ được tạo bởi LLM và công cụ kiểm thử phần mềm dựa trên tìm kiếm hiện đại EvoSuite, bằng cách xem xét tính chính xác, khả năng đọc hiểu, độ phủ mã và khả năng phát hiện lỗi. Tương tự, Bhatia [43] điều tra thực nghiệm

chất lượng của các bài kiểm thử đơn vị được tạo ra bởi LLM so với một Công cụ tạo bài kiểm tra thường dùng Pynguin.

Hiệu năng của việc tạo trường hợp kiểm thử đơn vị. Vì Các nghiên cứu nói trên về việc tạo trường hợp kiểm thử đơn vị là Dựa trên các tập dữ liệu khác nhau, khó có thể đưa ra một kết luận công bằng. để so sánh và chúng tôi trình bày chi tiết trong Bảng 3 để Người đọc có được cái nhìn tổng quan. Chúng ta có thể thấy rằng trong Theo tiêu chuẩn SF110, cả ba mô hình LLM được đánh giá đều có điểm khá thấp. hiệu suất, tức là độ phủ 2% [39]. SF110 là một Evosuite (một kỹ thuật tạo trường hợp kiểm thử đơn vị dựa trên tìm kiếm) Bộ dữ liệu chuẩn bao gồm 111 dự án Java mã nguồn mở. Được tái xuống từ SourceForge, chứa 23.886 lớp, hơn 800.000 nhánh ở cấp độ mã byte và 6,6 triệu dòng mã. mã. Các tác giả không trình bày lý do chi tiết cho việc này. hiệu năng thấp, có thể được nghiên cứu sâu hơn trong phần tiếp theo. tương lai.

4.2 Kiểm thử quá trình tạo Oracle

Công cụ kiểm thử là nguồn thông tin về việc liệu... đầu ra của một hệ thống phần mềm (hoặc chương trình hoặc hàm hoặc...) phương pháp) là đúng hay không [138]. Hầu hết các nghiên cứu được thu thập Trong danh mục này, mục tiêu là tạo ra các khẳng định kiểm thử, đó là bên trong một trường hợp kiểm thử đơn vị. Tuy nhiên, chúng tôi đã chọn xử lý những điều này các nghiên cứu được chia thành các phần riêng biệt để tạo điều kiện thuận lợi cho việc nghiên cứu kỹ lưỡng hơn. Phân tích. Kiểm thử khẳng định, nhằm chỉ ra các vấn đề tiềm ẩn. Trong đoạn mã được kiểm thử, đây là một khía cạnh quan trọng có thể phân biệt các trường hợp kiểm thử đơn vị với mã thông thường. Đó là lý do tại sao Một số nghiên cứu tập trung cụ thể vào việc tạo ra các khẳng định kiểm thử hiệu quả. Trên thực tế, trước khi sử dụng LLM, các nhà nghiên cứu đã đề xuất các phương pháp dựa trên RNN nhằm mục đích học tập từ hàng ngàn phương pháp kiểm thử đơn vị để tạo ra các câu lệnh khẳng định có ý nghĩa [139], nhưng chỉ có 17% được tạo ra Các khẳng định có thể hoàn toàn trùng khớp với các khẳng định thực tế. Sau đó, để cải thiện hiệu suất, một số nhà nghiên cứu đã... đã sử dụng các mô hình LLM cho nhiệm vụ này.

Mastropaolo et al. [46], [132] đã huấn luyện trước một mô hình T5 trên một tập dữ liệu bao gồm văn bản tiếng Anh tự nhiên và mã nguồn. Sau đó, nó đã tinh chỉnh mô hình đó bằng cách tái sử dụng. Các tập dữ liệu được sử dụng trong bốn công trình nghiên cứu trước đây sử dụng các kỹ thuật học sâu (như RNN đã đề cập trước đó), bao gồm tạo khẳng định kiểm thử và sửa chữa chương trình, v.v. Kết quả cho thấy tỷ lệ khớp trích xuất của bài kiểm tra được tạo ra khẳng định là 57%. Tufano et al. [44] đề xuất một cách tiếp cận tương tự, trong đó đào tạo trước LLM riêng biệt với tiếng Anh đã sử dụng ngữ liệu và ngữ liệu mã, sau đó tinh chỉnh nó dựa trên các khẳng định. tập dữ liệu (bao gồm các phương pháp kiểm thử, phương pháp trọng tâm và các câu lệnh khẳng định). Hiệu suất được cải thiện hơn nữa, đạt 62% so với kết quả trùng khớp chính xác. tỷ lệ. Bên cạnh dữ liệu cấp độ cú pháp như các nghiên cứu trước đây, Nie et al. [45] đã tinh chỉnh LLM với sáu loại ngữ nghĩa mã dữ liệu, bao gồm kết quả thực thi (ví dụ: các kiểu của cục bộ) các biến) và ngữ cảnh thực thi (ví dụ: phương thức được gọi cuối cùng) (trong phương pháp thử nghiệm), cho phép LLM học cách hiểu thông tin thực thi mã. Tỷ lệ trùng khớp chính xác

là 17% (Lưu ý rằng bài báo này dựa trên một tập dữ liệu khác so với tất cả các nghiên cứu khác được đề cập trong chủ đề này).

Các nghiên cứu đã đề cập ở trên sử dụng lược đồ huấn luyện trước và tinh chỉnh khi sử dụng LLM, và với khả năng ngày càng mạnh mẽ của LLM, chúng có thể hoạt động tốt trên các nhiệm vụ cụ thể mà không cần các tập dữ liệu huấn luyện trước hoặc tinh chỉnh chuyên biệt này. Sau đó, Nashid et al. [47] đã sử dụng kỹ thuật gợi ý cho nhiệm vụ này và đề xuất một kỹ thuật tạo gợi ý tự động truy xuất các bản trình bày mã tương tự với nhiệm vụ, dựa trên phân tích nhúng hoặc tần suất. Họ cũng trình bày các đánh giá về học tập ít mẫu với số lượng khác nhau (ví dụ: không mẫu, một mẫu hoặc n mẫu) và các hình thức (ví dụ: ngẫu nhiên so với có hệ thống, hoặc có so với không có mô tả ngôn ngữ tự nhiên) của các gợi ý, để điều tra tính khả thi của nó trên việc tạo khẳng định kiểm thử. Chỉ với một vài bản trình bày mã có liên quan, phương pháp này có thể đạt được độ chính xác 76% cho các kết quả khớp chính xác trong việc tạo khẳng định kiểm thử, đây là hiệu suất tiên tiến nhất cho nhiệm vụ này.

4.3 Tạo dữ liệu đầu vào kiểm thử hệ thống Mục

này bao gồm các nghiên cứu liên quan đến việc tạo dữ liệu đầu vào kiểm thử hệ thống nhằm cho phép tự động hóa quá trình thực thi kiểm thử. Chúng tôi sử dụng ba tiểu mục để trình bày phân tích từ ba góc nhìn khác nhau, và mỗi nghiên cứu được thu thập có thể được phân tích trong một hoặc nhiều tiểu mục này.

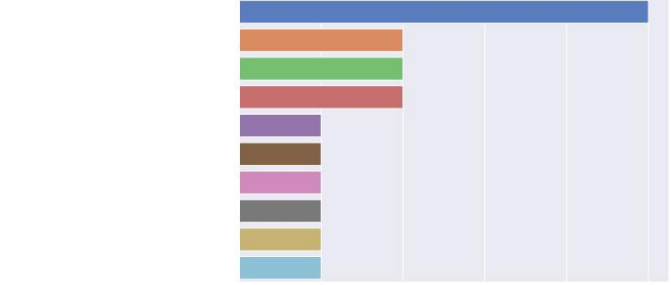
Phần phụ đầu tiên là tạo đầu vào theo loại phần mềm. Việc tạo đầu vào kiểm thử cấp hệ thống cho việc kiểm thử phần mềm khác nhau tùy thuộc vào loại phần mềm cụ thể đang được kiểm thử. Ví dụ, đối với các ứng dụng di động, việc tạo đầu vào kiểm thử yêu cầu cung cấp nhiều loại đầu vào văn bản hoặc tổ hợp thao tác khác nhau (ví dụ: nhấp vào nút, nhấn giữ danh sách) [14], [49], đây là chìa khóa để kiểm thử chức năng và giao diện người dùng của ứng dụng; trong khi đối với các thư viện Học sâu (DL), đầu vào kiểm thử là một chương trình bao gồm các API DL đa dạng [58], [59]. Phần phụ này sẽ trình bày cách LLM được sử dụng để tạo đầu vào cho các loại phần mềm khác nhau.

Phần thứ hai đề cập đến việc tạo ra dữ liệu đầu vào dưới góc độ các kỹ thuật kiểm thử. Chúng tôi nhận thấy rằng một số phương pháp nhất định đóng vai trò là các loại kỹ thuật kiểm thử cụ thể. Ví dụ, hàng chục nghiên cứu mà chúng tôi đã thu thập tập trung cụ thể vào việc sử dụng LLM cho kiểm thử mờ (fuzz testing). Do đó, phần này sẽ phân tích các nghiên cứu đã thu thập dưới góc độ các kỹ thuật kiểm thử, trình bày cách LLM được sử dụng để nâng cao các kỹ thuật kiểm thử truyền thống.

Phần thứ ba đề cập đến việc tạo ra dữ liệu đầu vào xét về cả đầu vào và đầu ra. Mặc dù hầu hết các nghiên cứu đã thu thập đều lấy mã nguồn hoặc chính phần mềm làm đầu vào và trực tiếp xuất ra kết quả kiểm thử của phần mềm, nhưng cũng có những nghiên cứu sử dụng các hình thức đầu vào và đầu ra thay thế. Phần này sẽ phân tích các nghiên cứu đó, làm nổi bật các phương pháp khác nhau và đặc điểm đầu vào-đầu ra của chúng.

4.3.1 Tạo dữ liệu đầu vào theo loại phần mềm Hình 5 minh

họa các loại phần mềm được thử nghiệm trong các nghiên cứu mà chúng tôi đã thu thập. Rõ ràng, loại nổi bật nhất là ứng dụng di động, với năm nghiên cứu sử dụng LLM để thử nghiệm, có thể là do sự phổ biến và tầm quan trọng của chúng.



Hình 5: Phân bố phần mềm được kiểm thử

Trong kinh doanh và cuộc sống hàng ngày hiện nay, LLM (Learning Learning Module) đóng vai trò quan trọng. Ngoài ra, có hai nghiên cứu tập trung vào việc kiểm thử các thư viện học sâu, trình biên dịch và bộ giải SMT. Hơn nữa, các kỹ thuật kiểm thử dựa trên LLM cũng đã được áp dụng cho các lĩnh vực như hệ thống vật lý mạng, nền tảng điện toán lượng tử, v.v. Việc áp dụng rộng rãi LLM chứng tỏ hiệu quả của chúng trong việc xử lý các đầu vào kiểm thử đa dạng và nâng cao các hoạt động kiểm thử trên nhiều lĩnh vực phần mềm khác nhau. Phân tích chi tiết được trình bày bên dưới.

Kiểm thử việc tạo đầu vào cho ứng dụng di động. Đối với việc kiểm thử ứng dụng di động, một khó khăn là tạo ra các đầu vào văn bản phù hợp để chuyển sang trang tiếp theo, điều này vẫn là một trở ngại nổi bật đối với phạm vi kiểm thử. Xem xét sự đa dạng và yêu cầu ngữ nghĩa của các đầu vào hợp lệ (ví dụ: giữ khởi hành chuyển bay, tên phim), các kỹ thuật truyền thống với các kỹ thuật dựa trên kinh nghiệm hoặc dựa trên ràng buộc [10], [140] còn xa mới tạo ra được đầu vào văn bản có ý nghĩa. Liu et al. [49] sử dụng LLM để tạo ra văn bản đầu vào ngữ nghĩa một cách thông minh theo ngữ cảnh GUI. Cụ thể, QTypist được đề xuất của họ tự động trích xuất thông tin thành phần liên quan đến EditText để tạo ra các lời nhắc, sau đó nhập các lời nhắc vào LLM để tạo ra văn bản đầu vào.

Ngoài việc nhập văn bản, còn có các hình thức nhập liệu khác cho ứng dụng di động, ví dụ như các thao tác 'nhấp vào nút' và 'chọn danh sách'. Để kiểm tra đầy đủ một ứng dụng, cần phải bao phủ nhiều trang GUI hơn và thực hiện các dấu vết khám phá có ý nghĩa hơn thông qua các thao tác GUI, tuy nhiên các nghiên cứu hiện có với các phương pháp ngẫu nhiên/dựa trên quy tắc [9], [10], các phương pháp dựa trên mô hình [11], [12] và các phương pháp dựa trên học máy [13] không thể hiểu thông tin ngữ nghĩa của trang GUI, do đó không thể lập kế hoạch theo dõi một cách hiệu quả. Liu et al. [14] xây dựng bài toán tạo đầu vào thử nghiệm GUI di động dưới dạng nhiệm vụ Hỏi & Đáp, yêu cầu LLM trò chuyện với các ứng dụng di động bằng cách chuyển thông tin trang GUI cho LLM để thu thập các kịch bản thử nghiệm (tức là thao tác GUI) và thực hiện chúng để tiếp tục chuyển phản hồi của ứng dụng cho LLM, lặp lại toàn bộ quá trình. GPTDroid được đề xuất trích xuất ngữ cảnh tĩnh của trang giao diện người dùng (GUI) và ngữ cảnh động của quy trình kiểm thử lặp đi lặp lại, đồng thời thiết kế các lời nhắc để nhập thông tin này vào LLM, giúp LLM hiểu rõ hơn về trang GUI cũng như toàn bộ quy trình kiểm thử. Nó cũng giới thiệu một cơ chế nhắc nhở bộ nhớ nhận biết chức năng, trang bị cho LLM khả năng lưu giữ kiến thức kiểm thử của toàn bộ quy trình và thực hiện suy luận dựa trên chức năng dài hạn để hướng dẫn quá trình khám phá. Tương tự, Zimmermann et al. sử dụng LLM để

diễn giải các trường hợp thử nghiệm ngôn ngữ tự nhiên và điều hướng theo chương trình thông qua ứng dụng đang được thử nghiệm [54].

Yu et al. [61] điều tra khả năng của LLM trong việc tạo và di chuyển kịch bản kiểm thử ứng dụng di động, bao gồm việc tạo kiểm thử dựa trên kịch bản và di chuyển kiểm thử ứng dụng/nền tảng chéo.

Tạo đầu vào thử nghiệm cho thư viện DL. Đầu vào để thử nghiệm thư viện DL là các chương trình DL, và khó khăn trong việc tạo ra các chương trình DL đầu vào đa dạng là chúng cần phải đáp ứng cả cú pháp/ngữ nghĩa của ngôn ngữ đầu vào (ví dụ: Python) và các ràng buộc về hình dạng/đầu vào API cho các phép tính tensor. Các kỹ thuật truyền thống với làm mờ ở cấp độ API [141], [142] hoặc làm mờ ở cấp độ mô hình [143], [144] gặp phải những hạn chế sau: 1) thiếu chuỗi API đa dạng, do đó không thể phát hiện ra lỗi do các chuỗi API nối tiếp gây ra; 2) không thể tạo mã tùy ý, do đó không thể khám phá không gian tìm kiếm khổng lồ tồn tại khi sử dụng thư viện DL. Vì LLM có thể bao gồm nhiều đoạn mã gọi API thư viện DL trong tập dữ liệu huấn luyện của chúng, chúng có thể học ngầm cả cú pháp/ngữ nghĩa của ngôn ngữ và các ràng buộc API phức tạp để tạo ra chương trình DL hợp lệ.

Theo nghĩa này, Deng et al. [59] đã sử dụng cả LLM tạo sinh và LLM điền vào để tạo và biến đổi các chương trình DL đầu vào hợp lệ/đa dạng để làm mờ các thư viện DL. Cụ thể, trước tiên nó sử dụng một LLM tạo sinh (CodeX) để tạo ra một tập hợp các chương trình hạt giống (tức là các đoạn mã sử dụng API DL mục tiêu). Sau đó, nó thay thế một phần chương trình hạt giống bằng các mã thông báo được che giấu bằng cách sử dụng các toán tử đột biến khác nhau và tận dụng khả năng của LLM lấp đầy (InCoder) để thực hiện lấp đầy mã nhằm tạo ra mã mới thay thế các mã thông báo được che giấu. Nghiên cứu tiếp theo của họ [58] tiến thêm một bước nữa để chuẩn bị LLM nhằm tổng hợp các chương trình bất thường cho các thư viện DL mờ. Nó được xây dựng dựa trên giả thuyết nổi tiếng rằng các chương trình gây ra lỗi trong quá khứ có thể bao gồm các thành phần mã hiếm/có giá trị, rất quan trọng cho việc tìm lỗi và cho thấy hiệu suất phát hiện lỗi được cải thiện.

Kiểm thử việc tạo dữ liệu đầu vào cho các loại phần mềm khác. Ngoài ra còn có hàng tá nghiên cứu về các nhiệm vụ kiểm thử trong nhiều lĩnh vực khác nhau; do hạn chế về không gian, chúng tôi sẽ trình bày một số nghiên cứu tiêu biểu trong các lĩnh vực này.

Việc tìm lỗi trong một công cụ phát triển hệ thống vật lý-mạng (CPS) thương mại như Simulink thậm chí còn khó khăn hơn. Với sự phức tạp của ngôn ngữ Simulink, việc tạo ra các tệp mô hình Simulink hợp lệ để thử nghiệm là một nhiệm vụ đầy tham vọng đối với các kỹ thuật học máy truyền thống hoặc học sâu. Shrestha et al. [51] sử dụng một tập hợp nhỏ dữ liệu huấn luyện dành riêng cho Simulink để tinh chỉnh LLM nhằm tạo ra các mô hình Simulink. Kết quả cho thấy rằng nó có thể tạo ra các mô hình Simulink khá giống với các mô hình mã nguồn mở và có thể tìm thấy một tập hợp lớn hơn các lỗi mà các phương pháp làm mờ truyền thống đã tìm thấy.

Sun et al. [63] sử dụng LLM để tạo ra các công thức kiểm thử cho việc làm mờ các bộ giải SMT. Họ huấn luyện lại các LLM trên một kho dữ liệu lớn gồm các công thức SMT để cho phép chúng có được kiến thức miền cụ thể về SMT. Sau đó, họ tiếp tục tinh chỉnh các LLM trên các công thức gây lỗi trong quá khứ, được biết là liên quan đến các cấu trúc có nhiều khả năng gây ra lỗi và các hành vi cụ thể của bộ giải. Bộ làm mờ trình biên dịch dựa trên LLM do Yang et al. [69] đề xuất áp dụng khung mô hình kép: (1) một LLM phân tích kiểm tra mã nguồn tối ưu hóa cấp thấp và đưa ra các yêu cầu về

các chương trình kiểm thử cấp cao có thể kích hoạt quá trình tối ưu hóa; (2) LLM tạo ra các chương trình kiểm thử dựa trên các yêu cầu tóm tắt. Ye et al. [48] sử dụng LLM để tạo ra các chương trình JavaScript và sau đó sử dụng các đặc tả ECMAScript có cấu trúc tốt để tự động tạo dữ liệu kiểm thử cùng với các chương trình kiểm thử, sau đó họ áp dụng kiểm thử khác biệt để phát hiện lỗi.

4.3.2 Tạo dữ liệu đầu vào trong các kỹ thuật kiểm thử Bằng cách sử dụng dữ liệu đầu vào kiểm thử hệ thống được tạo ra bởi LLM, các nghiên cứu được tổng hợp nhằm mục đích nâng cao các kỹ thuật kiểm thử truyền thống và làm cho chúng hiệu quả hơn. Trong số các kỹ thuật này, kiểm thử mờ (fuzz testing) là kỹ thuật được sử dụng phổ biến nhất. Kiểm thử mờ, về mặt khái niệm chung, xoay quanh việc tạo ra dữ liệu không hợp lệ, không mong đợi hoặc ngẫu nhiên làm đầu vào để đánh giá hành vi của phần mềm. LLM đóng vai trò quan trọng trong việc cải thiện kiểm thử mờ truyền thống bằng cách tạo điều kiện thuận lợi cho việc tạo ra dữ liệu đầu vào đa dạng và thực tế. Điều này cho phép kiểm thử mờ phát hiện ra các lỗi tiềm ẩn trong phần mềm bằng cách đưa nó vào nhiều kịch bản đầu vào khác nhau. Ngoài kiểm thử mờ, LLM cũng góp phần nâng cao các kỹ thuật kiểm thử khác, sẽ được thảo luận chi tiết hơn sau đây.

Khung fuzzing phổ quát. Xia et al. [67] trình bày Fuzz4All có thể nhắm mục tiêu nhiều ngôn ngữ đầu vào khác nhau và nhiều tính năng khác nhau của các ngôn ngữ này. Ý tưởng chính đằng sau nó là tận dụng LLM làm công cụ tạo và đột biến đầu vào, cho phép phương pháp này tạo ra các đầu vào đa dạng và thực tế cho bất kỳ ngôn ngữ nào có liên quan đến thực tế. Để hiện thực hóa tiềm năng này, họ trình bày một kỹ thuật tự động nhắc nhở mới, tạo ra các lời nhắc LLM phù hợp cho fuzzing, và một vòng lặp fuzzing được hỗ trợ bởi LLM mới, cập nhật lời nhắc lặp đi lặp lại để tạo ra các đầu vào fuzzing mới. Họ thử nghiệm với sáu ngôn ngữ khác nhau (C, C++, Go, SMT2, Java và Python) làm đầu vào và chứng minh phạm vi bao phủ cao hơn so với các công cụ fuzzing dành riêng cho ngôn ngữ hiện có. Hu et al. [52] đề xuất một công cụ fuzzing hộp xám được tăng cường bởi LLM, chọn một hạt giống trong nhóm hạt giống của công cụ fuzzing và nhắc LLM tạo ra các hạt giống đột biến có thể kích hoạt một vùng mã mới của phần mềm. Họ thử nghiệm với ba loại định dạng đầu vào, tức là các tệp dữ liệu được định dạng (ví dụ: json, xml), mã nguồn trong các ngôn ngữ lập trình khác nhau (ví dụ: JS, SQL, C), văn bản không có quy tắc cú pháp rõ ràng (ví dụ: phản hồi HTTP, tổng kiểm tra md5). Ngoài ra, việc làm mờ hiệu quả phụ thuộc vào trình điều khiển làm mờ hiệu quả, và Zhang et al. [66] sử dụng LLM trong việc tạo trình điều khiển làm mờ, trong đó năm chiến lược truy vấn được thiết kế và phân tích từ cơ bản đến nâng cao.

Các kỹ thuật làm mờ cho phần mềm cụ thể. Có những nghiên cứu tập trung vào các kỹ thuật làm mờ được thiết kế riêng cho phần mềm cụ thể, ví dụ: thư viện học sâu [58], [59], trình biên dịch [69], bộ giải SMT [63], tiện ích nhập liệu của ứng dụng di động [65], hệ thống mạng vật lý [51], v.v. Một trọng tâm chính của các kỹ thuật làm mờ này là tạo ra các đầu vào thử nghiệm đa dạng để đạt được độ phủ cao hơn. Điều này thường đạt được bằng cách kết hợp kỹ thuật đột biến với tạo dựa trên LLM, trong đó kỹ thuật trước tạo ra nhiều ứng viên khác nhau trong khi kỹ thuật sau chịu trách nhiệm tạo ra các đầu vào thử nghiệm có thể thực thi [59], [63]. Một trọng tâm khác của các kỹ thuật làm mờ này là tạo ra các đầu vào thử nghiệm rủi ro có thể kích hoạt lỗi sớm hơn. Để đạt được điều này, một thực tiễn phổ biến là

thu thập các chương trình kích hoạt lỗi lịch sử để tinh chỉnh LLM [63] hoặc coi chúng như các bản trình diễn khi truy vấn LLM [58], [65].

Các kỹ thuật thử nghiệm khác. Có những nghiên cứu sử dụng LLM để tăng cường thử nghiệm GUI nhằm tạo ra đầu vào văn bản có ý nghĩa [49] và dấu vết khám phá hướng chức năng [14], đã được giới thiệu trong phần Tạo đầu vào thử nghiệm cho ứng dụng di động của Mục 4.3.1.

Ngoài ra, Deng et al. [62] tận dụng LLM để thực hiện các nhiệm vụ kiểm thử xâm nhập một cách tự động. Việc này bao gồm thiết lập mục tiêu kiểm thử xâm nhập cho LLM, yêu cầu nó thực hiện thao tác thích hợp, triển khai nó trong môi trường thử nghiệm và đưa đầu ra thử nghiệm trở lại LLM để suy luận bước tiếp theo.

4.3.3 Tạo đầu vào về mặt đầu vào và đầu ra Các định dạng

đầu ra khác của việc tạo thử nghiệm. Mặc dù hầu hết các công trình sử dụng LLM để tạo trực tiếp các trường hợp thử nghiệm, nhưng cũng có một số công trình tạo ra các đầu vào gián tiếp như mã thử nghiệm, kịch bản thử nghiệm, mối quan hệ biến đổi, v.v. Liu et al. [65] đề xuất InputBlaster, tận dụng LLM để tự động tạo ra các đầu vào văn bản bất thường để làm mờ các tiện ích nhập văn bản trong ứng dụng di động. Nó xây dựng văn đề tạo đầu vào bất thường như một nhiệm vụ tạo ra một tập hợp các trình tạo thử nghiệm, mỗi trình tạo có thể tạo ra một loạt các đầu vào văn bản bất thường theo cùng một quy tắc đột biến. Cụ thể, InputBlaster tận dụng LLM để tạo ra các trình tạo thử nghiệm cùng với các quy tắc đột biến đóng vai trò là chuỗi suy luận và sử dụng lược đồ học tập theo ngữ cảnh để chứng minh LLM bằng các ví dụ nhằm nâng cao hiệu suất. Deng et al. [64] sử dụng LLM để trích xuất thông tin chính liên quan đến kịch bản thử nghiệm từ một quy tắc giao thông và biểu diễn thông tin được trích xuất trong lược đồ kịch bản thử nghiệm, sau đó tổng hợp các kịch bản tương ứng để xây dựng kịch bản thử nghiệm.

Luu et al. [56] kiểm tra hiệu quả của LLM trong việc tạo ra các mối quan hệ biến chất (MR) để thử nghiệm biến chất. Kết quả của họ cho thấy ChatGPT có thể được sử dụng để nâng cao trí tuệ kiểm thử phần mềm bằng cách đề xuất các ứng viên MR có thể được điều chỉnh sau này để thực hiện các bài kiểm thử, nhưng trí tuệ con người vẫn cần phải tham gia để xác nhận và sửa chữa tính đúng đắn của chúng.

Định dạng đầu vào khác của việc tạo kiểm thử. Các nghiên cứu đã đề cập chủ yếu sử dụng mã nguồn hoặc phần mềm làm đầu vào cho LLM, tuy nhiên cũng có những nghiên cứu sử dụng mô tả bằng ngôn ngữ tự nhiên làm đầu vào cho việc tạo kiểm thử.

Mathur et al. [53] đề xuất tạo ra các trường hợp thử nghiệm từ các yêu cầu được mô tả bằng ngôn ngữ tự nhiên. Ackerman et al. [60] tạo ra các trường hợp từ các yêu cầu được mô tả bằng ngôn ngữ tự nhiên một cách đệ quy để làm ví dụ ban đầu cho một công cụ làm mờ đột biến.

4.4 Phân tích lỗi

Danh mục này bao gồm việc phân tích và phân loại các lỗi phần mềm đã được xác định để nâng cao hiểu biết về lỗi, và tạo điều kiện thuận lợi cho việc gỡ lỗi và sửa chữa lỗi sau đó. Mukher-jeet et al. [73] tạo ra các câu trả lời liên quan cho các câu hỏi tiếp theo đối với các báo cáo lỗi thiếu sót để tạo điều kiện thuận lợi cho việc phân loại lỗi. Su et al. [76] chuyển đổi việc phân loại thành phần lỗi thành một nhiệm vụ phân loại đa cấp và một nhiệm vụ tạo ra bằng LLM, sau đó kết hợp các kết quả dự đoán từ chúng để cải thiện

hiệu suất của việc phân loại lỗi-thành phần tiếp theo. Zhang et al. [72] trước tiên tận dụng LLM trong thiết lập không có dữ liệu huấn luyện để lấy thông tin cần thiết về báo cáo lỗi, sau đó sử dụng thông tin cần thiết làm đầu vào để phát hiện các báo cáo lỗi trùng lặp. Mahbub et al. [74] đề xuất giải thích các lỗi phần mềm bằng LLM, tạo ra các giải thích bằng ngôn ngữ tự nhiên cho các lỗi phần mềm bằng cách học từ một kho ngữ liệu lớn về các cam kết sửa lỗi. Zhang et al. [70] nhắm mục tiêu tự động tạo tiêu đề lỗi từ mô tả lỗi, nhằm giúp các nhà phát triển viết tiêu đề sự cố và tạo điều kiện thuận lợi cho quá trình phân loại lỗi và theo dõi sửa chữa.

4.5 Gỡ lỗi

Mục này đề cập đến quá trình xác định và định vị nguyên nhân của một vấn đề phần mềm (tức là lỗi). Nó bao gồm việc phân tích mã, theo dõi luồng thực thi, thu thập thông tin lỗi để hiểu nguyên nhân gốc rễ của vấn đề và khắc phục vấn đề. Một số nghiên cứu tập trung vào quy trình gỡ lỗi toàn diện, trong khi những nghiên cứu khác đi sâu vào các hoạt động phụ cụ thể trong quy trình.

Khung gỡ lỗi tổng thể. Bui et al. [77] đề xuất một khung Phát hiện-Định vị-Sửa chữa thống nhất dựa trên LLM để gỡ lỗi, trước tiên xác định xem một đoạn mã nhất định có bị lỗi hay không, sau đó xác định các dòng bị lỗi và dịch mã bị lỗi sang phiên bản đã sửa. Kang et al. [83] đề xuất gỡ lỗi khoa học tự động, một kỹ thuật mà khi được cung cấp mã bị lỗi và một bài kiểm tra tiết lộ lỗi, sẽ nhắc LLM tự động tạo ra các giả thuyết, sử dụng trình gỡ lỗi để tương tác tích cực với mã bị lỗi và do đó tự động đưa ra kết luận trước khi tạo bản vá. Chen et al. [88] chứng minh rằng việc tự gỡ lỗi có thể dạy LLM thực hiện gỡ lỗi vẹt cao su; tức là, không cần bất kỳ phản hồi nào của con người về tính đúng đắn của mã hoặc thông báo lỗi, mô hình có thể xác định lỗi của nó bằng cách điều tra kết quả thực thi và giải thích mã được tạo ra bằng ngôn ngữ tự nhiên. Cao et al. [89] tiến hành một nghiên cứu về khả năng gỡ lỗi của LLM đối với các chương trình học sâu, bao gồm phát hiện lỗi, định vị lỗi và sửa chữa chương trình.

Định vị lỗi. Wu et al. [85] so sánh hai LLM (ChatGPT và GPT-4) với các kỹ thuật định vị lỗi hiện có và điều tra tính nhất quán của LLM trong việc định vị lỗi, cũng như cách kỹ thuật nhanh chóng và độ dài ngữ cảnh mã ảnh hưởng đến kết quả. Kang et al. [79] đề xuất AutoFL, một kỹ thuật định vị lỗi tự động chỉ yêu cầu một bài kiểm tra thất bại duy nhất, và trong quá trình định vị lỗi, nó cũng tạo ra lời giải thích về lý do tại sao bài kiểm tra đã cho thất bại. Yang et al. [84] đề xuất LLMAO để khắc phục bản chất từ trái sang phải của LLM bằng cách tinh chỉnh một tập hợp nhỏ các lớp bộ điều hợp hai chiều trên các biểu diễn được học bởi LLM, có thể định vị các dòng mã bị lỗi mà không cần bất kỳ thông tin về phạm vi kiểm tra nào. Tu et al. [86] đề xuất LLM4CBI để chế ngự LLM nhằm tạo ra các chương trình kiểm tra hiệu quả để tìm các tệp đáng ngờ.

Tái tạo lỗi. Cũng có những nghiên cứu tập trung vào một giai đoạn phụ của quá trình gỡ lỗi. Ví dụ, Kang et al. [78] và Plein et al. [81] lần lượt đề xuất khung để khai thác LLM nhằm tái tạo lỗi và đề xuất các trường hợp thử nghiệm tái tạo lỗi cho nhà phát triển để hỗ trợ gỡ lỗi. Li et al. [87] tập trung vào một khía cạnh tương tự là tìm các trường hợp thử nghiệm gây lỗi mà đầu vào thử nghiệm có thể kích hoạt

Lỗi là do phần mềm. Nó kết hợp một cách hiệu quả LLM và kiểm thử vi sai để thực hiện điều đó.

Cũng có những nghiên cứu tập trung vào việc tái tạo lỗi của ứng dụng di động để tạo ra kịch bản phát lại. Feng et al. [75] đề xuất AdbGPT, một phương pháp nhẹ mới để tự động tái tạo lỗi từ báo cáo lỗi thông qua kỹ thuật nhắc nhở, mà không cần đào tạo và nỗ lực mã hóa cứng. Nó tận dụng học tập ít mẫu và suy luận chuỗi suy nghĩ để khơi gợi kiến thức của con người và suy luận logic từ LLM để thực hiện phát lại lỗi theo cách tương tự như một nhà phát triển. Huang et al. [71] đề xuất CrashTranslator để tự động tái tạo lỗi trực tiếp từ dấu vết ngăn xếp.

Nó thực hiện điều này bằng cách tận dụng LLM để dự đoán các bước khám phá gây ra sự cố và thiết kế một kỹ thuật dựa trên học tăng cường để giảm thiểu dự đoán không chính xác và hướng dẫn tìm kiếm một cách toàn diện. Taeb et al. [55] chuyển đổi các hướng dẫn kiểm tra khả năng truy cập thủ công thành các video có thể phát lại và điều hướng được bằng cách sử dụng LLM và các mô hình phát hiện phần tử UI, điều này cũng có thể giúp tiết lộ các vấn đề về khả năng truy cập.

Giải thích lỗi. Taylor et al. [82] tích hợp LLM vào Trình biên dịch C gỡ lỗi để tạo ra các giải thích độc đáo, tập trung vào người mới bắt đầu được điều chỉnh cho từng lỗi. Widjojo et al. [80] nghiên cứu hiệu quả của Stack Overflow và LLM trong việc giải thích lỗi trình biên dịch.

4.6 Sửa chữa chương trình

Mục này đề cập đến nhiệm vụ khắc phục các lỗi phần mềm đã được xác định. Tần suất cao của các nghiên cứu liên quan đến sửa chữa có thể là do mối quan hệ chặt chẽ giữa nhiệm vụ này và mã nguồn. Với khả năng xử lý và hiểu ngôn ngữ tự nhiên tiên tiến, LLM được trang bị tốt để xử lý và phân tích mã nguồn, trở thành công cụ lý tưởng để thực hiện các nhiệm vụ liên quan đến mã nguồn như sửa lỗi.

Đã có các kỹ thuật sửa chữa chương trình tự động dựa trên mẫu [145], dựa trên kinh nghiệm [146] và dựa trên ràng buộc [147], [148]. Và với sự phát triển của các kỹ thuật học sâu trong vài năm qua, đã có một số nghiên cứu sử dụng các kỹ thuật học sâu để sửa chữa chương trình. Chúng thường áp dụng các mô hình học sâu để lấy một chương trình phần mềm bị lỗi làm đầu vào và tạo ra một chương trình đã được vá lỗi. Dựa trên dữ liệu huấn luyện, họ sẽ xây dựng một mô hình mạng nơ-ron học được mối quan hệ giữa mã bị lỗi và mã đã sửa tương ứng.

Tuy nhiên, những kỹ thuật này vẫn không thể khắc phục được phần lớn lỗi, và chúng thường phải tạo ra hàng trăm đến hàng nghìn bản vá lỗi tiềm năng và mất hàng giờ để xác thực các bản vá này nhằm khắc phục đủ số lỗi. Hơn nữa, các mô hình sửa chữa chương trình dựa trên học sâu cần được huấn luyện với lượng lớn dữ liệu huấn luyện được gắn nhãn (thường là các cặp mã bị lỗi và mã đã được sửa), điều này tốn nhiều thời gian và công sức để thu thập bộ dữ liệu chất lượng cao. Do đó, với sự phổ biến và khả năng đã được chứng minh của các mô hình học máy tuyến tính (LLM), các nhà nghiên cứu bắt đầu khám phá việc ứng dụng LLM trong sửa chữa chương trình.

Và các lỗi chỉ trên một dòng. Trong giai đoạn đầu của việc sửa chữa chương trình, trọng tâm chủ yếu là giải quyết các lỗi liên quan đến mã lệnh trên một dòng, vốn tương đối đơn giản và không yêu cầu sửa chữa logic chương trình phức tạp. Lajko và cộng sự

al. [95] đề xuất tinh chỉnh LLM bằng các đoạn mã JavaScript để phục vụ mục đích sửa chữa chương trình JavaScript. Zhang et al. [116] sử dụng cắt lát chương trình để trích xuất thông tin ngữ cảnh liên quan trực tiếp đến câu lệnh bị lỗi đã cho làm thành phần sửa chữa từ đồ thị phụ thuộc chương trình tương ứng, giúp việc tinh chỉnh tập trung hơn vào mã bị lỗi. Zhang et al. [121] đề xuất một khung STEAM theo từng giai đoạn để vá các lỗi một dòng, mô phỏng hành vi tương tác của nhiều lập trình viên tham gia quản lý lỗi, ví dụ: báo cáo lỗi, chẩn đoán lỗi, tạo bản vá và xác minh bản vá.

Vì hầu hết các lỗi trong thực tế đều liên quan đến nhiều dòng mã, và các nghiên cứu sau này sẽ khám phá những tình huống phức tạp hơn này (mặc dù một số nghiên cứu cũng có thể vá các lỗi chỉ có một dòng mã).

Vá lỗi nhiều dòng. Các nghiên cứu trong danh mục này sẽ đưa một hàm bị lỗi vào LLM, và mục tiêu là xuất ra hàm đã được vá, có thể liên quan đến hiểu biết ngữ nghĩa phức tạp, sửa đổi khối mã, cũng như tái cấu trúc chương trình. Các nghiên cứu trước đây thường sử dụng chiến lược tinh chỉnh để cho phép LLM hiểu rõ hơn ngữ nghĩa của mã. Fu et al. [123] tinh chỉnh LLM bằng cách sử dụng mã hóa BPE để xử lý các vấn đề ngoài từ vựng (OOV), điều này khiến phương pháp tạo ra các to-ken mới không bao giờ xuất hiện trong hàm huấn luyện nhưng được giới thiệu mới trong quá trình sửa chữa. Wang et al. [120] huấn luyện LLM dựa trên cả đầu vào bị lỗi và các ví dụ sửa lỗi được truy xuất dựa trên sự tương đồng về từ vựng và ngữ nghĩa. Các nghiên cứu đã đề cập ở trên (bao gồm cả những nghiên cứu về vá lỗi một dòng) sẽ dự đoán trực tiếp các chương trình đã sửa, và Hu et al. [92] sử dụng một thiết lập khác dự đoán các tập lệnh có thể sửa lỗi khi được thực thi với ngữ pháp xóa và chèn. Ví dụ, nó dự đoán liệu một dòng mã gốc có nên bị xóa hay không và nội dung nào nên được chèn vào.

Tuy nhiên, việc tinh chỉnh có thể gặp phải những hạn chế về mặt phụ thuộc vào lượng dữ liệu được gắn nhãn chất lượng cao dồi dào, tài nguyên tính toán đáng kể và khả năng quá khớp. Để tiếp cận vấn đề sửa chữa chương trình hiệu quả hơn, các nghiên cứu sau này tập trung vào cách thiết kế một lời nhắc hiệu quả cho việc sửa chữa chương trình. Một số nghiên cứu điều tra thực nghiệm hiệu quả của các biến thể lời nhắc của LLM mới nhất để sửa chữa chương trình trong các thiết lập sửa chữa khác nhau và các tiêu chuẩn thường được sử dụng (sẽ được khám phá sâu hơn sau này), trong khi các nghiên cứu khác tập trung vào việc đề xuất các kỹ thuật mới. Ribeiro et al. [109] tận dụng LLM để thực hiện hoàn thành mã trong một dòng bị lỗi để tạo bản vá và trình bày chi tiết cách khắc phục tính chất mở của việc tạo mã để phù hợp với mã mới trong chương trình gốc. Xia et al. [115] đề xuất phương pháp sửa chữa chương trình dựa trên hội thoại, xen kẽ việc tạo bản vá với phản hồi tức thì để thực hiện sửa chữa theo kiểu hội thoại. Đầu tiên, họ cung cấp cho LLM thông tin lỗi kiểm thử có liên quan, sau đó học hỏi từ cả những thất bại và thành công của các bản vá lỗi trước đó đối với cùng một lỗi để sửa chữa mạnh mẽ hơn. Đối với các bản vá trước đó không vượt qua tất cả các bài kiểm tra, chúng sẽ kết hợp các bản vá không chính xác với thông tin lỗi kiểm tra tương ứng để tạo ra một lời nhắc mới cho LLM tạo ra bản vá tiếp theo, nhằm tránh mắc phải những lỗi tương tự. Đối với các bản vá trước đó

BẢNG 4: Hiệu suất sửa chữa chương trình

Bộ dữ	% Bản vá chính xác	Thạc sĩ Luật	Giấy
liệu Defects4J v1.2, Defects4J v2.0, QuixBugs, HumanEval-Java	22/40 lỗi Java (bộ dữ liệu QuixBugs, với InCoder-6B, chính xác) (cài đặt điền mã)	PLBART, CodeT5, CodeGen, In-Bộ mã hóa (mỗi bộ có các tham số khác nhau, tổng cộng 10 LLM)	[113]
QuixBugs	23/40 lỗi Python, 14/40 lỗi Java (cài đặt tạo hàm hoàn chỉnh)	Bộ luật 12B	[100]
Defects4J v1.2, Defects4J v2.0, QuixBugs, Many-Bugs	39/40 lỗi Python, 34/40 lỗi Java (bộ dữ liệu QuixBugs, với Codex-12B, cài đặt điền mã chính xác); 37/40 lỗi Python, 32/40 lỗi Java (bộ dữ liệu QuixBugs, với Codex-12B, đầy đủ) (cài đặt tạo chức năng)	Codex, GPT-Neo, CodeT5, In-Bộ mã hóa (mỗi bộ có các tham số khác nhau, tổng cộng 9 LLM)	[93]
QuixBugs	31/40 lỗi Python (cài đặt tạo hàm hoàn thành)	ChatGPT-175B	[96]
DL các chương trình từ Stack-Overflow	16/72 lỗi Python (cài đặt tạo hàm hoàn chỉnh)	ChatGPT-175B	[89]

Lưu ý rằng, đối với các nghiên cứu sử dụng nhiều tập dữ liệu hoặc mô hình tuyến tính hỗn hợp (LLM), chúng tôi chỉ trình bày kết quả tốt nhất hoặc trên tập dữ liệu được sử dụng phổ biến nhất.

Các bản vá đã vượt qua tất cả các bài kiểm tra (tức là các bản vá khả thi), Họ còn yêu cầu LLM tạo ra các biến thể thay thế khác. của các bản vá lỗi khả thi ban đầu. Điều này có thể tiếp tục được xây dựng dựa trên... và học hỏi từ những thành công trước đó để tạo ra những ý tưởng khả thi hơn. và lỗi để tăng khả năng có được các bản vá chính xác. Zhang et al. [94] đề xuất một cách tiếp cận tương tự được thiết kế bởi tận dụng các gợi ý đa phương thức (ví dụ: ngôn ngữ tự nhiên) mô tả, thông báo lỗi, các trường hợp kiểm thử dựa trên đầu vào-đầu ra), truy vấn lặp đi lặp lại, lựa chọn ít mẫu dựa trên trường hợp thử nghiệm để sản xuất các bản sửa chữa. Moon et al. [102] đề xuất sửa lỗi với phản hồi. Nó bao gồm một mô hình phê bình để tạo ra Phản hồi, một trình biên tập để chỉnh sửa mã dựa trên phản hồi đó. và một bộ chọn phản hồi để chọn phản hồi tốt nhất có thể. Từ nhà phê bình.

Wei et. al. [103] đề xuất Repilot để hỗ trợ AI “copi-lots” (tức là LLM) bằng cách tổng hợp các bản vá hợp lệ hơn trong quá trình quá trình sửa chữa. Điểm mấu chốt là nhiều mô hình LLM tạo ra đầu ra theo kiểu tự hồi quy (tức là từng token một), và bằng cách Tương tự như các chương trình viết của con người, quá trình sửa chữa có thể được tăng cường và hướng dẫn đáng kể thông qua một công cụ hoàn thiện. Brownlee et al. [105] đề xuất sử dụng LLM như một đột biến các toán tử cho các kỹ thuật sửa chữa chương trình dựa trên tìm kiếm.

Sửa chữa bằng trình phân tích mã tĩnh. Hầu hết các chương trình Các nghiên cứu sửa chữa sẽ dựa trên giả định rằng lỗi đã được phát hiện. trong khi Jin et al. [114] đề xuất một khung sửa chữa chương trình kết hợp với một công cụ phân tích tĩnh để phát hiện lỗi trước tiên, và sau đó sửa chữa chúng. Cụ thể, bộ phân tích tĩnh trước tiên phát hiện ra một lỗi (ví dụ: truy cập con trỏ null) và thông tin ngữ cảnh do trình phân tích tĩnh cung cấp sẽ được gửi vào LLM đã yêu cầu vá lỗi cụ thể này. Wadhwa et al. [110] tập trung vào một nhiệm vụ tương tự và bổ sung thêm việc sử dụng Bảng Thạc sĩ Luật (LLM) được sử dụng làm tiêu chí xếp hạng để đánh giá khả năng được chấp nhận. các bản vá được tạo ra có thể nắm bắt hiệu quả những điều khả thi nhưng lại sửa lỗi sai và giảm bớt gánh nặng cho nhà phát triển.

Sửa chữa các lỗi cụ thể. Các nghiên cứu đã đề cập ở trên tất cả đều coi đoạn mã lỗi là đầu vào cho quá trình tự động. sửa chữa chương trình, trong khi các nghiên cứu khác tiến hành sửa chữa chương trình dựa trên các loại mô tả lỗi khác, cụ thể các loại lỗi, v.v. Fakhoury et al. [122] tập trung vào chương trình Sửa lỗi dựa trên mô tả sự cố bằng ngôn ngữ tự nhiên, tức là tạo bản vá với thông tin về lỗi và cách khắc phục. được mô tả trong các báo cáo vấn đề. Garg et al. [119] nhằm mục đích ghép nối lại các vấn đề về hiệu suất, trong đó họ trước tiên truy xuất một hướng dẫn nhanh chóng từ cơ sở kiến thức được xây dựng sẵn các bản sửa lỗi hiệu suất trước đó và sau đó tạo lại lời nhắc ghép nối bằng cách sử dụng hướng dẫn đã được truy xuất. Có những nghiên cứu tập trung vào việc sửa lỗi của các chương trình Rust [108] hoặc

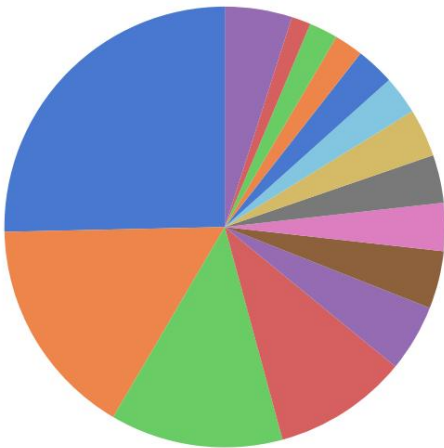
Chương trình OCaml (một ngôn ngữ lập trình mạnh mẽ trong công nghiệp) [111].

Nghiên cứu thực nghiệm về sửa chữa chương trình. Có một số các nghiên cứu liên quan đến đánh giá thực nghiệm hoặc thử nghiệm của các LLM khác nhau về sửa chữa chương trình, và chúng tôi tóm tắt hiệu suất trong Bảng 4. Jiang et al. [113], Xia et al. [93] và Zhang et al. [118] lần lượt tiến hành đánh giá thực nghiệm toàn diện với nhiều LLM khác nhau và trên các tiêu chuẩn sửa chữa chương trình tự động khác nhau, trong khi các nhà nghiên cứu khác [89], [96], [98], [100] tập trung vào một vấn đề cụ thể LLM và trên một tập dữ liệu, ví dụ như QuixBugs. Ngoài ra, Gao et al. [124] điều tra thực nghiệm tác động của ngữ cảnh trong ngữ cảnh Các ví dụ minh họa để sửa lỗi, bao gồm việc lựa chọn, thử tự và số lượng ví dụ minh họa. Prenner và cộng sự. [117] nghiên cứu thực nghiệm về bối cảnh địa phương (tức là mã mà (Xuất hiện trước hoặc sau vị trí lỗi) ảnh hưởng đến hiệu suất sửa chữa. Horvath et al. [99] nghiên cứu thực nghiệm tác động của biểu diễn chương trình và kiến trúc mô hình lên Hiệu suất sửa chữa.

Có hai thiết lập sửa chữa thường được sử dụng khi dùng LLM để tạo bản vá: 1) tạo hàm hoàn chỉnh (tức là tạo toàn bộ hàm vá), 2) điền mã chính xác (tức là điền vào một đoạn mã được cung cấp).

tiền tố và hậu tố), và các nghiên cứu khác nhau có thể sử dụng các thiết lập khác nhau được đánh dấu trong Bảng 4. Các bộ dữ liệu thường được sử dụng là QuixBugs, Defects4J, v.v. Các bộ dữ liệu này chỉ bao gồm các chức năng cơ bản như phân loại. các thuật toán, số dòng trung bình của mỗi chương trình dao động từ 13 đến 22, thực hiện một chức năng và liên quan đến một số ít phụ thuộc. Để giải quyết vấn đề này, Cao et al. [89] tiến hành một nghiên cứu thực nghiệm trên một tập dữ liệu phức tạp hơn. Các chương trình học sâu (DL) được thu thập từ StackOverflow. Mỗi chương trình trung bình chứa khoảng 46 dòng mã, thực hiện một số chức năng bao gồm tiền xử lý dữ liệu, xây dựng mô hình DL, huấn luyện mô hình và đánh giá.

Và tập dữ liệu này bao gồm hơn 6 mối quan hệ phụ thuộc cho mỗi đối tượng. chương trình của họ, bao gồm TensorFlow, Keras và Pytorch. Kết quả cho thấy tỷ lệ vá lỗi chính xác thấp hơn nhiều. so với các tập dữ liệu khác, điều này một lần nữa cho thấy tiềm năng độ khó của nhiệm vụ này. Tương tự, Haque et al. [106] giới thiệu một tập dữ liệu bao gồm các bài nộp mã lỗi và thông tin của chúng. Các bản sửa lỗi tương ứng được thu thập từ các nền tảng chấm điểm trực tuyến. trong đó nó cung cấp một bộ sưu tập phong phú các bài kiểm thử đơn vị. cho phép đánh giá về tính chính xác của các bản sửa lỗi và thông tin bổ sung liên quan đến thời gian, hạn chế bộ nhớ, và Chấp nhận dựa trên phản quyết.



Hình 6: Các mô hình LLM được sử dụng trong các bài báo đã thu thập.

5 PHÂN TÍCH TỪ GÓC NHÌN CỦA CHƯƠNG TRÌNH LUẬT SƯ (LLM)

Phần này thảo luận về phân tích dựa trên các quan điểm. Cụ thể hơn, chương trình LLM được trình bày từ các quan điểm sau: sử dụng LLM, các loại kỹ thuật nhanh, đầu vào của Các mô hình LLM, cũng như các kỹ thuật đi kèm khi sử dụng

Thay vì luật (LLM).

5.1 Mô hình LLM

Như thể hiện trong Hình 6, LLM được sử dụng phổ biến nhất Trong các nhiệm vụ kiểm thử phần mềm, ChatGPT là một công cụ đã được phát hành. Được OpenAI phát triển vào tháng 11 năm 2022. Nó được huấn luyện trên một kho dữ liệu lớn. dữ liệu văn bản ngôn ngữ tự nhiên, và chủ yếu được thiết kế cho Xử lý ngôn ngữ tự nhiên và hội thoại. ChatGPT là Đây là chương trình LLM được công nhận rộng rãi và phổ biến nhất cho đến nay. Được biết đến với hiệu năng vượt trội trong nhiều nhiệm vụ khác nhau. Do đó, không có gì ngạc nhiên khi nó nằm trong top đầu. vị trí xét về các nghiên cứu mà chúng tôi đã thu thập được. Codex, một mô hình LLM dựa trên GPT-3, là mô hình LLM được sử dụng phổ biến thứ hai trong các nghiên cứu mà chúng tôi đã thu thập. Nó được huấn luyện trên một... Kho mã nguồn khổng lồ chứa các ví dụ từ nhiều ngôn ngữ lập trình như JavaScript, Python, C/C++, và Java. Codex được OpenAI phát hành vào tháng 9 năm 2021. Nền tảng này cung cấp sức mạnh cho GitHub Copilot - một lập trình viên AI có khả năng tạo ra toàn bộ đoạn mã, dựa trên mô tả bằng ngôn ngữ tự nhiên được cung cấp làm lời nhắc. Vì phần lớn các nghiên cứu chúng tôi thu thập được liên quan đến mã nguồn (ví dụ: sửa chữa, tạo trường hợp kiểm thử đơn vị), nên không có gì ngạc nhiên khi các nhà nghiên cứu chọn Codex. với vai trò là người hướng dẫn chương trình LLM trong việc hỗ trợ họ hoàn thành các nhiệm vụ liên quan đến lập trình.

Phần mềm quản lý học tập (LLM) xếp thứ ba là CodeT5, một phần mềm LLM mã nguồn mở được phát triển bởi salesforce3 . Nhờ vào tính chất mở của nó, CodeT5 có nhiều tính năng hữu ích. Nhờ nguồn thông tin này, các nhà nghiên cứu có thể dễ dàng tiến hành đào tạo trước và Tinh chỉnh bằng dữ liệu chuyên ngành để đạt được kết quả tốt hơn. hiệu năng. Tương tự, CodeGen cũng là mã nguồn mở và được xếp hạng tương đối cao hơn. Ngoài ra, đối với CodeT5 và CodeGen, Hơn một nửa số nghiên cứu liên quan đến vấn đề này.

các đánh giá thực nghiệm (sử dụng nhiều mô hình LLM), ví dụ: sửa chữa chương trình [112], [113], tạo trường hợp kiểm thử đơn vị [39].

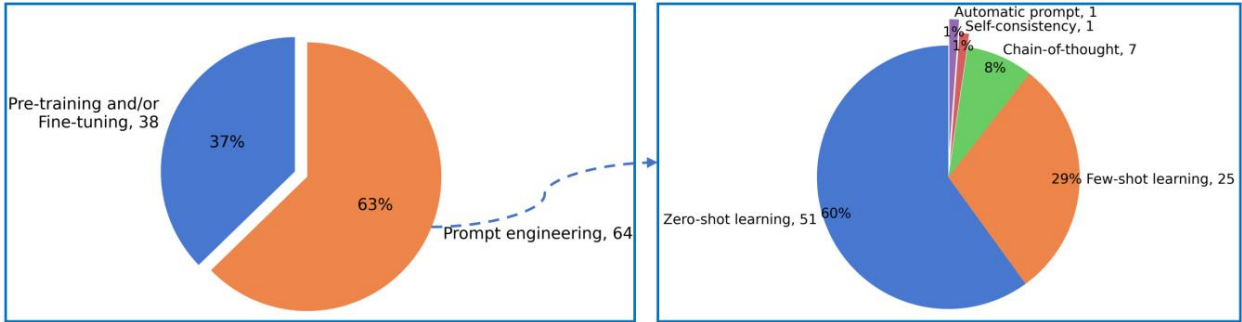
3. <https://blog.salesforceairesearch.com/codet5/>

Hiện đã có 14 nghiên cứu sử dụng GPT-4, xếp hạng ở vị trí thứ tư, được ra mắt vào tháng 3 năm 2023. Một số nghiên cứu trực tiếp sử dụng LLM tiên tiến này của OpenAI, vì nó thể hiện hiệu suất tuyệt vời trên nhiều lĩnh vực. nhiều loại nhiệm vụ tạo sinh và suy luận. Ví dụ: Xie et al. sử dụng GPT-4 để tạo ra đầu vào mở [67], trong khi Vikram và cộng sự sử dụng nó để tạo ra các bài kiểm tra dựa trên thuộc tính với sự hỗ trợ của tài liệu API [34]. Ngoài ra, một số Các nghiên cứu tiến hành thí nghiệm sử dụng cả GPT-4 và Chat-GPT hoặc các LLM khác để cung cấp đánh giá toàn diện hơn về hiệu suất của các mô hình này. Trong kỹ thuật kiểm thử xâm nhập tự động được hỗ trợ bởi LLM mà họ đề xuất, Deng et al. nhận thấy rằng GPT-4 vượt trội hơn ChatGPT và LaMDA từ Google [62]. Tương tự, Zhang et al. phát hiện ra rằng GPT-4 cho thấy Hiệu năng vượt trội của nó so với ChatGPT khi tạo trình điều khiển fuzz bằng cả hai chiến lược truy vấn cơ bản. và các chiến lược truy vấn nâng cao [66]. Hơn nữa, GPT-4, như LLM đa phương thức này khác biệt so với các LLM khác đã đề cập bằng cách thể hiện các khả năng bổ sung như: Tạo ra các câu chuyện bằng hình ảnh và trả lời các câu hỏi dựa trên đó. trên hình ảnh [149]. Tuy nhiên, chúng tôi chưa tìm thấy bất kỳ nghiên cứu nào nghiên cứu việc tận dụng các tính năng liên quan đến hình ảnh của GPT-4. (ví dụ: ảnh chụp màn hình giao diện người dùng, video hướng dẫn lập trình) trong phần mềm các nhiệm vụ kiểm thử.

5.2 Các loại kỹ thuật nhắc nhở

Như thể hiện trong Hình 7, trong số các nghiên cứu mà chúng tôi đã thu thập, có 38 nghiên cứu. Các nghiên cứu sử dụng LLM thông qua huấn luyện trước hoặc tinh chỉnh lược đồ, trong khi 64 nghiên cứu sử dụng lời nhắc kỹ thuật để giao tiếp với LLM nhằm định hướng nó. hành vi hướng tới kết quả mong muốn mà không cần cập nhật mô hình trọng số. Khi sử dụng các LLM đời đầu, hiệu suất của chúng có thể không ấn tượng lắm, vì vậy các nhà nghiên cứu thường sử dụng các kỹ thuật huấn luyện trước hoặc tinh chỉnh để điều chỉnh mô hình cho các lĩnh vực và nhiệm vụ cụ thể nhằm mục đích cải thiện chúng. hiệu suất. Sau đó, với việc nâng cấp công nghệ LLM, đặc biệt là với sự ra mắt của GPT-3 và các thế hệ sau này. LLMs, kiến thức chứa đựng trong các mô hình và Khả năng hiểu/suy luận của họ đã tăng lên đáng kể. Do đó, các nhà nghiên cứu thường sẽ dựa vào thúc đẩy kỹ sư xem xét cách thiết kế phù hợp Các gợi ý nhằm kích thích kiến thức của mô hình.

Trong số 64 nghiên cứu sử dụng kỹ thuật xử lý dữ liệu tức thời, 51 nghiên cứu liên quan đến học không cần dữ liệu huấn luyện (zero-shot learning), và 25 nghiên cứu liên quan đến học có ít dữ liệu huấn luyện (few-shot learning) (một nghiên cứu có thể liên quan đến nhiều loại). cũng có các nghiên cứu liên quan đến chuỗi suy luận (7 nghiên cứu), Tinh nhất quán tự thân (1 nghiên cứu) và lời nhắc tự động (1 nghiên cứu). Học không cần huấn luyện (zero-shot learning) đơn giản chỉ là cung cấp văn bản bài tập cho hệ thống. mô hình và yêu cầu kết quả. Nhiều nghiên cứu được thu thập sử dụng Codex, CodeT5 và CodeGen (như được trình bày trong Phần 5.1), đã được huấn luyện trên mã nguồn. Do đó, đối với các tác vụ liên quan đến mã nguồn như tạo trường hợp kiểm thử đơn vị và sửa chữa chương trình như đã trình bày trong các phần trước, Việc truy vấn trực tiếp LLM bằng các lời nhắc là cách phổ biến. thực hành. Nói chung có hai loại phương thức học không cần dữ liệu huấn luyện, tức là có và không có hướng dẫn. Ví dụ, Xie et al. [36] sẽ cung cấp cho LLM các hướng dẫn như “vui lòng giúp tôi tạo một bài kiểm tra JUnit cho một cụ thể Phương thức Java ...” để hỗ trợ việc tạo trường hợp kiểm thử đơn vị. Ngược lại, Siddiq et al. [39] chỉ cung cấp tiêu đề mã



Hình 7: Phân bố về cách sử dụng LLM (Lưu ý rằng, một nghiên cứu có thể bao gồm nhiều loại kỹ thuật nhắc nhở khác nhau)

của trường hợp kiểm thử đơn vị (ví dụ: “class \${className}\${suffix}Test {}”), và LLM sẽ tự động thực hiện việc tạo trường hợp kiểm thử đơn vị. Nói chung, các lời nhắc có hướng dẫn rõ ràng sẽ cho kết quả chính xác hơn, trong khi các lời nhắc không có hướng dẫn thường chỉ phù hợp với những tình huống rất cụ thể.

Học tập ít mẫu trình bày một tập hợp các bản trình diễn chất lượng cao, mỗi bản bao gồm cả đầu vào và đầu ra mong muốn, về nhiệm vụ mục tiêu. Khi mô hình lần đầu tiên nhìn thấy các ví dụ, nó có thể hiểu rõ hơn ý định và tiêu chí của con người về loại câu trả lời mong muốn, điều này đặc biệt quan trọng đối với các nhiệm vụ không đơn giản hoặc trực quan đối với LLM. Ví dụ, khi thực hiện tạo kiểm thử tự động từ các báo cáo lỗi chung, Kang et al. [78] cung cấp các ví dụ về báo cáo lỗi (câu hỏi) và các bài kiểm thử tái tạo lỗi tương ứng (câu trả lời) cho LLM, và kết quả của họ cho thấy rằng hai ví dụ có thể đạt được hiệu suất cao nhất so với không có ví dụ hoặc số lượng ví dụ khác. Một ví dụ khác về tạo khẳng định kiểm thử, Nashid et al. [47] cung cấp các bản trình diễn của phương pháp trọng tâm, phương pháp kiểm thử chứa <AssertPlaceholder> và khẳng định mong đợi, cho phép LLM hiểu rõ hơn về nhiệm vụ.

Gợi ý chuỗi suy nghĩ (CoT) tạo ra một chuỗi các câu ngắn để mô tả logic suy luận từng bước (còn được gọi là chuỗi suy luận hoặc lý lẽ) cho LLM để tạo ra câu trả lời cuối cùng. Ví dụ, để sửa chữa chương trình từ mô tả sự cố ngôn ngữ tự nhiên [122], với mã bị lỗi và báo cáo sự cố, các tác giả trước tiên yêu cầu LLM xác định vị trí lỗi, sau đó họ yêu cầu nó giải thích tại sao các dòng được xác định vị trí lại bị lỗi, cuối cùng, họ yêu cầu LLM sửa lỗi. Một ví dụ khác là để tạo ra các chương trình bất thường cho việc làm mờ các thư viện học sâu, Deng et al. [58] trước tiên tạo ra một “lỗi” có thể xảy ra (mô tả lỗi) trước khi tạo ra đoạn mã “kích hoạt lỗi” thực tế gọi API mục tiêu. Mô tả lỗi được dự đoán cung cấp thêm gợi ý cho LLM, cho biết rằng mã được tạo ra nên cố gắng bao phủ hành vi lỗi tiềm ẩn cụ thể.

Tính nhất quán tự thân liên quan đến việc đánh giá tính mạch lạc và nhất quán của các phản hồi của LLM trên cùng một đầu vào trong các ngữ cảnh khác nhau. Có một nghiên cứu với loại lời nhắc này, và nó liên quan đến việc gỡ lỗi. Kang et al. [83] sử dụng vòng lặp giả thuyết-quan sát-kết luận, trong đó đầu tiên tạo ra một giả thuyết về lỗi là gì và xây dựng một thí nghiệm để xác minh, sử dụng LLM, sau đó quyết định xem giả thuyết có đúng hay không dựa trên kết quả thí nghiệm.

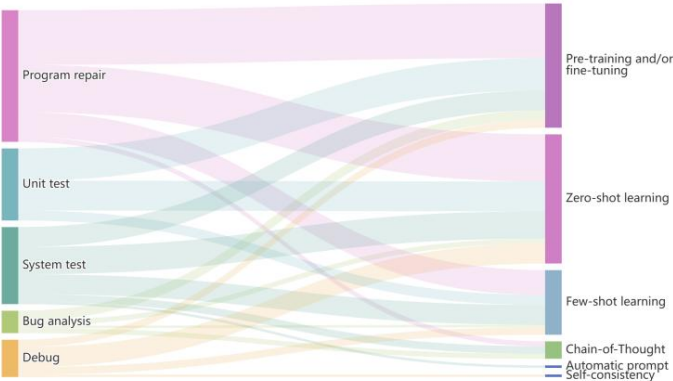
(với trình gỡ lỗi hoặc thực thi mã) sử dụng LLM, sau đó, tùy thuộc vào kết luận, nó sẽ bắt đầu với một giả thuyết mới hoặc chọn kết thúc quá trình gỡ lỗi và tạo ra bản sửa lỗi.

Lời nhắc tự động nhằm mục đích tự động tạo và chọn hướng dẫn phù hợp cho LLM, thay vì yêu cầu người dùng phải tự thiết kế lời nhắc. Xia et al. [67] giới thiệu một bước tự động nhắc nhở, tự động chất lọc tất cả các đầu vào do người dùng cung cấp thành một lời nhắc ngắn gọn và hiệu quả để làm mờ. Cụ thể, trước tiên họ tạo ra một danh sách các lời nhắc ứng cử bằng cách kết hợp đầu vào của người dùng và hướng dẫn tự động nhắc nhở trong khi đặt LLM ở nhiệt độ cao, sau đó tiến hành một thí nghiệm làm mờ quy mô nhỏ để đánh giá từng lời nhắc ứng cử và chọn lời nhắc tốt nhất.

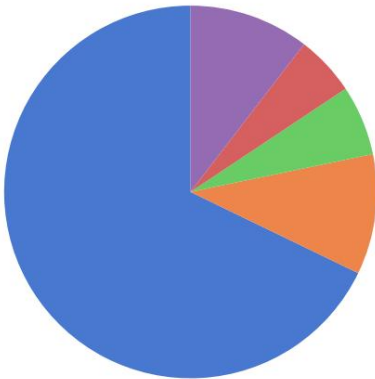
Lưu ý rằng có mười bốn nghiên cứu áp dụng thiết kế nhắc nhở lặp đi lặp lại khi sử dụng học không có dữ liệu hoặc học ít dữ liệu, trong đó phương pháp này liên tục tinh chỉnh các lời nhắc với thông tin đang chạy của nhiệm vụ kiểm thử, ví dụ: thông tin lỗi kiểm thử. Ví dụ, đối với việc sửa chữa chương trình, Xia et al. [115] xen kẽ việc tạo bản và với phản hồi xác thực kiểm thử để nhắc nhở việc tạo ra thể hệ tiếp theo một cách lặp đi lặp lại.

Cụ thể, chúng kết hợp nhiều thông tin từ một bài kiểm tra thất bại bao gồm tên của nó, dòng mã liên quan gây ra lỗi kiểm tra và thông báo lỗi được tạo ra trong vòng nhắc nhở tiếp theo, điều này có thể giúp mô hình hiểu lý do thất bại và cung cấp hướng dẫn để tạo ra bản sửa lỗi chính xác. Một ví dụ khác là đối với thử nghiệm GUI trên thiết bị di động, Liu et al. [14] truy vấn LLM lặp đi lặp lại về thao tác (ví dụ: nhấp vào nút, nhập văn bản) sẽ được thực hiện trong ứng dụng di động và ở mỗi lần lặp, họ sẽ cung cấp cho LLM thông tin ngữ cảnh hiện tại như các trang GUI và tiện ích nào vừa được khám phá.

Ảnh xạ giữa các nhiệm vụ kiểm thử và cách sử dụng LLM. Hình 8 minh họa ảnh xạ giữa các nhiệm vụ kiểm thử (được đề cập trong Phần 4) và cách sử dụng LLM (như đã giới thiệu trong phần này). Việc tạo trường hợp kiểm thử đơn vị và sửa chữa chương trình có chung các mô hình giao tiếp với LLM, vì cả hai nhiệm vụ đều liên quan chặt chẽ đến mã nguồn. Thông thường, các nhà nghiên cứu sử dụng phương pháp huấn luyện trước và/hoặc tinh chỉnh và học không cần dữ liệu huấn luyện (zero-shot learning) cho hai nhiệm vụ này. Học không cần dữ liệu huấn luyện phù hợp vì các nhiệm vụ này tương đối đơn giản và LLM có thể dễ dàng hiểu được. Hơn nữa, vì dữ liệu huấn luyện cho hai nhiệm vụ này có thể được thu thập tự động từ kho mã nguồn, nên các phương pháp huấn luyện trước và/hoặc tinh chỉnh cũng rất hữu ích.



Hình 8: Sơ đồ mối liên hệ giữa các nhiệm vụ kiểm thử và cách sử dụng LLM.



Hình 9: Đầu vào của LLM

Chúng được sử dụng rộng rãi cho hai nhiệm vụ này, có thể giúp nâng cao hiểu biết của các học viên luật về kiến thức chuyên ngành.

Ngược lại, đối với việc tạo dữ liệu đầu vào kiểm thử hệ thống, các phương pháp học không cần huấn luyện (zero-shot learning) và học cần huấn luyện ít (few-shot learning) thường được sử dụng. Điều này có thể là do nhiệm vụ này thường liên quan đến việc tạo ra các loại dữ liệu đầu vào cụ thể, và các ví dụ minh họa trong học cần huấn luyện ít có thể giúp các mô hình học máy (LLM) hiểu rõ hơn những gì cần được tạo ra. Bên cạnh đó, đối với nhiệm vụ này, việc sử dụng các phương pháp huấn luyện trước và/hoặc tinh chỉnh không phổ biến như trong việc tạo trường hợp kiểm thử đơn vị và sửa chữa chương trình. Điều này có thể là do dữ liệu huấn luyện cho kiểm thử hệ thống khác nhau giữa các phần mềm và tương đối khó thu thập tự động.

5.3 Đầu vào của LLM

Chúng tôi cũng nhận thấy rằng các nhiệm vụ kiểm thử khác nhau hoặc phần mềm đang được kiểm thử có thể liên quan đến các đầu vào đa dạng khi truy vấn LLM, như được minh họa trong Hình 9.

Đầu vào được sử dụng phổ biến nhất là mã nguồn vì phần lớn các nghiên cứu được thu thập liên quan đến việc sửa chữa chương trình hoặc tạo trường hợp kiểm thử đơn vị mà đầu vào là mã nguồn. Đối với việc tạo trường hợp kiểm thử đơn vị, thông tin liên quan đến mã diễn hình sẽ là (i) phương thức trọng tâm hoàn chỉnh, bao gồm chữ ký và phần thân; (ii) tên của lớp trọng tâm (tức là lớp mà phương thức trọng tâm thuộc về); (iii) trường trong lớp trọng tâm; và (iv) chữ ký của tất cả các phương thức được định nghĩa trong lớp trọng tâm [7], [26]. Đối với việc sửa chữa chương trình, có thể có

các thiết lập khác nhau và liên quan đến các đầu vào khác nhau, bao gồm (i) nhập một hàm bị lỗi với mục tiêu xuất ra hàm đã được vá, (ii) nhập vị trí bị lỗi với mục tiêu tạo ra mã thay thế chính xác (có thể là thay đổi một dòng) dựa trên tiền tố và hậu tố của hàm bị lỗi [93]. Ngoài ra, có thể có các biến thể cho đầu vào vị trí bị lỗi, tức là, (i) không chứa các dòng bị lỗi (nhưng vị trí lỗi vẫn được biết), (ii) đưa ra các dòng bị lỗi dưới dạng các dòng bình luận.

Ngoài ra còn có 12 nghiên cứu sử dụng mô tả lỗi làm đầu vào cho LLM. Ví dụ, Kang et al. [78] sử dụng mô tả lỗi làm đầu vào khi truy vấn LLM và để LLM tạo ra các trường hợp thử nghiệm tái tạo lỗi. Fakhoury et al. [122] nhập mô tả lỗi bằng ngôn ngữ tự nhiên vào LLM và tạo ra các bản sửa lỗi mã chính xác.

Có 7 nghiên cứu sẽ cung cấp thông tin về lỗi trung gian, ví dụ như thông tin về lỗi kiểm thử, cho LLM và sẽ tiến hành nhắc nhở lặp đi lặp lại (như mô tả trong Phần 5.2) để làm phong phú thêm ngữ cảnh được cung cấp cho LLM.

Các nghiên cứu này liên quan đến việc tạo trường hợp kiểm thử đơn vị và sửa chữa chương trình, vì trong những trường hợp này, thông tin đang chạy có thể được thu thập dễ dàng.

Khi kiểm thử ứng dụng di động, vì mô hình LLM được sử dụng không thể hiểu hình ảnh của trang giao diện người dùng (GUI), nên tệp cấu trúc phân cấp hiển thị, thể hiện chi tiết của trang GUI, thường được dùng làm đầu vào cho các mô hình LLM. Tuy nhiên, với sự xuất hiện của GPT-4, một mô hình đa phương thức chấp nhận cả hình ảnh và văn bản làm đầu vào, ảnh chụp màn hình GUI có thể được sử dụng trực tiếp làm đầu vào cho mô hình LLM.

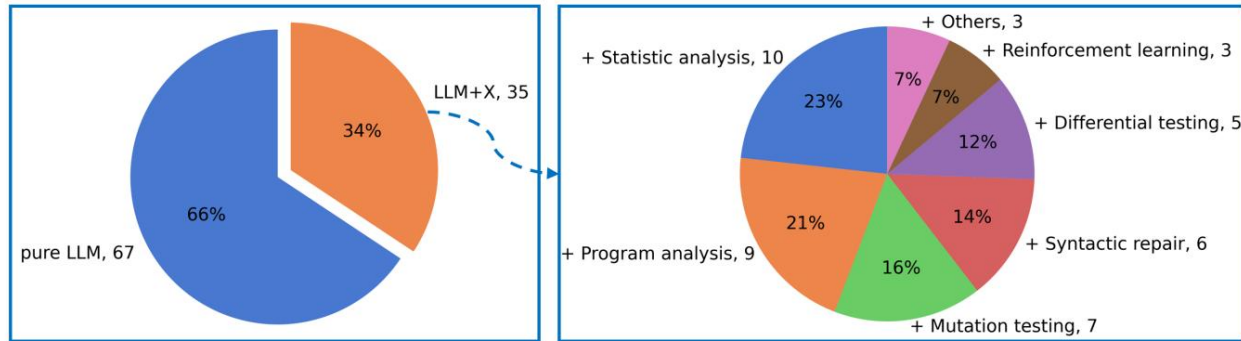
5.4 Kết hợp các kỹ thuật khác với LLM Có nhiều ý kiến trái

chiều về việc liệu LLM đã đạt đến trạng thái toàn năng mà không cần đến các kỹ thuật khác hay chưa. Như thể hiện trong Hình 10, trong số các nghiên cứu chúng tôi đã thu thập, 67 nghiên cứu sử dụng LLM để giải quyết toàn bộ nhiệm vụ kiểm thử, trong khi 35 nghiên cứu kết hợp thêm các kỹ thuật khác. Các kỹ thuật này bao gồm kiểm thử đột biến, kiểm thử khác biệt, kiểm tra cú pháp, phân tích chương trình, phân tích thống kê, v.v.

Lý do các nhà nghiên cứu vẫn lựa chọn kết hợp LLM với các kỹ thuật khác có thể là vì, mặc dù thể hiện tiềm năng to lớn trong nhiều nhiệm vụ, LLM vẫn còn những hạn chế như hiểu ngữ nghĩa mã và xử lý cấu trúc chương trình phức tạp. Do đó, việc kết hợp LLM với các kỹ thuật khác tối ưu hóa điểm mạnh và điểm yếu của chúng để đạt được kết quả tốt hơn trong các tình huống cụ thể. Ngoài ra, điều quan trọng cần lưu ý là mặc dù LLM có khả năng tạo ra mã chính xác, nhưng chúng có thể không nhất thiết tạo ra đủ trường hợp thử nghiệm để kiểm tra các trường hợp ngoại lệ hoặc các tình huống hiếm gặp. Đây là lúc các kỹ thuật đột biến và các kỹ thuật kiểm thử khác phát huy tác dụng, vì chúng cho phép tạo ra mã đa dạng và phức tạp hơn, có thể mô phỏng tốt hơn các tình huống thực tế. Theo nghĩa này, một phương pháp kiểm thử có thể kết hợp nhiều kỹ thuật khác nhau, bao gồm cả LLM và các chiến lược kiểm thử khác, để đảm bảo phạm vi bao phủ toàn diện và hiệu quả.

LLM kết hợp với phân tích thống kê. Vì LLM thường tạo ra rất nhiều kết quả đầu ra, việc sàng lọc và xác định kết quả đầu ra chính xác một cách thủ công có thể rất tốn công sức.

Do đó, các nhà nghiên cứu đã chuyển sang các kỹ thuật phân tích thống kê như xếp hạng và phân cụm [28], [45], [78], [93], [116]



Hình 10: Phân bố các kỹ thuật khác được kết hợp với LLM (Lưu ý rằng, một nghiên cứu có thể bao gồm nhiều loại kỹ thuật khác nhau)

Đề lọc hiệu quả các kết quả đầu ra của LLM và cuối cùng thu được kết quả chính xác hơn.

LLM + phân tích chương trình. Khi sử dụng LLM để thực hiện các nhiệm vụ như tạo trường hợp kiểm thử đơn vị và sửa chữa mã phần mềm, điều quan trọng cần lưu ý là mã phần mềm vốn dĩ chứa thông tin cấu trúc mà LLM có thể không hiểu đầy đủ. Do đó, các nhà nghiên cứu thường sử dụng các kỹ thuật phân tích chương trình, bao gồm cây cú pháp trừu tượng mã (AST) [74], để biểu diễn cấu trúc của mã hiệu quả hơn và tăng khả năng hiểu mã chính xác của LLM.

Các nhà nghiên cứu cũng thực hiện việc phân chia tập con dựa trên cấu trúc của các dòng mã để thu hẹp phạm vi tập trung cho LLM [94] hoặc trích xuất ngữ cảnh mã bổ sung từ các tệp mã khác [7], để cho phép các mô hình tập trung vào thông tin liên quan nhất đến nhiệm vụ trong cơ sở mã và dẫn đến các dự đoán chính xác hơn.

LLM + thử nghiệm đột biến. Mục tiêu chính là tạo ra các đầu vào thử nghiệm đa dạng hơn. Ví dụ, Deng et al. [59] trước tiên sử dụng LLM để tạo ra các chương trình hạt giống (ví dụ: các đoạn mã sử dụng API DL mục tiêu) để làm mờ các thư viện học sâu. Để làm phong phú thêm nhóm các chương trình thử nghiệm này, họ thay thế các phần của chương trình hạt giống bằng các mã thông báo được che bằng cách sử dụng các toán tử đột biến (ví dụ: thay thế các đối số gọi API bằng mã thông báo span) để tạo ra các đầu vào được che, và lại sử dụng LLM để thực hiện điền mã nhằm tạo ra mã mới thay thế các mã thông báo được che.

LLM + kiểm tra cú pháp. Mặc dù LLM đã thể hiện hiệu suất đáng kể trong nhiều nhiệm vụ xử lý ngôn ngữ tự nhiên, mã được tạo ra từ các mô hình này đôi khi có thể không chính xác về mặt cú pháp, dẫn đến các lỗi tiềm ẩn và giảm khả năng sử dụng. Do đó, các nhà nghiên cứu đã đề xuất tận dụng kiểm tra cú pháp để xác định và sửa lỗi trong mã được tạo ra. Ví dụ, trong công trình của họ về tạo trường hợp kiểm thử đơn vị, Alagarsamy et al. [29] đã giới thiệu thêm một phương pháp xác minh để kiểm tra và sửa chữa tính nhất quán về tên (tức là sửa đổi tên phương thức kiểm thử để nhất quán với tên phương thức trọng tâm) và các chữ ký kiểm thử (tức là thêm các từ khóa bị thiếu như public, void hoặc chú thích @test). Xie et al. [36] cũng xác thực trường hợp kiểm thử đơn vị được tạo ra và sử dụng sửa chữa dựa trên quy tắc để khắc phục các lỗi cú pháp và lỗi biên dịch đơn giản.

LLM + kiểm thử khác biệt. Kiểm thử khác biệt rất phù hợp để tìm các lỗi ngữ nghĩa hoặc logic không thể hiện các hành vi sai sót rõ ràng như sự cố sập chương trình hoặc lỗi khẳng định. Trong nhóm các nghiên cứu đã thu thập này, LLM chủ yếu chịu trách nhiệm tạo ra các đầu vào hợp lệ và đa dạng, trong khi kiểm thử khác biệt giúp xác định

liệu có lỗi nào được kích hoạt dựa trên đầu ra của phần mềm hay không. Ví dụ, Ye et al. [48] trước tiên sử dụng LLM để tạo ra các chương trình JavaScript ngẫu nhiên và tận dụng tài liệu đặc tả ngôn ngữ để tạo dữ liệu thử nghiệm, sau đó tiến hành thử nghiệm khác biệt trên các công cụ JavaScript như JavaScriptCore, ChakraCore, SpiderMonkey, QuickJS, v.v. Cũng có những nghiên cứu sử dụng LLM để tạo đầu vào thử nghiệm và sau đó tiến hành thử nghiệm khác biệt để làm mờ các thư viện DL [58], [59] và bộ giải SAT [63]. Li et al. [87] sử dụng LLM để tìm các trường hợp thử nghiệm gây lỗi.

Cụ thể hơn, khi được cung cấp một chương trình cần kiểm thử, trước tiên họ yêu cầu LLM suy luận mục đích của chương trình, sau đó yêu cầu LLM tạo ra các chương trình có cùng mục đích, là các cách triển khai thay thế của chương trình và có khả năng không chứa lỗi của chương trình. Sau đó, họ thực hiện kiểm thử so sánh giữa chương trình cần kiểm thử và các chương trình được tạo ra để tìm ra các trường hợp kiểm thử gây ra lỗi.

6 THÁCH THỨC VÀ CƠ HỘI Dựa trên phân tích trên từ

góc độ kiểm thử phần mềm và LLM, chúng tôi tóm tắt những thách thức và cơ hội khi thực hiện kiểm thử phần mềm với LLM.

6.1 Thách thức Như

khảo sát này đã chỉ ra, việc kiểm thử phần mềm với LLM đã có sự tăng trưởng đáng kể trong hai năm qua.

Tuy nhiên, nó vẫn đang trong giai đoạn phát triển ban đầu và còn nhiều thách thức cũng như câu hỏi chưa được giải đáp cần phải được giải quyết.

6.1.1 Thách thức để đạt được độ phủ cao Việc khám phá các

hành vi đa dạng của phần mềm được kiểm thử để đạt được độ phủ cao luôn là mối quan tâm đáng kể trong kiểm thử phần mềm. Trong bối cảnh này, việc tạo kiểm thử khác với việc tạo mã, vì việc tạo mã chủ yếu tập trung vào việc tạo ra một đoạn mã chính xác duy nhất, trong khi kiểm thử phần mềm yêu cầu tạo ra các đầu vào kiểm thử đa dạng để đảm bảo độ phủ tốt hơn của phần mềm. Mặc dù việc đặt nhiệt độ cao có thể hỗ trợ LLM trong việc tạo ra các đầu ra khác nhau, nhưng việc LLM trực tiếp đạt được sự đa dạng cần thiết vẫn là một thách thức. Ví dụ, đối với việc tạo trường hợp kiểm thử đơn vị, trong tập dữ liệu SF110, độ phủ dòng chỉ là 2% và độ phủ nhánh chỉ là 1% [39]. Đối với việc tạo đầu vào kiểm thử hệ thống, về mặt làm mờ các thư viện DL, độ phủ API cho TensorFlow được báo cáo là 66% (2215/3316) [59].

Từ các nghiên cứu đã thu thập, chúng tôi nhận thấy rằng các nhà nghiên cứu thường sử dụng thử nghiệm đột biến cùng với LLM để tạo ra nhiều kết quả đầu ra đa dạng hơn. Ví dụ, khi làm mờ một thư viện DL, thay vì trực tiếp tạo đoạn mã bằng LLM, Deng et al. [59] thay thế các phần của hạt giống được chọn (mã được tạo bởi LLM) bằng các mã thông báo được che giấu bằng cách sử dụng các toán tử đột biến khác nhau để tạo ra đầu vào được che giấu. Sau đó, họ tận dụng LLM để thực hiện điền mã nhằm tạo ra mã mới thay thế các mã thông báo được che giấu, điều này có thể làm tăng đáng kể tính đa dạng của các bài kiểm tra được tạo ra. Liu et al. [65] tận dụng LLM để tạo ra các trình tạo bài kiểm tra (mỗi trình tạo có thể tạo ra một loạt đầu vào văn bản bất thường theo cùng một quy tắc đột biến) cùng với các quy tắc đột biến để làm mờ theo hướng văn bản, giúp giảm bớt nỗ lực của con người cần thiết để thiết kế các quy tắc đột biến.

Một hướng nghiên cứu tiềm năng có thể liên quan đến việc sử dụng dữ liệu chuyên biệt cho việc kiểm thử để huấn luyện hoặc tinh chỉnh một mô hình học máy (LLM) chuyên dụng được thiết kế đặc biệt để hiểu bản chất của việc kiểm thử. Bằng cách đó, LLM có thể tự động nhận biết các yêu cầu của việc kiểm thử và tự động tạo ra nhiều kết quả đầu ra khác nhau.

6.1.2 Những thách thức trong kiểm thử Oracle

Vấn đề oracle là một thách thức lâu dài trong nhiều ứng dụng thử nghiệm, ví dụ như thử nghiệm hệ thống học máy [150] và thử nghiệm thư viện học sâu [59]. Để giảm bớt vấn đề oracle đối với các hoạt động thử nghiệm tổng thể, một phương pháp phổ biến trong các nghiên cứu đã thu thập của chúng tôi là chuyển đổi nó thành một dạng dễ suy ra hơn, thường bằng cách sử dụng thử nghiệm khác biệt [63] hoặc chỉ tập trung vào việc xác định lỗi gây sự cố [14].

Có những ứng dụng thành công của việc kiểm thử khác biệt với LLM, như được thể hiện trong Hình 10. Ví dụ, khi kiểm thử các bộ giải SMT, Sun et al. áp dụng kiểm thử khác biệt bao gồm việc so sánh kết quả của nhiều bộ giải SMT (tức là Z3, cvc5 và Bitwuzla) trên cùng một công thức kiểm thử được tạo bởi LLM [63]. Tuy nhiên, cách tiếp cận này bị hạn chế đối với các hệ thống mà phần mềm hoặc môi trường chạy tương ứng có thể dễ dàng tìm thấy, có khả năng hạn chế khả năng áp dụng của nó. Hơn nữa, để giảm thiểu vấn đề oracle, các nghiên cứu khác chỉ tập trung vào các lỗi gây sập dễ dàng được quan sát tự động. Điều này đặc biệt đúng đối với việc kiểm thử ứng dụng di động, trong đó LLM hướng dẫn việc kiểm thử khám phá các trang đa dạng hơn, thực hiện các hành động vận hành phức tạp hơn và bao gồm các chuỗi hoạt động có ý nghĩa hơn [14]. Tuy nhiên, điều này hạn chế đáng kể tiềm năng sử dụng LLM để phát hiện ra nhiều loại lỗi phần mềm khác nhau.

Việc khám phá việc sử dụng LLM để tạo ra các loại oracle thử nghiệm khác là một hướng nghiên cứu thú vị và có giá trị. Cụ thể, thử nghiệm biến hình cũng được sử dụng rộng rãi trong thực tiễn kiểm thử phần mềm để giúp giảm thiểu vấn đề oracle, tuy nhiên trong hầu hết các trường hợp, việc xác định các mối quan hệ biến hình dựa vào sự khéo léo của con người. Luu et al. [56] đã kiểm tra hiệu quả của LLM trong việc tạo ra các mối quan hệ biến hình, nhưng họ chỉ thử nghiệm với các lời nhắc đơn giản bằng cách truy vấn trực tiếp ChatGPT. Việc khám phá sâu hơn, có thể kết hợp tương tác người-máy hoặc kiến thức miền, rất được khuyến khích. Một hướng đi đầy hứa hẹn khác là khám phá khả năng của LLM để tự động

Tạo các trường hợp thử nghiệm dựa trên các mối quan hệ biến đổi, bao gồm nhiều loại đầu vào khác nhau.

Sự phát triển của các mô hình LLM đa mô hình như GPT-4 có thể mở ra nhiều khả năng để khám phá khả năng phát hiện lỗi trong giao diện người dùng phần mềm và hỗ trợ việc xây dựng các oracle kiểm thử. Bằng cách tận dụng khả năng hiểu hình ảnh và suy luận của các mô hình này, người ta có thể nghiên cứu tiềm năng của chúng trong việc tự động xác định sự không nhất quán, lỗi hoặc các vấn đề về khả năng sử dụng trong giao diện người dùng.

6.1.3 Thách thức đối với việc đánh giá nghiêm ngặt Việc

thiếu các bộ dữ liệu chuẩn và các vấn đề tiềm ẩn về rò rỉ dữ liệu liên quan đến các kỹ thuật dựa trên LLM đặt ra những thách thức trong việc tiến hành đánh giá nghiêm ngặt và so sánh toàn diện các phương pháp được đề xuất.

Đối với việc sửa chữa chương trình, chỉ có hai tiêu chuẩn đánh giá nổi tiếng và được sử dụng phổ biến, đó là Defect4J và QuixBugs, như được trình bày trong Bảng 4. Hơn nữa, các tập dữ liệu này không được thiết kế đặc biệt để kiểm tra LLM. Ví dụ, như Xia et al. [93] đã báo cáo, 39 trong số 40 lỗi Python trong tập dữ liệu QuixBugs có thể được sửa bởi Codex, nhưng trong thực tế, tỷ lệ sửa lỗi thành công có thể không cao đến vậy.

Đối với việc tạo trường hợp kiểm thử đơn vị, hiện chưa có tiêu chuẩn đánh giá nào được công nhận rộng rãi, và các nghiên cứu khác nhau sẽ sử dụng các tập dữ liệu khác nhau để đánh giá hiệu năng, như được minh họa trong Bảng 3. Điều này cho thấy cần phải xây dựng các tiêu chuẩn đánh giá chuyên biệt và đa dạng hơn.

Hơn nữa, các LLM có thể đã thấy các tiêu chuẩn được sử dụng rộng rãi trong dữ liệu huấn luyện trước của chúng, tức là các vấn đề rò rỉ dữ liệu. Jiang et al. [113] kiểm tra CodeSearchNet và BigQuery, là các nguồn dữ liệu của các LLM phổ biến, và kết quả cho thấy bốn kho lưu trữ được sử dụng bởi tiêu chuẩn Defect4J cũng có trong CodeSearchNet và toàn bộ kho lưu trữ Defects4J được bao gồm bởi BigQuery.

Do đó, rất có thể các tiêu chuẩn sửa chữa chương trình hiện có được các LLM nhìn thấy trong quá trình huấn luyện trước. Vấn đề rò rỉ dữ liệu này cũng đã được điều tra trong các nghiên cứu liên quan đến học máy. Ví dụ, Tu et al. [151] tập trung vào rò rỉ dữ liệu trong dữ liệu theo dõi sự cố, và kết quả cho thấy thông tin bị rò rỉ từ “tương lai” khiến các mô hình dự đoán lạc quan một cách sai lệch. Điều này nhắc nhở chúng ta rằng hiệu suất của LLM trong các nhiệm vụ kiểm thử phần mềm có thể không tốt như đã được báo cáo trong các nghiên cứu trước đây. Nó cũng cho thấy rằng chúng ta cần nhiều tập dữ liệu chuyên biệt hơn mà LLM chưa từng thấy để làm tiêu chuẩn. Một cách là thu thập nó từ các nguồn chuyên biệt, ví dụ: nội dung do người dùng tạo ra từ các cộng đồng trực tuyến chuyên biệt.

6.1.4 Thách thức trong ứng dụng thực tế của LLM trong kiểm thử phần mềm Như đã đề cập ở Mục

5.2, trong giai đoạn đầu sử dụng LLM, huấn luyện trước và tinh chỉnh là những phương pháp thường được sử dụng, do số lượng tham số của mô hình tương đối ít dẫn đến khả năng của mô hình yếu hơn (ví dụ: T5). Theo thời gian, số lượng tham số của mô hình tăng lên đáng kể, dẫn đến sự xuất hiện của các mô hình có khả năng cao hơn (ví dụ: ChatGPT). Và trong các nghiên cứu gần đây, kỹ thuật tức thời đã trở thành một phương pháp phổ biến. Tuy nhiên, do lo ngại về quyền riêng tư dữ liệu, khi xem xét thực tiễn trong thế giới thực, hầu hết các tổ chức phần mềm có xu hướng tránh sử dụng phương pháp này.

đang sử dụng các hệ thống quản lý vòng đời doanh nghiệp (LLM) thương mại và muốn áp dụng các hệ thống mã nguồn mở có khả năng đào tạo hoặc tinh chỉnh bằng dữ liệu đặc thù của tổ chức. Hơn nữa, một số công ty cũng xem xét những hạn chế hiện tại về mặt năng lực tính toán hoặc

Hãy chú ý kỹ đến mức tiêu thụ năng lượng, họ thường có xu hướng Tinh chỉnh các mô hình cỡ trung bình. Điều này khá khó khăn đối với những mô hình này nhằm đạt được hiệu suất tương tự như của chúng tôi. Các bài báo đã được thu thập đã báo cáo. Ví dụ, trong bộ dữ liệu QuixBugs được sử dụng rộng rãi, người ta đã báo cáo rằng 39 trong số... 40 lỗi Python và 34 trong số 40 lỗi Java có thể được sửa tự động [93]. Tuy nhiên, khi nói đến các chương trình DL Thông tin được thu thập từ Stack Overflow, phản ánh thực tế. thực hành lập trình, chỉ có 16 trong số 72 lỗi Python có thể được sửa tự động [89].

Nghiên cứu gần đây đã nhấn mạnh tầm quan trọng của dữ liệu huấn luyện chất lượng cao trong việc cải thiện hiệu suất của các mô hình cho các nhiệm vụ liên quan đến mã [152], tuy nhiên việc xây dựng thủ công các bộ dữ liệu chất lượng cao dành riêng cho tổ chức để huấn luyện hoặc tinh chỉnh tốn nhiều thời gian và công sức. Để giải quyết Do đó, người ta khuyến khích sử dụng các kỹ thuật tự động. khai thác kho lưu trữ phần mềm để xây dựng các tập dữ liệu, cho ví dụ, các kỹ thuật như kỹ thuật trích xuất thông tin chính từ Stack Overflow [153] cung cấp các giải pháp tiềm năng Để tự động thu thập dữ liệu liên quan.

Ngoài ra, việc nghiên cứu phương pháp luận để tinh chỉnh tốt hơn các mô hình LLM với dữ liệu chuyên biệt cho phần mềm cũng đáng được xem xét, bởi vì dữ liệu chuyên biệt cho phần mềm khác với dữ liệu tự nhiên. dữ liệu ngôn ngữ vì nó chứa nhiều thông tin cấu trúc hơn. chẳng hạn như luồng dữ liệu và luồng điều khiển. Nghiên cứu trước đây về Các biểu diễn mã đã cho thấy lợi ích của việc kết hợp luồng dữ liệu, giúp nắm bắt cấu trúc ở cấp độ ngữ nghĩa. của mã và thể hiện mối quan hệ giữa các biến trong về mặt “giá trị có đến từ đâu” [154]. Những hiểu biết này có thể cung cấp hướng dẫn hữu ích để tinh chỉnh hiệu quả. Mô hình LLM với dữ liệu dành riêng cho phần mềm.

6.2 Cơ hội

Ngoài ra, còn có nhiều cơ hội nghiên cứu trong lĩnh vực kiểm thử phần mềm với LLM, điều này có thể mang lại lợi ích lớn cho các nhà phát triển, người dùng, và cộng đồng nghiên cứu. Mặc dù không nhất thiết là những thách thức, những cơ hội này góp phần vào sự tiến bộ trong... Kiểm thử phần mềm, mang lại lợi ích cho các chuyên gia và cộng đồng nghiên cứu rộng lớn hơn.

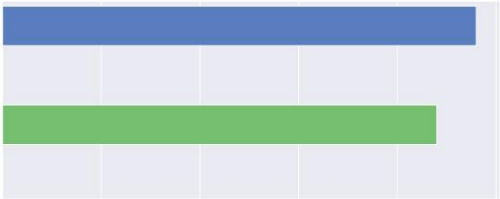
6.2.1 Khám phá các mô hình LLM trong giai đoạn thử nghiệm ban đầu

Như thể hiện trong Hình 4, LLM chưa được sử dụng trong giai đoạn đầu. giai đoạn thử nghiệm, ví dụ: yêu cầu thử nghiệm và lập kế hoạch thử nghiệm. Có thể có hai lý do chính đằng sau điều đó. Lý do đầu tiên là Tính chủ quan trong các nhiệm vụ kiểm tra giai đoạn đầu. Nhiều nhiệm vụ trong các giai đoạn đầu của quá trình thử nghiệm, chẳng hạn như thu thập yêu cầu, Việc lập kế hoạch thử nghiệm và xem xét thiết kế có thể bao gồm các đánh giá chủ quan đòi hỏi sự tham gia đáng kể của con người. các chuyên gia. Điều này có thể khiến nó kém phù hợp hơn đối với các chương trình LLM dựa vào... tập trung mạnh vào các phương pháp dựa trên dữ liệu. Thứ hai có thể là... Thiếu dữ liệu nguồn mở trong giai đoạn đầu. Không giống như trong giai đoạn sau này. Trong các giai đoạn thử nghiệm, dữ liệu trực tuyến có thể bị hạn chế. trong các hoạt động giai đoạn đầu. Điều này có thể có nghĩa là các chương trình LLM có thể chưa từng thấy nhiều dữ liệu thuộc loại này, và do đó có thể không thực hiện tốt các nhiệm vụ này.

Áp dụng mô hình tương tác người-máy tính cho Việc giải quyết các nhiệm vụ kiểm thử ở giai đoạn đầu sẽ tận dụng kiến thức chuyên môn của các nhà phát triển phần mềm và khai thác tối đa tiềm năng của họ. Kiến thức tổng quát được tích hợp trong các chương trình LLM. Ngoài ra, nó còn là Rất khuyến khích các công ty phát triển phần mềm để ghi lại và cung cấp quyền truy cập vào dữ liệu thử nghiệm giai đoạn đầu, cho phép cải thiện việc đào tạo và hiệu suất của các học viên cao học (LLM). trong các hoạt động thử nghiệm quan trọng này.

6.2.2 Khám phá LLM trong các giai đoạn thử nghiệm khác

Chúng tôi đã phân tích sự phân bố các giai đoạn thử nghiệm cho các nghiên cứu đã được thu thập. Như thể hiện trong Hình 11, chúng ta có thể quan sát thấy rằng LLM được sử dụng phổ biến nhất trong kiểm thử đơn vị, tiếp theo là... kiểm thử hệ thống. Tuy nhiên, hiện vẫn chưa có nghiên cứu nào về việc sử dụng về các chương trình LLM trong kiểm thử tích hợp và kiểm thử nghiệm thu.



Hình 11: Phân bố các giai đoạn thử nghiệm (lưu ý rằng chúng tôi bỏ qua phần này) các nghiên cứu không nêu rõ các giai đoạn thử nghiệm, ví dụ: sửa chữa chương trình)

Đối với kiểm thử tích hợp, nó bao gồm việc kiểm thử các giao diện. giữa các mô-đun phần mềm khác nhau. Trong một số tổ chức phần mềm, kiểm thử tích hợp có thể được gộp chung với kiểm thử đơn vị. việc kiểm tra, đây có thể là lý do tại sao LLM hiếm khi được thực hiện. được sử dụng trong kiểm thử tích hợp. Một lý do khác có thể là... Kích thước và độ phức tạp của dữ liệu đầu vào trong trường hợp này có thể vượt quá khả năng xử lý của LLM và phân tích (ví dụ: mã nguồn của tất cả các mô-đun phần mềm liên quan), điều này có thể dẫn đến lỗi hoặc kết quả không đáng tin cậy. Để giải quyết Điều này, một tài liệu tham khảo tiềm năng, có thể được tìm thấy trong Phần 4.1, nơi mà... Xie et al. [36] thiết kế một phương pháp để tổ chức những thứ cần thiết thông tin vào mã thông báo nhắc nhở tối đa được xác định trước giới hạn của LLM. Hơn nữa, kiểm thử tích hợp yêu cầu Dữ liệu đa dạng sẽ được tạo ra để kiểm tra đầy đủ giao diện giữa nhiều mô-đun. Như đã đề cập trong Phần 4.3, các nghiên cứu trước đây đã chứng minh khả năng của LLM trong việc tạo ra các dữ liệu đầu vào đa dạng cho việc kiểm thử hệ thống, trong kết hợp với các kỹ thuật kiểm tra đột biến [48], [59]. Và Những điều này có thể cung cấp những hiểu biết sâu sắc về việc tạo ra sự đa dạng. Dữ liệu giao diện để kiểm thử tích hợp.

Kiểm thử chấp nhận thường được thực hiện bởi các nhà phân tích nghiệp vụ hoặc người dùng cuối để xác nhận tính hợp lệ của chức năng hệ thống. và tính khả dụng, đòi hỏi ngôn ngữ phi chuyên ngành hơn. và kiến thức chuyên ngành, do đó khiến nó trở nên đầy thách thức. để áp dụng LLM một cách hiệu quả. Vì thử nghiệm chấp nhận liên quan đến đối với con người, nó rất phù hợp cho việc sử dụng phương pháp “con người tham gia vào quy trình”. lược đồ với LLM. Điều này đã được nghiên cứu trong truyền thống. học máy [155], nhưng vẫn chưa được khám phá với Các LLM (Learning Learning Modules). Cụ thể, các LLM có thể chịu trách nhiệm tự động tạo các trường hợp kiểm thử, đánh giá độ bao phủ kiểm thử, v.v. Trong khi đó, người kiểm thử chịu trách nhiệm kiểm tra hoạt động của chương trình và xác minh oracle kiểm thử.

6.2.3 Khám phá LLM cho nhiều loại phần mềm hơn

Chúng tôi phân tích các loại phần mềm đã được nghiên cứu trong các công trình đã thu thập, như thể hiện trong Hình 5. Lưu ý rằng, vì phần lớn các nghiên cứu tập trung vào kiểm thử đơn vị hoặc sửa chữa chương trình, nên chúng được thực hiện trên các tập dữ liệu công khai và không liên quan đến các loại phần mềm cụ thể.

Từ phân tích trong Mục 4.3, LLM không chỉ có thể tạo ra mã nguồn để kiểm thử các thư viện học sâu mà còn cả dữ liệu văn bản để kiểm thử ứng dụng di động, thậm chí cả các mô hình để kiểm thử hệ thống vật lý mạng (CPS). Nhìn chung, LLM cung cấp một khung làm việc linh hoạt và mạnh mẽ để tạo ra dữ liệu đầu vào kiểm thử cho nhiều ứng dụng khác nhau. Tính linh hoạt của nó sẽ làm cho nó hữu ích cho việc kiểm thử phần mềm trong các lĩnh vực khác.

Từ một quan điểm nào đó, một số kỹ thuật được đề xuất có thể được áp dụng cho các loại phần mềm khác. Ví dụ, trong bài báo đề xuất để kiểm thử các thư viện học sâu [58], vì nó đề xuất các kỹ thuật để tạo ra các chương trình DL đa dạng, phức tạp và giống con người, các tác giả cho rằng phương pháp này có thể dễ dàng mở rộng để kiểm thử các hệ thống phần mềm từ các lĩnh vực ứng dụng khác, ví dụ như trình thông dịch, hệ thống cơ sở dữ liệu và các thư viện phổ biến khác. Hơn thế nữa, đã có những nghiên cứu tập trung vào các kỹ thuật làm mờ phổ quát [52], [67] được thiết kế để có thể thích ứng và áp dụng cho các loại đầu vào thử nghiệm và phần mềm khác nhau.

Từ một góc nhìn khác, các loại phần mềm khác cũng có thể hưởng lợi từ khả năng của LLM (Learning Learning Module) trong việc thiết kế các kỹ thuật kiểm thử phù hợp hơn với lĩnh vực và đặc điểm cụ thể của chúng. Ví dụ, metaverse, với môi trường ảo sống động và tương tác phức tạp, đặt ra những thách thức độc đáo cho việc kiểm thử phần mềm. LLM có thể được tận dụng để tạo ra các đầu vào đa dạng và thực tế, mô phỏng hành vi và tương tác của người dùng trong metaverse, những khía cạnh chưa từng được khám phá.

6.2.4 Khám phá LLM cho Kiểm thử phi chức năng

Trong các nghiên cứu đã thu thập của chúng tôi, LLM chủ yếu được sử dụng để kiểm thử chức năng và không có hoạt động nào trong kiểm thử hiệu năng, kiểm thử khả năng sử dụng hoặc các hoạt động khác. Một lý do có thể giải thích cho sự phổ biến của các giải pháp dựa trên LLM trong kiểm thử chức năng là chúng có thể chuyển đổi các vấn đề kiểm thử chức năng thành các vấn đề tạo mã hoặc tạo ngôn ngữ tự nhiên [14], [59], mà LLM đặc biệt giỏi trong việc giải quyết.

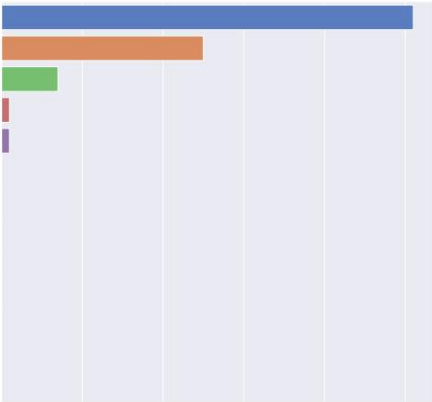
Mặt khác, kiểm thử hiệu năng và kiểm thử khả năng sử dụng có thể yêu cầu các mô hình chuyên biệt hơn, được thiết kế để phát hiện và phân tích các loại dữ liệu cụ thể, xử lý các phân tích thống kê phức tạp hoặc xác định các tiêu chí gây lỗi.

Hơn nữa, đã có hàng tá công cụ kiểm tra hiệu năng (ví dụ: LoadRunner [156]) có thể tạo ra khối lượng công việc mô phỏng các kịch bản sử dụng thực tế và đạt được hiệu năng tương đối tốt.

Các cơ hội tiềm năng có thể cho phép LLM tích hợp các công cụ kiểm thử hiệu năng và hoạt động như LangChain [157], để mô phỏng tốt hơn các loại khối lượng công việc khác nhau dựa trên hành vi người dùng thực. Hơn nữa, LLM có thể xác định các tổ hợp tham số và giá trị có tiềm năng cao nhất gây ra các vấn đề về hiệu năng. Về cơ bản, đây là một cách để xếp hạng và ưu tiên các thiết lập tham số khác nhau dựa trên tác động của chúng đến hiệu năng và cải thiện hiệu quả của việc kiểm thử hiệu năng.

6.2.5 Khám phá Kỹ thuật Nhắc nhở Nâng cao Có tổng cộng

11 kỹ thuật nhắc nhở thường được sử dụng như được liệt kê trong một hướng dẫn kỹ thuật nhắc nhở phổ biến [158], như được hiển thị trong Hình 12. Hiện tại, trong các nghiên cứu đã thu thập của chúng tôi, chỉ có năm kỹ thuật đầu tiên đang được sử dụng. Các kỹ thuật nâng cao hơn vẫn chưa được sử dụng và có thể được khám phá trong tương lai cho thiết kế nhắc nhở.



Hình 12: Danh sách các phương pháp kỹ thuật gợi ý nâng cao và những phương pháp được sử dụng trong các bài

báo đã thu thập. Ví dụ, gợi ý chuỗi suy nghĩ đa phương thức liên quan đến việc sử dụng các tín hiệu cảm giác và nhận thức đa dạng để kích thích tư duy và sự sáng tạo ở LLM [159]. Bảng cách cung cấp hình ảnh (ví dụ: ảnh chụp màn hình GUI) hoặc bản ghi âm liên quan đến phần mềm đang được thử nghiệm có thể giúp LLM hiểu rõ hơn về bối cảnh của phần mềm và các vấn đề tiềm ẩn. Bên cạnh đó, hãy cố gắng gợi ý cho LLM tưởng tượng mình ở các vai trò khác nhau, chẳng hạn như nhà phát triển, người dùng hoặc chuyên gia đảm bảo chất lượng. Bài tập thay đổi quan điểm này cho phép LLM tiếp cận việc kiểm thử phần mềm từ nhiều góc độ và khám phá các khía cạnh khác nhau có thể cần được chú ý hoặc điều tra.

Gợi ý đồ thị [160] liên quan đến việc biểu diễn thông tin bằng cách sử dụng đồ thị hoặc cấu trúc trực quan để tạo điều kiện thuận lợi cho việc hiểu và giải quyết vấn đề. Gợi ý đồ thị có thể phù hợp tự nhiên với kỹ thuật phần mềm, vì nó liên quan đến nhiều mối quan hệ phụ thuộc, luồng điều khiển, luồng dữ liệu, chuyển đổi trạng thái hoặc cấu trúc đồ thị liên quan khác. Gợi ý đồ thị có thể có lợi trong việc phân tích thông tin cấu trúc này và cho phép LLM hiểu phần mềm đang được kiểm thử một cách hiệu quả. Ví dụ, người kiểm thử có thể sử dụng gợi ý đồ thị để hình dung phạm vi kiểm thử, xác định các khu vực hoặc đường dẫn chưa được kiểm thử và đảm bảo thực hiện kiểm thử đầy đủ.

6.2.6 Kết hợp LLM với các kỹ thuật truyền thống Hiện nay chưa có sự

đồng thuận rõ ràng về mức độ mà LLM có thể giải quyết các vấn đề kiểm thử phần mềm. Từ phân tích trong Phần 5.4, chúng ta đã thấy một số kết quả khả quan từ các nghiên cứu kết hợp LLM với các kỹ thuật kiểm thử phần mềm truyền thống. Điều này ngụ ý rằng LLM không phải là giải pháp duy nhất cho kiểm thử phần mềm. Xét đến sự sẵn có của nhiều kỹ thuật và công cụ kiểm thử phần mềm đã hoàn thiện, và khả năng hạn chế của LLM, cần phải tìm kiếm những cách tốt hơn để kết hợp LLM với các kỹ thuật và công cụ kiểm thử hoặc phân tích chương trình truyền thống nhằm đạt được hiệu quả kiểm thử phần mềm tốt hơn.

Dựa trên các nghiên cứu đã thu thập, LLM đã được sử dụng thành công cùng với nhiều kỹ thuật khác nhau như kiểm thử khác biệt (ví dụ, [63]), kiểm thử đột biến (ví dụ, [59]), phân tích chương trình (ví dụ, [104], như được hiển thị trong Hình 10). Từ một góc nhìn, các nghiên cứu trong tương lai có thể khám phá sự tích hợp được cải tiến của các kỹ thuật truyền thống này với LLM. Lấy kiểm thử đột biến làm ví dụ, các thực tiễn hiện tại chủ yếu dựa vào các quy tắc đột biến do con người thiết kế để đột biến các bài kiểm thử ứng cử viên và để LLM tạo lại các bài kiểm thử mới [38], [59], [67], trong khi Liu et al. trực tiếp sử dụng LLM để tạo ra các quy tắc đột biến cùng với các bài kiểm thử đột biến [65].

Việc tiếp tục nghiên cứu theo hướng này rất đáng quan tâm.

Từ một quan điểm khác, các kỹ thuật truyền thống hơn có thể được kết hợp trong LLM để kiểm thử phần mềm. Ví dụ, bên cạnh các kỹ thuật truyền thống đã đề cập, LLM đã được kết hợp với xác minh hình thức để phát hiện phần mềm tự phục hồi trong lĩnh vực an ninh phần mềm [161]. Cần khuyến khích thêm nhiều nỗ lực. Hơn nữa, xét đến sự tồn tại của nhiều công cụ kiểm thử phần mềm hoàn thiện, người ta có thể khám phá việc tích hợp LLM với các công cụ này, cho phép chúng hoạt động như một “Chuỗi ngôn ngữ” để khai thác tốt hơn tiềm năng của các công cụ này.

7 CÔNG VIỆC LIÊN QUAN

Tổng quan tài liệu có hệ thống là một phương pháp quan trọng để hiểu rõ các xu hướng hiện tại và định hướng tương lai trong một lĩnh vực cụ thể. Nó giúp chúng ta nắm bắt và cập nhật những phát triển trong lĩnh vực đó.

Wang et al. đã khảo sát các kỹ thuật học máy và học sâu cho kỹ thuật phần mềm [162]. Yang et al. và Watson et al. lần lượt thực hiện các khảo sát về việc sử dụng học sâu trong lĩnh vực kỹ thuật phần mềm [163], [164]. Bajammal et al. đã khảo sát việc sử dụng các kỹ thuật thị giác máy tính để cải thiện các nhiệm vụ kỹ thuật phần mềm [165]. Zhang et al. đã cung cấp một khảo sát về các kỹ thuật để kiểm thử các hệ thống học máy [150]

Với sự phát triển của trí tuệ nhân tạo và LLM, các nhà nghiên cứu cũng tiến hành đánh giá tài liệu có hệ thống về LLM và các ứng dụng của chúng trong nhiều lĩnh vực (ví dụ: kỹ thuật phần mềm). Zhao et al. [17] đã xem xét những tiến bộ gần đây trong LLM bằng cách cung cấp tổng quan về bối cảnh, phát hiện chính và các kỹ thuật chính thống của chúng.

Họ tập trung vào bốn khía cạnh chính của LLM, cụ thể là đào tạo trước, điều chỉnh thích ứng, sử dụng và đánh giá năng lực. Ngoài ra, họ đã tóm tắt các nguồn lực hiện có để phát triển LLM và thảo luận về các vấn đề còn lại cho các hướng đi trong tương lai. Hou et al. đã tiến hành một đánh giá tài liệu có hệ thống về việc sử dụng LLM cho kỹ thuật phần mềm, với trọng tâm đặc biệt là hiểu cách LLM có thể được khai thác để tối ưu hóa các quy trình và kết quả [166]. Fan et al. đã tiến hành một cuộc khảo sát về LLM cho kỹ thuật phần mềm và đặt ra các thách thức nghiên cứu mở cho việc ứng dụng LLM vào các vấn đề kỹ thuật mà các kỹ sư phần mềm gặp phải [167]. Zan et al. đã tiến hành một cuộc khảo sát về các LLM hiện có cho nhiệm vụ NL2Code (tức là tạo mã từ mô tả ngôn ngữ tự nhiên) và xem xét các tiêu chuẩn và số liệu [168].

Trong khi các nghiên cứu này hoặc nhắm vào lĩnh vực kỹ thuật phần mềm rộng hơn (với trọng tâm hạn chế vào các nhiệm vụ kiểm thử phần mềm) hoặc tập trung vào các nhiệm vụ phát triển phần mềm khác.

(Không bao gồm kiểm thử phần mềm), bài báo này tập trung cụ thể vào việc sử dụng LLM cho kiểm thử phần mềm. Bài báo khảo sát các nghiên cứu liên quan, tóm tắt các thách thức chính và cơ hội tiềm năng, đồng thời đóng vai trò là lộ trình cho nghiên cứu trong tương lai về lĩnh vực này. khu vực.

8. KẾT LUẬN Bài báo này

cung cấp một đánh giá toàn diện về việc sử dụng LLM trong kiểm thử phần mềm. Chúng tôi đã phân tích các nghiên cứu liên quan đã sử dụng LLM trong kiểm thử phần mềm từ cả góc độ kiểm thử phần mềm và góc độ LLM. Bài báo này cũng nêu bật những thách thức và cơ hội tiềm năng theo hướng này. Kết quả của đánh giá này cho thấy rằng LLM đã được áp dụng thành công trong nhiều nhiệm vụ kiểm thử, bao gồm tạo trường hợp kiểm thử đơn vị, tạo oracle kiểm thử, tạo đầu vào kiểm thử hệ thống, gỡ lỗi chương trình và sửa chữa chương trình. Tuy nhiên, vẫn còn những thách thức trong việc đạt được độ phủ kiểm thử cao, giải quyết vấn đề oracle kiểm thử, tiến hành đánh giá nghiêm ngặt và áp dụng LLM trong các tình huống thực tế. Ngoài ra, người ta nhận thấy rằng LLM thường chỉ được sử dụng trong một phần nhỏ của toàn bộ vòng đời kiểm thử, ví dụ, chúng chủ yếu được sử dụng ở giai đoạn giữa và cuối của quá trình kiểm thử, chỉ phục vụ các giai đoạn kiểm thử đơn vị và hệ thống, và chỉ dành cho kiểm thử chức năng. Điều này làm nổi bật các cơ hội nghiên cứu để khám phá các lĩnh vực chưa được khai thác. Về cách thức sử dụng LLM, chúng tôi nhận thấy rằng nhiều phương pháp huấn luyện trước/tinh chỉnh và kỹ thuật nhanh chóng đã được phát triển để nâng cao khả năng của LLM trong việc giải quyết các nhiệm vụ kiểm thử. Tuy nhiên, các kỹ thuật tiên tiến hơn trong thiết kế lời nhắc vẫn chưa được khám phá và có thể là hướng nghiên cứu trong tương lai.

Bài viết này có thể đóng vai trò như một lộ trình cho các nghiên cứu trong tương lai trong lĩnh vực này, xác định những khoảng trống trong hiểu biết hiện tại của chúng ta về việc sử dụng LLM trong kiểm thử phần mềm và nêu bật những hướng nghiên cứu tiềm năng. Chúng tôi tin rằng những hiểu biết được cung cấp trong bài viết này sẽ có giá trị đối với cả các nhà nghiên cứu và những người thực hành trong lĩnh vực kỹ thuật phần mềm, giúp họ tận dụng LLM để cải thiện các phương pháp kiểm thử phần mềm và cuối cùng là nâng cao chất lượng và độ tin cậy của các hệ thống phần mềm.

TÀI LIỆU THAM KHẢO

[1] GJ Myers, Nghệ thuật kiểm thử phần mềm (ấn bản thứ 2). Wiley, 2004. [Trực tuyến]. Có sẵn tại: <http://eu.wiley.com/WileyCDA/WileyTitle/productCd-0471469122.html> [2] M. Pezze và M. Young, Kiểm thử và phân tích phần mềm - quy trình, nguyên tắc và kỹ thuật. Wiley, 2007. [3] M. Hazman và P. McMin, “Nghiên cứu lý thuyết và thực nghiệm về thử nghiệm dựa trên tìm kiếm: Tìm kiếm cục bộ, toàn cục và kết hợp,” tập 36, số 2, 2010, trang 226-247. [4] P. Delgado-Perez, A. Ram ırez, KJ Valle-Gomez, I. Medina- Bullo và JR Romero, “Interevo-tr: Tạo kiểm thử tiến hóa tương tác với đánh giá khả năng đọc hiểu,” IEEE Trans. Software Eng., tập 49, số 4, trang 2580-2596, 2023. [5] X. Xiao, S. Li, T. Xie và N. Tillmann, “Nghiên cứu đặc điểm của các vấn đề vòng lặp đối với việc tạo kiểm thử cấu trúc thông qua thực thi tượng trưng,” trong Hội nghị quốc tế IEEE/ACM lần thứ 28 về Kỹ thuật phần mềm tự động hóa, ASE 2013, Thung lũng Silicon, CA, Hoa Kỳ, ngày 11-15 tháng 11 năm 2013, E. Denney, T. Bultan và A. Zeller, Biên tập viên. IEEE, 2013, trang 246-256. [6] C. Pacheco, SK Lahiri, MD Ernst và T. Ball, “Tạo thử nghiệm ngẫu nhiên theo hướng phản hồi,” trong Hội nghị Quốc tế lần thứ 29 về Kỹ thuật Phần mềm (ICSE 2007), Minneapolis, MN, Hoa Kỳ, ngày 20-26 tháng 5 năm 2007. IEEE Computer Society, 2007, trang 75-84.

[7] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen và X. Peng, “Không còn kiểm thử thủ công nữa? Đánh giá và cải tiến chatgpt để tạo kiểm thử đơn vị,” bản thảo arXiv arXiv:2305.04207, 2023.

[8] Y. Tang, Z. Liu, Z. Zhou và X. Luo, “Chatgpt so với SBST: Đánh giá so sánh về việc tạo bộ kiểm thử đơn vị,” CoRR, tập abs/2307.00588, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2307.00588> [9] A. Developers, “Ui/application exerciser monkey,” 2012.

[10] Y. Li, Z. Yang, Y. Guo và X. Chen, “Droidbot: một trình tạo đầu vào thử nghiệm hướng dẫn bằng giao diện người dùng nhẹ cho Android,” trong ICSE. IEEE, 2017.

[11] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu và Z. Su, “Kiểm thử GUI dựa trên mô hình ngẫu nhiên có hướng dẫn của ứng dụng Android,” trong Kỷ yếu Hội nghị chung lần thứ 11 về Cơ sở kỹ thuật phần mềm năm 2017, 2017, trang 245-256.

[12] Z. Dong, M. Bohme, L. Cojocar và A. Roychoudhury, “Thử nghiệm du hành thời gian của các ứng dụng Android,” trong ICSE. IEEE, 2020.

[13] M. Pan, A. Huang, G. Wang, T. Zhang và X. Li, “Kiểm thử ứng dụng Android dựa trên sự tò mò bằng học tăng cường,” trong Kỷ yếu Hội nghị chuyên đề quốc tế ACM SIGSOFT lần thứ 29 về Kiểm thử và Phân tích Phần mềm, 2020, trang 153-164.

[14] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang và Q. Wang, “Biến LLM thành chuyên gia kiểm thử: Mang tương tác giống con người vào kiểm thử GUI di động thông qua các quyết định nhận biết chức năng,” CoRR, tập. abs/2310.15780, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.15780> [15] T. Su, J. Wang và Z. Su, “Đánh giá hiệu năng kiểm thử GUI tự động cho Android so với các lỗi thực tế,” trong ESEC/FSE '21: Hội nghị và Hội thảo chung về Kỹ thuật Phần mềm Châu Âu lần thứ 29 của ACM về Nền tảng Kỹ thuật Phần mềm, Athens, Hy Lạp, ngày 23-28 tháng 8 năm 2021. ACM, 2021, trang 119-130.

[16] M. Shanahan, “Nói về các mô hình ngôn ngữ lớn,” CoRR, tập abs/2212.03551, 2022. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2212.03551> [17] WX Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J. Nie và J. Wen, “Khảo sát về các mô hình ngôn ngữ lớn,” CoRR, tập abs/2303.18223, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2303.18223> [18] T. Kojima, SS Gu, M. Reid, Y. Matsuo và Y. Iwasawa, “Các mô hình ngôn ngữ lớn là những bộ suy luận không cần huấn luyện,” trong NeurIPS, 2022. [Trực tuyến]. Có sẵn: <http://papers.nips.cc/paper files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html> [19] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, EH Chi, QV Le và D. Zhou, “Gợi ý chuỗi suy nghĩ khơi gợi suy luận trong các mô hình ngôn ngữ lớn,” trong NeurIPS, 2022. [Trực tuyến]. Có sẵn tại: <http://papers.nips.cc/paper files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>

[20] J. Li, G. Li, Y. Li và Z. Jin, “Gợi ý chuỗi suy nghĩ có cấu trúc để tạo mã,” 2023. [Trực tuyến]. Có sẵn tại: <https://api.semanticscholar.org/CorpusID:258615421>

[21] J. Li, Y. Li, G. Li, Z. Jin, Y. Hao và X. Hu, “Skcoder: Một phương pháp dựa trên phác thảo để tạo mã tự động,” trong Hội nghị quốc tế IEEE/ACM lần thứ 45 về Kỹ thuật phần mềm (ICSE), 2023, trang 2124-2135.

[22] J. Li, Y. Zhao, Y. Li, G. Li và Z. Jin, “Acocoder: Sử dụng mã hiện có để tăng cường tạo mã,” 2023. [Trực tuyến]. Có sẵn tại: <https://api.semanticscholar.org/CorpusID:257901190>

[23] Y. Dong, X. Jiang, Z. Jin và G. Li, “Tạo mã tự hợp tác thông qua chatgpt,” CoRR, tập. abs/2304.07590, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2304.07590> [24] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang và X. Wu, “Tổng nhất các mô hình ngôn ngữ lớn và đồ thị tri thức: Một lộ trình,” CoRR, tập. abs/2306.08302, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2306.08302> [25] GJ Myers, T. Badgett, TM Thomas và C. Sandler, Nghệ thuật kiểm thử phần mềm. Thư viện trực tuyến Wiley, 2004, tập. 2.

[26] M. Tufano, D. Drain, A. Svyatkovskiy, SK Deng và N. Sun-daresan, “Tạo trường hợp kiểm thử đơn vị với bộ chuyển đổi và ngữ cảnh trọng tâm,” bản in trước arXiv arXiv:2009.05617, 2020.

[27] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, và W. Chen, “Codet: Tạo mã với các bài kiểm tra được tạo ra,” bản in trước arXiv arXiv:2207.10397, 2022.

[28] SK Lahiri, A. Naik, G. Sakkas, P. Choudhury, C. von Veh, M. Musuvathi, JP Inala, C. Wang và J. Gao, “Tương tác “Tạo mã thông qua hình thức hóa ý định người dùng dựa trên kiểm thử,” bản thảo arXiv arXiv:2208.05950, 2022.

[29] S. Alagarsamy, C. Tantithamthavorn và A. Aleti, “A3test: Tạo trường hợp kiểm thử tự động được tăng cường bằng khẳng định,” bản in trước arXiv arXiv:2302.10352, 2023.

[30] M. Schafer, S. Nadi, A. Eghbali và F. Tip, “Đánh giá thực nghiệm về việc sử dụng các mô hình ngôn ngữ lớn để tạo kiểm thử đơn vị tự động,” Giao dịch IEEE về Kỹ thuật Phần mềm, trang 1-21, 2023.

[31] V. Guilherme và A. Vincenzi, “Một cuộc điều tra ban đầu về khả năng tạo kiểm thử đơn vị chatgpt,” trong Hội nghị chuyên đề Brazil lần thứ 8 về Kiểm thử phần mềm tự động và có hệ thống, SAST 2023, Campo Grande, MS, Brazil, ngày 25-29 tháng 9 năm 2023, AL Fontao, DMB Paiva, H. Borges, MI Cagnin, PG Fernandes, V. Borges, SM Melo, VHS Durelli và ED Canedo, Eds. ACM, 2023, trang 15-24. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3624032.3624035> [32] S. Hashtroudi, J. Shin, H. Hemmati và S. Wang, “Tạo trường hợp kiểm thử tự động bằng cách sử dụng mô hình mã và thích ứng miễn,” CoRR, tập abs/2308.08033, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.08033> [33] L. Plein, WC Ouedraogo, J. Klein, và TF Bissyand e, “Tự động tạo trường hợp thử nghiệm dựa trên báo cáo lỗi: nghiên cứu khả thi với các mô hình ngôn ngữ lớn,” CoRR, tập. abs/2310.06320, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.06320> [34] V. Vikram, C. Lemieux, và R. Padhye, “Các mô hình ngôn ngữ lớn có thể viết các bài kiểm tra dựa trên thuộc tính tốt không?” CoRR, tập. abs/2307.04346, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2307.04346>

[35] N. Rao, K. Jain, U. Alon, CL Goues và VJ Hellendoorn, “CAT-LM đào tạo mô hình ngôn ngữ trên mã và bài kiểm tra được cân chỉnh,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 38 về Kỹ thuật Phần mềm Tự động, ASE 2023, Luxembourg, ngày 11-15 tháng 9 năm 2023. IEEE, 2023, trang 409-420. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/ASE56229.2023.00193> [36] Z. Xie, Y. Chen, C. Zhi, S. Deng và J. Yin, “Chatunitest: một công cụ tạo bài kiểm tra đơn vị tự động dựa trên chatgpt,” bản in trước arXiv arXiv:2305.04764, 2023.

[37] C. Lemieux, JP Inala, SK Lahiri và S. Sen, “Codamosa: Thoát khỏi tình trạng bão hòa độ phủ trong quá trình tạo thử nghiệm với các mô hình ngôn ngữ lớn được đào tạo trước,” trong Hội nghị quốc tế về kỹ thuật phần mềm (ICSE), 2023.

[38] AM Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh và MC Desmarais, “Tạo thử nghiệm hiệu quả bằng cách sử dụng các mô hình ngôn ngữ lớn được đào tạo trước và thử nghiệm đột biến,” CoRR, tập abs/2308.16557, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2308.16557> [39] ML Siddiq, J. Santos, RH Tanvir, N. Ulfat, FA Rifat và VC Lopes, “Khám phá hiệu quả của các mô hình ngôn ngữ lớn trong việc tạo ra các bài kiểm thử đơn vị,” bản thảo arXiv arXiv:2305.00418, 2023.

[40] Y. Zhang, W. Song, Z. Ji, D. Yao và N. Meng, “LLM tạo ra các bài kiểm tra bảo mật tốt đến mức nào?” CoRR, tập. abs/2310.00710, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.00710> [41] V. Li và N. Doiron, “Thúc đẩy trình thông dịch mã viết các bài kiểm tra đơn vị tốt hơn trên các hàm quixbugs,” CoRR, tập. abs/2310.00483, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.00483>

[42] B. Steenhoeck, M. Tufano, N. Sundaresan và A. Svyatkovskiy, “Học tăng cường từ phản hồi tự động để tạo ra bài kiểm tra đơn vị chất lượng cao,” 2023.

[43] S. Bhatia, T. Gandhi, D. Kumar và P. Jalote, “Tạo bài kiểm tra đơn vị bằng trí tuệ nhân tạo tạo sinh: Phân tích hiệu suất so sánh của các công cụ tự động tạo sinh,” 2023.

[44] M. Tufano, D. Drain, A. Svyatkovskiy và N. Sundaresan, “Tạo câu lệnh khẳng định chính xác cho các trường hợp kiểm thử đơn vị bằng cách sử dụng bộ chuyển đổi được đào tạo trước,” trong Kỷ yếu Hội nghị Quốc tế ACM/IEEE lần thứ 3 về Tự động hóa Kiểm thử Phần mềm, 2022, trang 54-64.

[45] P. Nie, R. Banerjee, JJ Li, RJ Mooney và M. Gligoric, “Học ngữ nghĩa sâu để hoàn thành bài kiểm tra,” bản in trước arXiv arXiv:2302.10166, 2023.

[46] A. Mastropaolo, N. Cooper, D. Nader-Palacio, S. Scalabrino, D. Poshvanyk, R. Oliveto và G. Bavota, “Sử dụng học chuyển giao cho các nhiệm vụ liên quan đến mã,” IEEE Trans. Software Eng., tập 49, số 4, trang 1580-1598, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.1109/TSE.2022.3183297>

[47] N. Nashid, M. Sintaha và A. Mesbah, “Lựa chọn gợi ý dựa trên truy xuất cho việc học ít mẫu liên quan đến mã,” trong Kỷ yếu Hội nghị Quốc tế lần thứ 45 về Kỹ thuật Phần mềm (ICSE’23), 2023.

[48] G. Ye, Z. Tang, SH Tan, S. Huang, D. Fang, X. Sun, L. Bian, H. Wang và Z. Wang, “Kiểm thử tuân thủ tự động cho các công cụ javascript thông qua làm mờ trình biên dịch sâu,” trong Kỷ yếu Hội nghị quốc tế ACM SIGPLAN lần thứ 42 về thiết kế và triển khai ngôn ngữ lập trình, 2021, trang 435-450.

[49] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu và Q. Wang, “Điền vào chỗ trống: Tạo đầu vào văn bản tự động nhận biết ngữ cảnh để kiểm thử giao diện người dùng di động,” bản in trước arXiv arXiv:2212.04732, 2022.

[50] MR Taesiri, F. Macklon, Y. Wang, H. Shen, và C.-P. Bezemer, “Các mô hình ngôn ngữ lớn là những công cụ phát hiện lỗi trò chơi điện tử không cần dữ liệu khá tốt,” bản in trước arXiv arXiv:2210.02506, 2022.

[51] SL Shrestha và C. Csallner, “Slgpt: sử dụng học chuyển giao để trực tiếp tạo các tệp mô hình simulink và tìm lỗi trong chuỗi công cụ simulink,” trong Đánh giá và Thẩm định trong Kỹ thuật Phần mềm, 2021, trang 260-265.

[52] J. Hu, Q. Zhang và H. Yin, “Tăng cường làm mờ hộp xám bằng AI tạo sinh,” CoRR, tập. abs/2306.06782, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2306.06782>

[53] A. Mathur, S. Pradhan, P. Soni, D. Patel và R. Regunathan, “Tạo trường hợp thử nghiệm tự động bằng t5 và gpt-3,” trong Hội nghị quốc tế lần thứ 9 về Hệ thống tính toán và truyền thông tiên tiến (ICACCS) năm 2023, tập 1, 2023, trang 1986-1992.

[54] D. Zimmermann và A. Koziolok, “Tự động hóa thử nghiệm phần mềm dựa trên gui với gpt-3,” trong Hội nghị quốc tế IEEE năm 2023 về Hội thảo Kiểm thử, Xác minh và Thẩm định Phần mềm (ICSTW), 2023, trang 62-65.

[55] M. Taeb, A. Searngin, E. Schoop, R. Cheng, Y. Jiang và J. Nichols, “Axnav: Phát lại các bài kiểm tra khả năng truy cập từ ngôn ngữ tự nhiên,” CoRR, tập. abs/2310.02424, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.02424> [56] Q. Luu, H. Liu và TY Chen, “Chatgpt có thể nâng cao trí thông minh kiểm thử phần mềm không? Báo cáo kinh nghiệm về kiểm thử biến đổi,” CoRR, tập. abs/2310.19204, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.19204> [57] A. Khanfir, R. Degiovanni, M. Papadakis và YL Traon, “Kiểm thử đột biến hiệu quả thông qua các mô hình ngôn ngữ được đào tạo trước,” bản in trước arXiv arXiv:2301.03543, 2023.

[58] Y. Deng, CS Xia, C. Yang, SD Zhang, S. Yang và L. Zhang, “Các mô hình ngôn ngữ lớn là các công cụ làm mờ trường hợp ngoại lệ: Kiểm tra các thư viện học sâu thông qua fuzzgpt,” bản in trước arXiv arXiv:2304.02014, 2023.

[59] —, “Các mô hình ngôn ngữ lớn là các công cụ làm mờ không cần dữ liệu: Làm mờ các thư viện học sâu thông qua các mô hình ngôn ngữ lớn,” bản in trước arXiv arXiv:2209.11515, 2023.

[60] J. Ackerman và G. Cybenko, “Các mô hình ngôn ngữ lớn cho trình phân tích cú pháp mờ (báo cáo đã đăng ký),” trong Kỷ yếu Hội thảo mờ quốc tế lần thứ 2, FUZZING 2023, Seattle, WA, Hoa Kỳ, ngày 17 tháng 7 năm 2023, M. Bohme, Y. Noller, B. Ray và L. Szekeres, Biên tập viên. ACM, 2023, trang 31-38. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3605157.3605173> [61] S. Yu, C. Fang, Y. Ling, C. Wu và Z. Chen, “LLM để tạo và di chuyển kịch bản kiểm thử: Thách thức, khả năng và cơ hội,” CoRR, tập abs/2309.13574, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2309.13574>

[62] G. Deng, Y. Liu, VM Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger và S. Rass, “Pentestgpt: Một công cụ kiểm tra thâm nhập tự động được trao quyền bởi llm,” CoRR, tập abs/2308.06782, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.06782> [63] M. Sun, Y. Yang, Y. Wang, M. Wen, H. Jia và Y. Zhou, “Xác thực bộ giải SMT được hỗ trợ bởi các mô hình ngôn ngữ được đào tạo trước quy mô lớn,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 38 về Kỹ thuật Phần mềm Tự động, ASE 2023, Luxembourg, ngày 11-15 tháng 9 năm 2023. IEEE, 2023, trang 1288-1300. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/ASE56229.2023.00180> [64] Y. Deng, J. Yao, Z. Tu, X. Zheng, M. Zhang và T. Zhang, “Mục tiêu: Tạo kịch bản tự động từ các quy tắc giao thông để thử nghiệm xe tự hành,” 2023. [Trực tuyến]. Có sẵn: <https://api.semanticscholar.org/CorpusID:258588387>

[65] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang và Q. Wang, “Kiểm tra giới hạn: Tạo đầu vào văn bản bất thường để phát hiện sự cố ứng dụng di động với ngôn ngữ lớn mô hình,” tập. abs/2310.15657, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.15657> [66] C. Zhang, M. Bai, Y. Zheng, Y. Li, X. Xie, Y. Li, W. Ma, L. Sun và Y. Liu, “Hiểu mô hình ngôn ngữ lớn dựa trên việc tạo trình điều khiển mờ,” CoRR, tập. abs/2307.12469, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2307.12469>

[67] C. Xia, M. Paltenghi, J. Tian, M. Pradel và L. Zhang, “Kiểm thử mờ phổ quát thông qua các mô hình ngôn ngữ lớn,” ArXiv, tập. abs/2308.04748, 2023. [Trực tuyến]. Có sẵn: <https://api.semanticscholar.org/CorpusID:260735598> [68] C. Tsigkanos, P. Rani, S. Muller và T. Kehrer, “Khám phá biến với các mô hình ngôn ngữ lớn để kiểm thử biến hình phần mềm khoa học,” trong Khoa học Máy tính - ICCS 2023 - Hội nghị Quốc tế lần thứ 23, Praha, Cộng hòa Séc, ngày 3-5 tháng 7 năm 2023, Kỷ yếu, Phần I, loạt. Ghi chú bài giảng về Khoa học Máy tính, J. Mikyska, C. de Mulatier, M. Paszynski, VV Krzhizhanovskaya, JJ Dongarra, và PMA Sloot, Eds., tập 14073. Springer, 2023, trang 321-335. [Trực tuyến]. Có sẵn: https://doi.org/10.1007/978-3-031-35995-8_23 [69] C. Yang, Y. Deng, R. Lu, J. Yao, J. Liu, R. Jabbarvand, và L. Zhang, “Kiểm thử mờ trình biên dịch hộp trắng được hỗ trợ bởi các mô hình ngôn ngữ lớn,” CoRR, tập abs/2310.15991, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2310.15991>

[70] T. Zhang, IC Irsan, F. Thung, D. Han, D. Lo và L. Jiang, “itiger: một công cụ tạo tiêu đề vấn đề tự động,” trong Kỷ yếu Hội nghị Kỹ thuật Phần mềm Châu Âu chung lần thứ 30 của ACM và Hội thảo về Nền tảng Kỹ thuật Phần mềm, 2022, trang 1637-1641.

[71] Y. Huang, J. Wang, Z. Liu, Y. Wang, S. Wang, C. Chen, Y. Hu và Q. Wang, “Crasht translator: Tự động tái tạo sự cố ứng dụng di động trực tiếp từ dấu vết ngăn xếp,” CoRR, tập. abs/2310.07128, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.07128> [72] T. Zhang, IC Irsan, F. Thung và D. Lo, “Cupid: Tận dụng chatgpt để phát hiện báo cáo lỗi trùng lặp chính xác hơn,” CoRR, tập. abs/2308.10022, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.10022> [73] U. Mukherjee và MM Rahman, “Sử dụng học sâu và truy xuất thông tin có cấu trúc để trả lời các câu hỏi làm rõ về báo cáo lỗi,” 2023. [Trực tuyến]. Có sẵn: <https://api.semanticscholar.org/CorpusID:259501524>

[74] P. Mahbub, O. Shuvo và MM Rahman, “Giải thích các lỗi phần mềm bằng cách tận dụng cấu trúc mã trong dịch máy thần kinh,” bản in trước arXiv arXiv:2212.04584, 2022.

[75] S. Feng và C. Chen, “Chỉ cần nhắc nhở là đủ: Phát lại lỗi Android tự động với các mô hình ngôn ngữ lớn,” CoRR, tập abs/2306.01987, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2306.01987> [76] Y. Su, Z. Han, Z. Gao, Z. Xing, Q. Lu và X. Xu, “Vẫn còn khó hiểu khi phân loại lỗi-thành phần? Học đặc trưng sâu và thiết lập tập hợp để giải cứu,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 31 về Hiểu chương trình, ICPC 2023, Melbourne, Úc, ngày 15-16 tháng 5 năm 2023. IEEE, 2023, trang 316-327. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/ICPC58990.2023.00046> [77] ND Bui, Y. Wang và S. Hoi, “Phát hiện-định vị-sửa chữa: Một khung thống nhất để học cách gỡ lỗi với codet5,” bản in trước arXiv arXiv:2211.14875, 2022.

[78] S. Kang, J. Yoon và S. Yoo, “Các mô hình ngôn ngữ lớn là những công cụ kiểm thử ít lần: Khám phá việc tái tạo lỗi chung dựa trên llm,” bản in trước arXiv arXiv:2209.11515, 2022.

[79] S. Kang, G. An và S. Yoo, “Đánh giá sơ bộ về định vị lỗi dựa trên llm,” CoRR, tập abs/2308.05487, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.05487>

[80] P. Widjojo và C. Treude, “Giải quyết lỗi trình biên dịch: Trần ngăn xếp hay mô hình ngôn ngữ lớn?” CoRR, tập. abs/2307.10793, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2307.10793>

[81] L. Plein và TF Bissyande, “Liệu llms có thể giải mã các báo cáo lỗi?” CoRR, tập. abs/2310.06310, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2310.06310> [82] A. Taylor, A. Vassar, J. Renzella và HA Pearce, “Dcc -help: Tạo ra các giải thích lỗi trình biên dịch nhận biết ngữ cảnh với các mô hình ngôn ngữ lớn,” 2023. [Trực tuyến]. Có sẵn: <https://api.semanticscholar.org/CorpusID:261076439>

[83] S. Kang, B. Chen, S. Yoo, và J.-G. Lou, “Gỡ lỗi tự động có thể giải thích được thông qua gỡ lỗi khoa học dựa trên mô hình ngôn ngữ lớn,” bản in trước arXiv arXiv:2304.02195, 2023.

[84] AZH Yang, R. Martins, CL Goues và VJ Hellendoorn, “Các mô hình ngôn ngữ lớn để định vị lỗi không cần kiểm thử,” CoRR, tập abs/2310.01726, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2310.01726>

[85] Y. Wu, Z. Li, JM Zhang, M. Papadakis, M. Harman và Y. Liu, “Các mô hình ngôn ngữ lớn trong việc bản địa hóa lỗi,” CoRR, tập abs/2308.15276, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.15276> [86] H. Tu, Z. Zhou, H. Jiang, INB Yusuf, Y. Li và L. Jiang, “LLM4CBI: thuần hóa llms để tạo ra các chương trình kiểm thử hiệu quả nhằm cô lập lỗi trình biên dịch,” CoRR, tập. abs/2307.00593, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2307.00593> [87] T.-O. Li, W. Zong, Y. Wang, H. Tian, Y. Wang, S.-C. Cheung và J. Kramer, “Những sắc thái là chìa khóa: Mở khóa chatgpt để tìm các bài kiểm tra gây lỗi với lời nhắc khác biệt,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 38 về Kỹ thuật Phần mềm Tự động (ASE), 2023, trang 14-26.

...

[88] X. Chen, M. Lin, N. Scharli và D. Zhou, “Dạy các mô hình ngôn ngữ lớn tự gỡ lỗi,” CoRR, tập. abs/2304.05128, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2304.05128> [89] J. Cao, M. Li, M. Wen và S.-c. Cheung, “Nghiên cứu về thiết kế nhắc nhở, ưu điểm và hạn chế của chatgpt để sửa chữa chương trình học sâu,” bản in trước arXiv arXiv:2304.08191, 2023.

[90] H. Pearce, B. Tan, B. Ahmad, R. Karri và B. Dolan-Gavitt, “Kiểm tra việc sửa chữa lỗi hồng không cần huấn luyện với các mô hình ngôn ngữ lớn,” trong Hội nghị chuyên đề IEEE về An ninh và Quyền riêng tư năm 2023 (SP). Hội Tin học IEEE, 2022, trang 1-18.

[91] Z. Fan, X. Gao, A. Roychoudhury và SH Tan, “Sửa chữa tự động các chương trình từ các mô hình ngôn ngữ lớn,” bản in trước arXiv arXiv:2205.10583, 2022.

[92] Y. Hu, X. Shi, Q. Zhou, và L. Pike, “Sửa lỗi bằng bộ chuyển đổi thông qua ngữ pháp chỉnh sửa thần kinh-ký hiệu,” bản in trước arXiv arXiv:2204.06643, 2022.

[93] CS Xia, Y. Wei và L. Zhang, “Sửa chữa chương trình thực tế trong kỷ nguyên của các mô hình ngôn ngữ được đào tạo trước quy mô lớn,” bản in trước arXiv arXiv:2210.14179, 2022.

[94] J. Zhang, J. Cambroneiro, S. Gulwani, V. Le, R. Piskac, G. Soares và G. Verbruggen, “Sửa lỗi trong bài tập python bằng mô hình ngôn ngữ lớn,” bản in trước arXiv arXiv:2209.14876, 2022.

[95] M. Lajko, V. Csuvi và L. Vidacs, “Hướng tới việc sửa chữa chương trình javascript bằng bộ chuyển đổi được đào tạo trước tạo sinh (gpt-2),” trong Kỷ yếu Hội thảo Quốc tế lần thứ ba về Sửa chữa Chương trình Tự động, 2022, trang 61-68.

[96] D. Sobania, M. Briesch, C. Hanna và J. Petke, “Phân tích hiệu suất sửa lỗi tự động của chatgpt,” bản in trước arXiv arXiv:2301.08653, 2023.

[97] K. Huang, X. Meng, J. Zhang, Y. Liu, W. Wang, S. Li và Y. Zhang, “Nghiên cứu thực nghiệm về việc tinh chỉnh các mô hình ngôn ngữ lớn của mã để sửa chữa chương trình tự động,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 38 về Kỹ thuật Phần mềm Tự động, ASE 2023, Luxembourg, ngày 11-15 tháng 9 năm 2023. IEEE, 2023, trang 1162-1174. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/ASE56229.2023.00181> [98] MC Wuisang, N. Kurniawan, KA Wira Santosa, A. Agung Santoso Gunawan và KE Saputra, “Đánh giá hiệu quả của chatgpt của openai để tự động sửa lỗi chương trình python bằng quixbugs,” trong Hội thảo quốc tế năm 2023 về Ứng dụng Công nghệ Thông tin và Truyền thông (iSemantic), 2023, trang 295-300.

[99] D. Horvath, V. Csuvi, T. Gyimóthy, và L. Vidacs, “Một nghiên cứu chuyên sâu về kiến trúc mô hình và biểu diễn chương trình trong lĩnh vực sửa chữa chương trình tự động dựa trên học máy,” trong Hội thảo quốc tế IEEE/ACM về Sửa chữa chương trình tự động, APR@ICSE 2023, Melbourne, Úc, ngày 16 tháng 5 năm 2023. IEEE, 2023, trang 31-38. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/APR59189.2023.00013> [100] JA Prenner, H. Babii và R. Robbes, “Liệu codex của openai có thể sửa lỗi không? Một đánh giá về quixbugs,” trong Kỷ yếu Hội thảo quốc tế lần thứ ba về Sửa chữa chương trình tự động, 2022, trang 69-75.

[101] W. Yuan, Q. Zhang, T. He, C. Fang, NQV Hung, X. Hao và H. Yin, “Circle: sửa chữa liên tục trên các ngôn ngữ lập trình,” trong Kỷ yếu Hội nghị chuyên đề quốc tế ACM SIGSOFT lần thứ 31 về Kiểm thử và Phân tích Phần mềm, 2022, trang 678-690.

[102] S. Moon, Y. Song, H. Chae, D. Kang, T. Kwon, KT iunn Ong, S. won Hwang và J. Yeo, “Cà phê: Tăng cường mã của bạn bằng cách sửa lỗi với phản hồi,” 2023.

[103] Y. Wei, CS Xia, và L. Zhang, “Điều khiển các phi công phụ: Kết hợp các mô hình ngôn ngữ lớn với các công cụ hoàn thành để sửa chữa chương trình tự động,” trong Kỷ yếu Hội nghị Kỹ thuật Phần mềm Châu Âu chung lần thứ 31 của ACM và Hội thảo về Nền tảng Kỹ thuật Phần mềm, ESEC/FSE 2023, San Francisco, CA, Hoa Kỳ, ngày 3-9 tháng 12 năm 2023, S. Chandra, K. Blincoe, và P. Tonella, Biên tập viên. ACM, 2023, trang 172-184. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3611643.3616271> [104] Y. Peng, S. Gao, C. Gao, Y. Huo, và MR Lyu, “Kiến thức miền rất quan trọng: Cải thiện lời nhắc với các mẫu sửa lỗi để sửa lỗi kiểu Python,” CoRR, tập. abs/2306.01394, 2023.

[Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2306.01394> [105] AEI Brownlee, J. Callan, K. Even-Mendoza, A. Geiger, C. Hanna, J. Petke, F. Sarro và D. Sobania, “Tăng cường đột biến cải tiến di truyền bằng cách sử dụng các mô hình ngôn ngữ lớn,” trong Kỹ thuật phần mềm dựa trên tìm kiếm - Hội nghị chuyên đề quốc tế lần thứ 15, SSBSE 2023, San Francisco, CA, Hoa Kỳ, ngày 8 tháng 12 năm 2023, Kỷ yếu, loạt Bài giảng về Khoa học máy tính, P. Arcaini, T. Yue và EM Fredericks, Biên tập viên, tập 14415. Springer, 2023, trang 153-159. [Trực tuyến]. Có sẵn: https://doi.org/10.1007/978-3-031-48796-5_13 [106] MMA Haque, WU Ahmad, I. Lourentzou và C. Brown, “Fixeval: Đánh giá dựa trên thực thi các bản sửa lỗi chương trình cho các vấn đề lập trình,” trong Hội thảo quốc tế IEEE/ACM về Sửa chữa chương trình tự động, APR@ICSE 2023, Melbourne, Úc, ngày 16 tháng 5 năm 2023. IEEE, 2023, trang 11-18. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/APR59189.2023.00009> [107] B. Ahmad, S. Thakur, B. Tan, R. Karri và H. Pearce, “Khắc phục lỗi bảo mật phần cứng bằng các mô hình ngôn ngữ lớn,” bản in trước arXiv arXiv:2302.01215, 2023.

[108] P. Deligiannis, A. Lal, N. Mehrotra và A. Rastogi, “Sửa lỗi biên dịch rust bằng llms,” CoRR, tập abs/2308.05177, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.05177> [109] F. Ribeiro, R. Abreu và J. Saraiva, “Định hình việc sửa chữa chương trình như là hoàn thành mã,” trong Kỷ yếu Hội thảo Quốc tế lần thứ ba về Sửa chữa Chương trình Tự động, 2022, trang 38-45.

[110] N. Wadhwa, J. Pradhan, A. Sonwane, SP Sahu, N. Natarajan, A. Kanade, S. Parthasarathy và SK Rajamani, “Bức bối với các vấn đề về chất lượng mã? llms có thể giúp!” CoRR, tập. abs/2309.12938, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2309.12938>

[111] F. Ribeiro, JNC de Macedo, K. Tsushima, R. Abreu và J. Saraiva, “Gỡ lỗi lỗi kiểu dữ liệu được hỗ trợ bởi Gpt-3: Nghiên cứu việc sử dụng các mô hình ngôn ngữ lớn để sửa chữa mã,” trong Kỷ yếu Hội nghị Quốc tế ACM SIGPLAN lần thứ 16 về Kỹ thuật Ngôn ngữ Phần mềm, SLE 2023, Cascais, Bồ Đào Nha, ngày 23-24 tháng 10 năm 2023, J. Saraiva, T. Dequeule và E. Scott, Biên tập viên. ACM, 2023, trang 111-124. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3623476.3623522> [112] Y. Wu, N. Jiang, HV Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin và S. Shah, “Mạng nơ-ron hiệu quả như thế nào trong việc khắc phục các lỗi hồng bảo mật,” bản in trước arXiv arXiv:2305.18607, 2023.

[113] N. Jiang, K. Liu, T. Lutellier và L. Tan, “Tác động của các mô hình ngôn ngữ mã lên việc sửa chữa chương trình tự động,” bản in trước arXiv arXiv:2302.05020, 2023.

[114] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan và A. Svyatkovskiy, “Inferfix: Sửa chữa chương trình từ đầu đến cuối với llms,” bản in trước arXiv arXiv:2303.07263, 2023.

[115] CS Xia và L. Zhang, “Tiếp tục cuộc trò chuyện: Sửa 162 trong số 337 lỗi với giá 0,42 đô la mỗi lỗi bằng cách sử dụng chatgpt,” bản in trước arXiv arXiv:2304.00385, 2023.

[116] Y. Zhang, G. Li, Z. Jin, và Y. Xing, “Sửa chữa chương trình thần kinh bằng phân tích phụ thuộc chương trình và cơ chế lọc hiệu quả,” bản in trước arXiv arXiv:2305.09315, 2023.

[117] JA Prenner và R. Robbes, “Ngoài ngữ cảnh: Ngữ cảnh cục bộ quan trọng như thế nào trong việc sửa chữa chương trình thần kinh?” 2023.

[118] Q. Zhang, C. Fang, B. Yu, W. Sun, T. Zhang và Z. Chen, “Sửa chữa lỗi hồng phần mềm tự động dựa trên mô hình được đào tạo trước: Chúng ta đã tiến xa đến đâu?” CoRR, tập. abs/2308.12533, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2308.12533>

[119] S. Garg, RZ Moghaddam và N. Sundaresan, “Rapgen: Một cách tiếp cận để khắc phục sự thiếu hiệu quả của mã trong zero-shot,”

CoRR, tập abs/2306.17077, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2306.17077> [120] W. Wang, Y. Wang, S. Joty

và SCH Hoi, “Rap-gen: Tạo bản và được tăng cường truy xuất với codet5 để sửa chữa chương trình tự động,” trong Kỳ yếu Hội nghị Kỹ thuật Phần mềm Châu Âu chung lần thứ 31 của ACM và Hội thảo về Nền tảng Kỹ thuật Phần mềm, ESEC/FSE 2023, San Francisco, CA, Hoa Kỳ, ngày 3-9 tháng 12 năm 2023, S. Chandra, K. Blincoe và P. Tonella, Biên tập viên. ACM, 2023, trang 146-158. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3611643.3616256> [121] Y. Zhang, Z. Jin, Y. Xing và G. Li, “STEAM: mô phỏng hành vi tương tác của lập trình viên để sửa lỗi tự động,” CoRR, tập. abs/2308.14460, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2308.14460> [122] S. Fakhoury, S.

Chakraborty, M. Musuvathi và SK Lahiri, “Hướng tới việc tạo ra các bản chỉnh sửa mã chính xác về mặt chức năng từ các mô tả vấn đề bằng ngôn ngữ tự nhiên,” bản in trước arXiv arXiv:2304.03816, 2023.

[123] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen và D. Phung, “Vulrepair: sửa chữa lỗ hổng phần mềm tự động dựa trên t5,” trong Kỳ yếu Hội nghị Kỹ thuật Phần mềm Châu Âu chung lần thứ 30 của ACM và Hội thảo về Nền tảng Kỹ thuật Phần mềm, 2022, trang 935-947.

[124] S. Gao, X. Wen, C. Gao, W. Wang, H. Zhang và MR Lyu, “Điều gì tạo nên các bản trình diễn ngữ cảnh tốt cho các nhiệm vụ trí tuệ mã với llms?” trong Hội nghị Quốc tế IEEE/ACM lần thứ 38 về Kỹ thuật Phần mềm Tự động, ASE 2023, Luxembourg, ngày 11-15 tháng 9 năm 2023. IEEE, 2023 , trang 761-773. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/ASE56229.2023.00109>

[125] C. Treude và H. Hata, “Có ấy đưa ra yêu cầu và anh ấy kiểm tra: Thiên kiến giới tính trong kỹ thuật phần mềm ở các mô hình ngôn ngữ lớn,” CoRR, tập. abs/2303.10131, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2303.10131> [126] R. Kocielnik, S.

Prabhumoye, V. Zhang, RM Alvarez và A. Anandkumar, “Autobiastest: Tạo câu có thể kiểm soát để kiểm tra thiên kiến xã hội tự động và mở trong các mô hình ngôn ngữ,” CoRR, tập. abs/2302.07371, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2302.07371> [127] M. Ciniselli, L. Pascarella và G. Bavota, “Các công cụ đề xuất mã dựa trên học sâu tạo ra dự đoán bằng cách sao chép mã từ tập huấn luyện ở mức độ nào?” trong Hội nghị Quốc tế IEEE/ACM lần thứ 19 về Khai thác Kho lưu trữ Phần mềm, MSR 2022, Pittsburgh, PA, Hoa Kỳ, ngày 23-24 tháng 5 năm 2022. ACM, 2022, trang 167-178.

[Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3524842.3528440> [128]

D. Erhabor, S. Udayashankar, M. Nagappan và S. Al-Kiswany, “Đo lường hiệu suất thời gian chạy của mã được tạo bằng github copilot,” CoRR, tập. abs/2305.06439, 2023. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.48550/arXiv.2305.06439>

[129] R. Wang, R. Cheng, D. Ford, và T. Zimmermann, “Nghiên cứu và thiết kế để tạo niềm tin trong các công cụ tạo mã được hỗ trợ bởi AI,” CoRR, tập. abs/2305.11248, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2305.11248> [130] B. Yetistiren, I. Ozsoy, M. Ayerdem, và E. T. uz- un, “Đánh giá chất lượng mã của các công cụ tạo mã được hỗ trợ bởi AI: Một nghiên cứu thực nghiệm về github copilot, amazon codewhisperer và chatgpt,” CoRR, tập. abs/2304.10778, 2023. [Trực tuyến]. Có sẵn: <https://doi.org/10.48550/arXiv.2304.10778> [131] C. Wohlin, “Hướng dẫn về phương pháp snowballing trong các nghiên cứu tài liệu có hệ thống và một bản sao trong kỹ thuật phần mềm,” trong Hội nghị Quốc tế lần thứ 18 về Đánh giá và Thẩm định trong Kỹ thuật Phần mềm, EASE '14, London, Anh, Vương quốc Anh, ngày 13-14 tháng 5 năm 2014, MJ Shepperd, T. Hall và I. Myrtveit, Biên tập viên. ACM, 2014, trang 38:1-38:10. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/2601248.2601268> [132] A. Mastropalo, S. Scalabrino, N. Cooper, D. Nader-Palacio, D. Poshyvanyk, R. Oliveto và G. Bavota, “Nghiên cứu việc sử dụng bộ chuyển đổi chuyển đổi văn bản sang văn bản để hỗ trợ các tác vụ liên quan đến mã,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 43 về Kỹ thuật Phần mềm, ICSE 2021, Madrid, Tây Ban Nha, 22-30 tháng 5 năm 2021. IEEE, 2021, trang 336-347.

..

[133] C. Tsigkanos, P. Rani, S. Muller, và T. Kehrer, “Các mô hình ngôn ngữ lớn: Biên giới tiếp theo cho việc khám phá biến trong quá trình kiểm thử biến hình?” trong Hội nghị quốc tế IEEE về Phân tích, Tiến hóa và Tái cấu trúc Phần mềm, SANER 2023, Taipa, Ma Cao, ngày 21-24 tháng 3 năm 2023, T. Zhang, X. Xia, và N. Novielli, Eds. IEEE, 2023, trang 678-682. [Trực tuyến]. Có sẵn: <https://doi.org/10.1109/SANER56733.2023.00070> [134] GJ Myers, Nghề thuật kiểm thử phần mềm (ấn bản thứ 2). Wiley, 2004. [Trực tuyến]. Có sẵn: <http://eu.wiley.com/WileyCDA/WileyTitle/productCd-0471469122.html> [135] P. Farrell-Vinay, Quản lý kiểm thử phần mềm. Auerbach Publ., 2008.

[136] A. Milil và F. Tchier, Kiểm thử phần mềm: Khái niệm và hoạt động. Nhà xuất bản John Wiley & Sons, năm 2015.

[137] S. Lukasczyk và G. Fraser, “Pynguin: Tự động tạo kiểm thử đơn vị cho Python,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 44 về Kỹ thuật Phần mềm: Kỳ yếu kèm theo, ICSE Companion 2022, Pittsburgh, PA, Hoa Kỳ, ngày 22-24 tháng 5 năm 2022. ACM/IEEE, 2022, trang 168-172. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3510454.3516829> [138] ET Barr, M. Harman, P. McMinn, M. Shahbaz và S. Yoo, “Vấn đề oracle trong kiểm thử phần mềm: Một khảo sát,” Giao dịch IEEE về kỹ thuật phần mềm, tập 41, số 5, trang 507-525, 2014.

[139] C. Watson, M. Tufano, K. Moran, G. Bavota và D. Poshyvanyk, “Về việc học các câu lệnh khẳng định có ý nghĩa cho các trường hợp kiểm thử đơn vị,” trong ICSE '20: Hội nghị quốc tế lần thứ 42 về Kỹ thuật phần mềm, Seoul, Hàn Quốc, 27 tháng 6 - 19 tháng 7 năm 2020, G. Rothermel và D. Bae, Biên tập viên. ACM, 2020, trang 1398-1409.

[140] Y. He, L. Zhang, Z. Yang, Y. Cao, K. Lian, S. Li, W. Yang, Z. Zhang, M. Yang, Y. Zhang và H. Duan, “Textexeriscer: Luyện tập nhập văn bản dựa trên phản hồi cho các ứng dụng Android,” trong Hội nghị chuyên đề IEEE về An ninh và Quyền riêng tư năm 2020, SP 2020, San Francisco, CA, Hoa Kỳ, ngày 18-21 tháng 5 năm 2020. IEEE, 2020, trang 1071-1087.

[141] A. Wei, Y. Deng, C. Yang và L. Zhang, “Bữa trưa miễn phí cho việc kiểm thử: Làm mờ các thư viện học sâu từ nguồn mở,” trong Hội nghị quốc tế IEEE/ACM lần thứ 44 về Kỹ thuật phần mềm, ICSE 2022, Pittsburgh, PA, Hoa Kỳ, ngày 25-27 tháng 5 năm 2022. ACM, 2022, trang 995-1007.

[142] D. Xie, Y. Li, M. Kim, HV Pham, L. Tan, X. Zhang và MW Godfrey, “Doctor: Kiểm thử mở dựa trên tài liệu để kiểm thử các hàm API học sâu,” trong ISSTA '22: Hội nghị chuyên đề quốc tế ACM SIGSOFT lần thứ 31 về Kiểm thử và Phân tích Phần mềm, Sự kiện trực tuyến, Hàn Quốc, ngày 18 - 22 tháng 7 năm 2022, S. Ryu và Y. Smaragdakis, Biên tập viên. ACM, 2022, trang 176-188.

[143] Q. Guo, X. Xie, Y. Li, X. Zhang, Y. Liu, X. Li và C. Shen, “Audee: Kiểm thử tự động cho các khung học sâu,” trong Hội nghị quốc tế IEEE/ACM lần thứ 35 về Kỹ thuật phần mềm tự động, ASE 2020, Melbourne, Úc, ngày 21-25 tháng 9 năm 2020. IEEE, 2020, trang 486-498.

[144] Z. Wang, M. Yan, J. Chen, S. Liu và D. Zhang, “Kiểm thử thư viện học sâu thông qua việc tạo mô hình hiệu quả,” trong ESEC/FSE '20: Hội nghị và Hội thảo chung về Kỹ thuật phần mềm châu Âu lần thứ 28 của ACM về Nền tảng của Kỹ thuật phần mềm, Sự kiện ảo, Hoa Kỳ, ngày 8-13 tháng 11 năm 2020, P. Devanbu, MB Cohen và T. Zimmermann, Biên tập viên. ACM, 2020, trang 788-799.

[145] J. Jiang, Y. Xiong, H. Zhang, Q. Gao và X. Chen, “Định hình không gian sửa chữa chương trình bằng các bản vá hiện có và mã tương tự,” trong Kỳ yếu Hội nghị chuyên đề quốc tế ACM SIGSOFT lần thứ 27 về Kiểm thử và Phân tích Phần mềm, ser. ISSTA 2018. New York, NY, Hoa Kỳ: Hiệp hội Máy tính, 2018, tr. 298-309. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3213846.3213871> [146] M. Wen, J. Chen, R. Wu, D. Hao và S.-C. Chung, “Tạo bản và nhận biết ngữ cảnh để sửa chữa chương trình tự động tốt hơn,” trong Kỳ yếu Hội nghị Quốc tế lần thứ 40 về Kỹ thuật Phần mềm, ser. ICSE '18. New York, NY, Hoa Kỳ: Hiệp hội Máy tính, 2018, trang 1-11. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3180155.3180233> [147] Y. Xiong, J. Wang, R. Yan, J. Zhang, S. Han, G. Huang và L. Zhang, “Tổng hợp điều kiện chính xác để sửa chữa chương trình,” trong Hội nghị quốc tế IEEE/ACM lần thứ 39 về Kỹ thuật phần mềm (ICSE), 2017, trang 416-426.

[148] J. Xuan, M. Martinez, F. DeMarco, M. Clement, SL Marcote, T. Durieux, D. Le Berre và M. Monperrus, “Nopol: Tự động sửa lỗi câu lệnh điều kiện trong chương trình Java,” Giao dịch IEEE về Kỹ thuật Phần mềm, tập 43, số 1, trang 34-55, 2017.

[149] S. Song, X. Li và S. Li, “Làm thế nào để thu hẹp khoảng cách giữa các phương thức: Một khảo sát toàn diện về mô hình ngôn ngữ lớn đa phương thức,” CoRR, tập. abs/2311.07594, 2023.

[150] JM Zhang, M. Harman, L. Ma và Y. Liu, “Kiểm thử máy học: Khảo sát, bối cảnh và triển vọng,” IEEE Trans. Software Eng., tập 48, số 2, trang 1-36, 2022.

[151] F. Tu, J. Zhu, Q. Zheng và M. Zhou, “Hãy cẩn thận về thời điểm: một nghiên cứu thực nghiệm về việc lạm dụng theo dõi vấn đề liên quan đến thời gian

dữ liệu,” trong Kỷ yếu Hội nghị chung ACM năm 2018 về Kỹ thuật phần mềm châu Âu và Hội thảo về Nền tảng của Kỹ thuật phần mềm, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, Hoa Kỳ, ngày 04-09 tháng 11 năm 2018, GT Leavens, A. Garcia và CS Pasareanu, Biên tập viên. ACM, 2018, trang 307-318. [Trực tuyến].
Có sẵn: <https://doi.org/10.1145/3236024.3236054> [152] Z. Sun, L. Li, Y. Liu, X. Du và L. Li, “Về tầm quan trọng của việc xây dựng bộ dữ liệu huấn luyện chất lượng cao cho tìm kiếm mã thần kinh,” trong Hội nghị quốc tế IEEE/ACM lần thứ 44 về Kỹ thuật phần mềm, ICSE 2022, Pittsburgh, PA, Hoa Kỳ, ngày 25-27 tháng 5 năm 2022. ACM, 2022, trang 1609-1620. [Trực tuyến]. Có sẵn: <https://doi.org/10.1145/3510003.3510160> [153] L. Shi, Z. Jiang, Y. Yang, X. Chen, Y. Zhang, F. Mu, H. Jiang và Q. Wang, “ISPY: trích xuất cặp vấn đề-giải pháp tự động từ các cuộc trò chuyện trực tiếp cộng đồng,” trong Hội nghị Quốc tế IEEE/ACM lần thứ 36 về Kỹ thuật Phần mềm Tự động, ASE 2021, Melbourne, Úc, ngày 15-19 tháng 11 năm 2021. IEEE, 2021, trang 142-154. [Trực tuyến].
Có sẵn: <https://doi.org/10.1109/ASE51524.2021.9678894> [154] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, SK Deng, CB Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang và M. Zhou, “Graphcodebert: Huấn luyện trước các biểu diễn mã với luồng dữ liệu,” trong Hội nghị Quốc tế lần thứ 9 về Biểu diễn Học tập, ICLR 2021, Sự kiện ảo, Áo, ngày 3-7 tháng 5 năm 2021. OpenReview.net, 2021. [Trực tuyến]. Có sẵn: <https://openreview.net/forum?id=jLoC4ez43PZ> [155] F. Yu, A. Seff, Y. Zhang, S. Song, T. Funkhouser và J. Xiao, “Lsun: Xây dựng bộ dữ liệu hình ảnh quy mô lớn bằng cách sử dụng học sâu với sự tham gia của con người,” bản in trước arXiv arXiv:1506.03365, 2015.
[156] LoadRunner, Inc., “Loadrunner,” 2023, microfocus.com.
[157] LangChain, Inc., “Langchain,” 2023, <https://docs.langchain.com/docs/>.
[158] Kỹ thuật nhắc nhở, “Hướng dẫn kỹ thuật nhắc nhở,” 2023, <https://github.com/dair-ai/Prompt-Engineering-Guide>.
[159] Z. Zhang, A. Zhang, M. Li, H. Zhao, G. Karypis và A. Smola, “Suy luận chuỗi tư duy đa phương thức trong các mô hình ngôn ngữ,” CoRR, tập abs/2302.00923, 2023.
[160] Z. Liu, X. Yu, Y. Fang và X. Zhang, “Graphprompt: Thống nhất các tác vụ huấn luyện trước và tác vụ tiếp theo cho mạng nơ-ron đồ thị,” trong Kỷ yếu Hội nghị Web ACM 2023, WWW 2023, Austin, TX, Hoa Kỳ, 30 tháng 4 năm 2023 - 4 tháng 5 năm 2023, Y. Ding, J. Tang, JF Sequeda, L. Aroyo, C. Castillo và G. Houben, Biên tập viên. ACM, 2023, trang 417- 428.
[161] Y. Charalambous, N. Tihanyi, R. Jain, Y. Sun, MA Ferrag và LC Cordeiro, “Một kỷ nguyên mới trong bảo mật phần mềm: Hướng tới phần mềm tự phục hồi thông qua các mô hình ngôn ngữ lớn và xác minh hình thức,” 2023.
[162] S. Wang, L. Huang, A. Gao, J. Ge, T. Zhang, H. Feng, I. Satyarth, M. Li, H. Zhang và V. Ng, “Máy học/học sâu cho kỹ thuật phần mềm: Đánh giá tài liệu có hệ thống,” IEEE Trans. Software Eng., tập 49, số 3, trang 1188-1231, 2023. [Trực tuyến].
Có sẵn tại: <https://doi.org/10.1109/TSE.2022.3173346>
[163] Y. Yang, X. Xia, D. Lo và JC Grundy, “Khảo sát về học sâu cho kỹ thuật phần mềm,” ACM Comput. Surv., tập 54, số 10s, trang 206:1-206:73, 2022. [Trực tuyến]. Có sẵn tại: <https://doi.org/10.1145/3505243>
[164] C. Watson, N. Cooper, D. Nader-Palacio, K. Moran và D. Poshyvanyk, “Đánh giá tài liệu có hệ thống về việc sử dụng học sâu trong nghiên cứu kỹ thuật phần mềm,” ACM Trans. Phương pháp kỹ thuật phần mềm, tập 31, số 2, trang 32:1-32:58, 2022. [Trực tuyến].
Có sẵn: <https://doi.org/10.1145/3485275> [165] M. Bajammal, A. Stocco, D. Mazinanian và A. Mesbah, “Một khảo sát về việc sử dụng thị giác máy tính để cải thiện các nhiệm vụ kỹ thuật phần mềm,” IEEE Trans. Software Eng., tập 48, số 5, trang 1722-1742, 2022. [Trực tuyến].
Có sẵn: <https://doi.org/10.1109/TSE.2020.3032986> [166] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, JC Grundy và H. Wang, “Các mô hình ngôn ngữ lớn cho kỹ thuật phần mềm: Một đánh giá tài liệu có hệ thống,” CoRR, tập abs/2308.10620, 2023. [Trực tuyến].
Có sẵn: <https://doi.org/10.48550/arXiv.2308.10620> [167] A. Fan, B. Gokkaya, M. Hazman, M. Lyubarskiy, S. Sengupta, S. Yoo và JM Zhang, “Các mô hình ngôn ngữ lớn cho kỹ thuật phần mềm: Khảo sát và các vấn đề mở,” CoRR, tập abs/2310.03533, 2023. [Trực tuyến]. Có sẵn tại : <https://doi.org/10.48550/arXiv.2310.03533>

[illegible]